109 Python Problems for CCPS 109

Ilkka Kokkarinen

Chang School of Continuing Education
Toronto Metropolitan University

Version of October 20, 2026

General requirements

This document contains specifications for the lab problems in the course **CCPS 109 Computer Science I**, developed and taught by Ilkka Kokkarinen for the Chang School of Continuing Education, Toronto Metropolitan University. It contains 109 individual problems to choose your battles from, listed roughly in order of increasing complexity. Fourteen advanced bonus problems are intended for students who want to be challenged and find this problem collection hard to complete. This collection is now complete, with no new problems added and no old problems removed. Both this document and the automated fuzz tester will be updated only to resolve bugs, or to plug a tester loophole that let some solution that was not correct still pass the test.

All these problems are designed to allow solutions **using only the core computational structures introduced during the first five weeks of this course**, assuming that the student has completed enough practice problems at the excellent CodingBat Python interactive training site or acquired the equivalent coding routine from earlier studies. A handful of problems that involve searching are best solved with recursion. Other than the standard library module `fractions`, you don't need any specialized Python libraries. However, **you are allowed to use anything in the Python 3 standard library** that you find useful in constructing your functions.

You must implement all your functions in a single source code file named `labs109.py`. This allows you to run the `tester109.py` script at any time to validate the functions you have completed so far, so you always know exactly where you stand in this course. These tests will be executed in the same order that your functions appear inside the `labs109.py` file.

Your functions may assume that **the arguments given to them are as promised in the problem specification,** so that these functions never have to handle and recover from arguments whose value or type is invalid. However, your functions must correctly handle zero integers and empty lists whenever these can appear as legitimate argument values.

Each function's test should take no more than a couple of seconds to complete on an off-the-shelf desktop from the past couple of years. This is usually achieved by shaving off one level of nesting from the structured loops, occasionally with the aid of a `set` or a `dict` to remember what your function has seen and done so far. The automated tester imposes a twenty-second cutoff for each test and forcefully terminates it after that.

Do not use any floating-point operations in any of your functions. **Every time you write any decimal number literal in your code, you are guaranteed to be doing something wrong.** The automated test cases can and will involve integers large enough to exhaust the fixed precision of floating-point numbers and give you the wrong answer. Even if you don't use any decimal literals, the usual suspects that round up when these problems occur are accidental single-slash division,

`pow`, and `log`. Instead of `pow`, just use either `**` or repeated `*` as appropriate. To find how many digits an integer `n` has, use `len(str(n)) - int(n < 0)`.

Silence is golden. **None of your functions should print anything to the console;** they should return the expected result silently. You can include some debugging output during development, but remember to comment out or remove all such printing statements before submission.

This specification document and the automated tester are released under GNU Public License version 3, so they can be adapted and distributed to all teachers and students of computer science. The author compiled these problems over time from a gallimaufry of sources ranging from the original lab and exam problems of his old Java version of this course to a multitude of programming puzzle and coding challenge sites such as LeetCode, CodeAbbey, Stack Exchange Code Golf, and Wolfram Programming Challenges. Martin Gardner's classic recreational mathematics (yes, that is a real thing), and his spiritual successors, have also inspired many problems here.

The author has tried to dodge not just the Scylla of well-worn problems you can find in basically every textbook and online problem collection, but also the Charybdis of pointless make-work drudgery with no inherent meaning or purpose. Some of these seemingly simple problems also touch the deeper issues of computer science that you will encounter in your later undergraduate and graduate courses. Occasionally, these problems relate to entirely separate fields of human life and endeavour, giving them nontrivial philosophical and practical worldview implications.

This problem set also has an official associated subreddit r/ccps109 for students to discuss the individual problems and associated issues. This discussion should take place at the level of ideas, so that no solution code should be posted or requested. Any issues with course management and the course schedule in any particular semester should be kept out of this subreddit to maintain a more permanent, fundamental nature. Furthermore, no student should at any time be stuck with any one problem. Once you embark on solving these problems, you should read ahead to find at least five problems that interest you. Keep them simmering on the back burner as you go about your normal days. At the actual keyboard, work on the problem where you feel closest to a working solution.

The author wishes to thank past students **Shu Zhu Su**, **Rimma Konoval**, **Mohammed Waqas**, **Zhouqin He** and **Karel Tutsu** for going above and beyond the call of duty in solving these problems in ways that either revealed errors in the original problem specifications or the instructor's private model solutions, or, more fortunately, independently agreed with the instructor's private model solution and, by this agreement, raised the instructor's confidence in the correctness of both. In their wake comes the Green Checkmark Gang, a legion of individual students who pointed out ambiguities, errors, and inconsistencies in particular problems and, in doing so, effectively acted together as a giant Zamboni, planing the ice so future students could skate a little easier. Members of that wild horde, you all know who you are. All remaining errors, ambiguities and inconsistencies, whether aesthetic, logical or extracurricular, remain the sole responsibility of Ilkka Kokkarinen. Report any such errors promptly to `ilkka.kokkarinen@gmail.com`.

List of Problems

1. Ryerson letter grade 9
2. Ascending list 10
3. Riffle shuffle kerfuffle 11
4. Even the odds 12
5. Cyclops numbers 13
6. Domino cycle 14
7. Colour trio 15
8. Count dominators 16
9. Beat the previous 17
10. Subsequent words 18
11. Taxi ℤum ℤum 19
12. Exact change only 20
13. Rooks on a rampage 21
14. Words with given shape 22
15. Chirality 23
16. The card that wins the trick 24
17. Do you reach many, do you reach one? 25
18. Sevens rule, zeros drool 26
19. Fulcrum 27
20. Fail while daring greatly 28
21. All your bases are belong to us 29
22. Count the balls off the brass monkey 30
23. Count growlers 31

24. Bulgarian solitaire 32
25. Scylla or Charybdis? 33
26. Longest arithmetic progression 34
27. Best one out of three 35
28. Collecting numbers 36
29. Between the soft and the NP–hard place 37
30. Count Troikanoff, I presume 38
31. Crack the crag 39
32. Three summers ago 40
33. Sum of two squares 41
34. Carry on Pythonista 42
35. As below, so above 43
36. Expand positive integer intervals 44
37. Collapse positive integer intervals 45
38. Prominently featured 46
39. Like a kid in a candy store, except without money 47
40. Dibs to dubs 48
41. Nearest smaller element 49
42. Iterated ordinal transform 50
43. Interesting, intersecting 51
44. So shall you sow 52
45. That's enough of you! 53
46. Brussel's choice 54
47. Cornered cases 55
48. McCulloch's second machine 56
49. That's enough for you! 57

50. Crab bucket list 58
51. What do you hear, what do you say? 59
52. Bishops on a binge 60
53. Dem's some mighty tall words, pardner 61
54. Up for the count 62
55. Revorse the vewels 63
56. Everybody on the floor, do the Scrooge Shuffle 64
57. Rational lines of action 65
58. Verbos regulares 66
59. Hippity hoppity, abolish loopity 67
60. In space, no one can hear you bounce 68
61. Nearest polygonal number 69
62. Don't worry, we will fix it in the post 70
63. Fractran interpreter 71
64. Permutation cycles 72
65. Whoever must play, cannot play 73
66. ztalloc ecneuqes 74
67. The solution solution 75
68. Reverse ascending sublists 76
69. Brangelin–o–matic for the people 77
70. Line with the most points 78
71. Om nom nom 79
72. Autocorrect for sausage fingers 80
73. Uambcsrlne the wrod 81
74. Substitution words 82
75. Manhattan skyline 83

76. Count overlapping disks 84
77. Ordinary cardinals 85
78. Count divisibles in range 86
79. Bridge hand shape 87
80. Milton Work point count 88
81. Never the twain shall meet 89
82. Bridge hand shorthand form 90
83. Points, schmoints 91
84. Bulls and cows 92
85. Frequency sort 93
86. Calling all units, B-and-E in progress 94
87. Lunatic multiplication 95
88. Distribution of abstract bridge hand shapes 96
89. Fibonacci sum 97
90. Wythoff array 98
91. Rooks with friends 99
92. Possible words in Hangman 100
93. All branches lead to Rome 101
94. Be there or be square 102
95. Flip of time 103
96. Bulgarian cycle 104
97. Spirit of the staircase 105
98. Square it out amongst yourselves 106
99. Sum of distinct cubes 107
100. Count maximal layers 108
101. Maximum checkers capture 109

102. Collatzy distance 110
103. Van Eck sequence 111
104. Tips and trips 112
105. Balanced ternary 113
106. Lords of Midnight 114
107. Optimal crag score 115
108. Painted into a corner 116
109. Operation Grand Slam: Infinite Fibonacci word 117
Bonus problem 110: Reverse the Rule 110 118
Bonus problem 111: Aye, eye, I 119
Bonus problem 112: Count domino tilings 120
Bonus problem 113: Invaders must die 121
Bonus problem 114: Stepping stones 122
Bonus problem 115: Ex iudiciis, lux 123
Bonus problem 116: Flatland golf 124
Bonus problem 117: TextmirrororrimtxeT 125
Bonus problem 118: Hy-phen-a-tion by com-put-er 126
Bonus problem 119: Jealous vs. zealous 127
Bonus problem 120: Count the permorsetations 128
Bonus problem 121: Count your lucky sevens 129
Bonus problem 122: Forks on the Gin Lane 130
Bonus problem 123: One, two, three, grow old with me 131

1. Ryerson letter grade

`def ryerson_letter_grade(pct):`

Before the name change back in 2022, Toronto Metropolitan University was known as Ryerson University. Given the percentage grade, calculate and return the letter grade that would appear in the Ryerson grades transcript, as defined on the page Ryerson Grade Scales. This letter grade should be returned as a string that consists of the uppercase letter followed by the modifier `'+'` or `'-'`, if there is one. This function should work correctly for all values of `pct` from 0 to 150.

Like all other programming problems that follow, you can solve this in various ways. The simplest way to solve this problem would probably be to use an **if-else ladder**. The file `labs109.py` in the repository `ikokkari/PythonProblems` already contains an implementation of this function, so you can run the tester script `tester109.py` to verify that everything is hunky dory.

`pct`	Expected result
`45`	`'F'`
`62`	`'C-'`
`89`	`'A'`
`107`	`'A+'`

As you learn more Python and techniques to make it dance for you, you may think up other ways to solve this problem. Some of these would suit productive tasks in a professional coding environment, while others are meant to be taken in jest as conceptual performance art. A popular genre of recreational puzzles in all programming languages is solving a straightforward problem with an algorithmic equivalent of a needlessly complicated Rube Goldberg machine, to demonstrate the universality and unity of computation.

2. Ascending list

`def is_ascending(items):`

Determine whether the sequence of `items` is **strictly ascending** so that each element is **strictly larger** (not just merely **equal to**) than the element that precedes it. Return `True` if the elements in the list of `items` are strictly ascending, and return `False` otherwise.

Note that the empty sequence is ascending, as is also every one-element sequence, so be careful to ensure that your function returns the correct answers in these seemingly insignificant **edge cases** of this problem. (If these sequences were not ascending, pray tell, what would be the two elements that violate the requirement and make that particular sequence not be ascending?)

`items`	Expected result
`[]`	`True`
`[-5, 10, 99, 123456]`	`True`
`[2, 3, 3, 4, 5]`	`False`
`[-99]`	`True`
`[4, 5, 6, 7, 3, 7, 9]`	`False`
`[1, 1, 1, 1]`	`False`

In the same spirit, note how every possible universal claim made about the elements of an empty sequence is trivially true! For example, if `items` is the empty sequence, the two claims "All elements of `items` are odd" and "All elements of `items` are even" are both equally true, as is also the claim "All elements of `items` are colourless green ideas that sleep furiously."

3. Riffle shuffle kerfuffle

`def riffle(items, out=True):`

Given a list of `items` whose length is guaranteed to be even (note that "oddly" enough, zero is an even number), create and return a list produced by performing a perfect **riffle** on the `items` by interleaving the items of the two halves of the list in an alternating fashion.

When performing a perfect riffle shuffle, also known as the Faro shuffle, split the list into two equal-sized halves, either conceptually or in actuality. The first two elements of the result are then the first elements of those halves. The next two elements of the result are the second elements of those halves, followed by the third elements, and so on up to the last elements. The parameter `out` determines whether this function performs an out shuffle or an in shuffle, which determines which half of the deck the alternating card is first taken from.

`items`	`out`	Expected result
`[1, 2, 3, 4, 5, 6, 7, 8]`	`True`	`[1, 5, 2, 6, 3, 7, 4, 8]`
`[1, 2, 3, 4, 5, 6, 7, 8]`	`False`	`[5, 1, 6, 2, 7, 3, 8, 4]`
`[]`	`True`	`[]`
`['bob', 'jack']`	`True`	`['bob', 'jack']`
`['bob', 'jack']`	`False`	`['jack', 'bob']`

4. Even the odds

`def only_odd_digits(n):`

Check that the given positive integer `n` contains only odd digits (1, 3, 5, 7 and 9) when it is written out. Return `True` if this is the case, and `False` otherwise. Note that this question is not asking whether the number `n` itself is odd or even. You therefore must check every digit of the given number before you can say it contains no even digits.

To extract the lowest digit of a positive integer `n`, use the expression `n%10`. To chop off the lowest digit and keep the rest of the digits, use the expression `n//10`. Or, if you don't want to be this fancy, you can first convert the number into a string and whack it from there with string operations.

`n`	Expected result
`8`	`False`
`1357975313579`	`True`
`42`	`False`
`71358`	`False`
`0`	`False`

5. Cyclops numbers

```
def is_cyclops(n):
```

A nonnegative integer is said to be a **cyclops number** if it consists of an **odd number of digits** so that the middle (more poetically, the "eye") digit is a zero, and all the other digits of that number are non-zero. This function should determine whether its parameter integer `n` is a cyclops number and return `True` or `False` accordingly.

`n`	Expected result
`0`	`True`
`101`	`True`
`98053`	`True`
`777888999`	`False`
`1056`	`False`
`675409820`	`False`

As an extra challenge, you can try to solve this problem using only loops, conditions and integer arithmetic operations, without first converting the integer into a string and working from there. Dividing any integer by 10 using integer division `//` effectively chops off its last digit, whereas taking the remainder when dividing by 10 using the `%` operator extracts that last digit. These operations let us iterate through an integer's digits one at a time, from lowest to highest, as if the integer were a sequence of digits.

Even better, this operation doesn't work only for the familiar base ten; it works for any base by using that base as the divisor. Especially using two as the divisor instead of ten allows you to iterate through the **bits** of the **binary representation** of any integer, which will come in handy in problems in your later courses that expect you to be able to manipulate these individual bits. (In practice, these division and remainder operations are often further condensed into equivalent but faster **bit shifts** and the **bitwise and** operations.)

6. Domino cycle

```
def domino_cycle(tiles):
```

A single **domino tile** is represented as a two-tuple of its **pip values**, such as `(2,5)` or `(6,6)`. This function should determine whether the given list of `tiles` forms a **cycle** so that each tile in the list ends with the same pip value that its successor tile starts with, the successor of the last tile being the first tile of the list, since this is supposed to be a cycle instead of a chain. Return `True` if the given list of `tiles` forms such a cycle, and `False` otherwise.

`tiles`	Expected result
`[(3, 5), (5, 2), (2, 3)]`	`True`
`[(4, 4)]`	`True`
`[]`	`True`
`[(2, 6)]`	`False`
`[(5, 2), (2, 3), (4, 5)]`	`False`
`[(4, 3), (3, 1)]`	`False`

7. Colour trio

```
def colour_trio(colours):
```

This problem was inspired by the Mathologer video "Secret of Row 10". To start, assume the existence of three values called "red", "yellow" and "blue". These names serve as colourful (heh) mnemonics and could as well have been 0, 1, and 2, or "foo", "bar" and "baz"; no connection to actual physical colours is implied. Next, define a rule to mix such colours so that mixing any colour with itself gives that same colour, whereas mixing any two different colours always gives the third colour. For example, mixing blue with blue gives blue, whereas mixing blue with yellow gives red, the same as mixing yellow with blue or red with red.

Given the first row of `colours` as a string of lowercase letters, this function should construct the rows below the first row one at a time according to the following discipline. Each row is one element shorter than the previous row. The *i*:th element of each row comes from mixing the colours in positions *i* and *i* + 1 of the previous row. Rinse and repeat until only the singleton element of the bottom row remains, returned as the final answer. For example, starting from `'rybyr'` leads to `'brrb'`, which leads to `'yry'`, which leads to `'bb'`, which leads to `'b'` for the final answer, Regis. When the Python virtual machine laughs and goes `'brrrrr'`, that will lead to `'yrrrr'`, `'brrr'`, `'yrr'`, and `'br'` for the final answer `'y'` for "Yes, please!"

`colours`	Expected result
`'y'`	`'y'`
`'bb'`	`'b'`
`'rybyry'`	`'r'`
`'brybbr'`	`'r'`
`'rbyryrrbyrbb'`	`'y'`
`'yrbbbbryyrybb'`	`'b'`

(Today's five-dollar power word to astonish your friends and coworkers is "quasigroup".)

8. Count dominators

```
def count_dominators(items):
```

An element in a list of `items` is a **dominator** if every element to its right (not just the one element immediately to its right, but all of them) is strictly smaller than that element. Note that, under this definition, the last item in the list is always a dominator, regardless of its value. This function should return the number of elements in the `items` that are dominators. For example, the dominators of the list `[42, 7, 12, 9, 13, 5]` would be 42, 13 and 5. Regardless of its value, the last element of the list is trivially a dominator, since nothing greater follows it in the list.

Before starting to write code for this function, you should consult the parable of "Shlemiel the painter" and think about how this seemingly silly tale from a simpler time relates to today's computational problems performed on lists, strings and other sequences. This problem will be the first of many that you will encounter during and after this course to illustrate the important principle of using only one loop to achieve in a tiny fraction of time the same result that Shlemiel achieves with two nested loops. Your workload increases only **linearly** with the number of items, whereas Shlemiel's total time grows **quadratically**, as a function of the **square** of the number of items.

Trying to hide the inner loop of some Shlemiel algorithm inside a function call (this includes Python built-ins such as `max` and list slicing) and pretending that this somehow makes those inner loops take a constant time will only summon the Gods of Compubook Headings to return with tumult to claim their lion's share of execution time.

`items`	Expected result
`[42, 7, 12, 9, 2, 5]`	`4`
`[]`	`0`
`[-2, 5, -1, -3]`	`3`
`[-10, -20, -30, -42]`	`4`
`[42, 42, 42, 42]`	`1`
`range(10**7)`	`1`
`range(10**7, 0, -1)`	`10000000`

9. Beat the previous

```
def extract_increasing(digits):
```

Given a string of digits guaranteed only to contain ordinary integer digit characters 0 to 9, create and return the list of increasing integers acquired from reading these digits in order from left to right. The first integer in the result list is made up of the first digit of the string. After that, each element is an integer made of as many consecutive digits as needed to make it **strictly larger** than the previous integer. Any leftover digits at the end of the digit string that do not form a sufficiently large integer are ignored.

This problem can be solved with a for-loop over the `digits`, examining each digit exactly once, regardless of its position at the beginning, end, or middle of the string. Keep track of the `current` number (initially zero) and the `previous` number to beat (initially equal to minus one). Each digit `d` is then processed by appending it to the end of the `current` number with the assignment `current=10*current+int(d)`, updating the `result` and `previous` as needed.

`digits`	Expected result
`'600005'`	`[6]`
`'045349'`	`[0, 4, 5, 34]`
`'7777777777777777777'`	`[7, 77, 777, 7777, 77777, 777777]`
`'122333444455555666666'`	`[1, 2, 23, 33, 44, 445, 555, 566, 666]`
`'271828182845904523536028747135266249775724709369995957496696762772407663035354759457138217852516642742746639193200305992181741359662904357290033 4295260'`	`[2, 7, 18, 28, 182, 845, 904, 5235, 36028, 74713, 526624, 977572, 4709369, 9959574, 96696762, 772407663, 3535475945, 7138217852, 51664274274, 66391932003, 599218174135, 966290435729]`
`'1234' * 100`	A list with 38 elements, the last one equal to `341234123412341234123`

10. Subsequent words

```
def words_with_letters(words, letters):
```

This problem introduces general discrete math terminology that will make many later problem specifications less convoluted and ambiguous. A **substring** of a string consists of characters taken **in order** from consecutive positions. Contrast this with the similar concept of a **subsequence** of characters still taken in order, but not necessarily at consecutive positions. For example, each of the five strings `''`, `'e'`, `'put'`, `'ompu'` and `'computer'` is both a substring and subsequence of the string `'computer'`, whereas `'cper'` and `'out'` are subsequences, but not substrings.

Note how the empty string is always a substring of every possible string, including itself. Every string is always its own substring, although not a **proper substring**, unlike all other substrings. Concepts of **sublist** and **subsequence** are defined for lists in an analogous manner. Since **sets** have no internal order on top of the element membership in that set, sets can meaningfully have both proper and improper subsets. (The concept of "subsetquence" might mean a subset that would be a subsequence, were the members of that set written out sequentially in sorted order. But I made that term up *ad hoc* for its comical value.)

Anyway, now that you know all that, given a list of `words` sorted in alphabetical order, and a string of required `letters`, find and return the list of words that contain `letters` as a sub*sequence*.

`letters`	Expected result (using the wordlist `words_sorted.txt`)
`'klore'`	`['booklore', 'booklores', 'folklore', 'folklores', 'kaliborite', 'kenlore', 'kiloampere', 'kilocalorie', 'kilocurie', 'kilogramme', 'kilogrammetre', 'kilolitre', 'kilometrage', 'kilometre', 'kilooersted', 'kiloparsec', 'kilostere', 'kiloware']`
`'brohiic'`	`['bronchiectatic', 'bronchiogenic', 'bronchitic', 'ombrophilic', 'timbrophilic']`
`'azaz'`	`['azazel', 'azotetrazole', 'azoxazole', 'diazoaminobenzene', 'hazardize', 'razzmatazz']`

11. Taxi ℤum ℤum

```
def taxi_zum_zum(moves):
```

🎶 *The sole spot of light in the night blinks on the roof of the taxi…* 🎶 that is casually cruising in the street grid of the dusky Manhattan, the city we know and love from classic hard-boiled *film noir* works such as "Blast of Silence". The taxicab starts its journey at the origin `(0,0)` of the infinite two-dimensional integer grid, denoted by $\mathbb{Z}^2$. Fitting the gaunt, angular spirit of the time that tolerates few deviations or gray areas, the taxicab always heads straight along one of the four main axes, initially north.

This taxicab then executes the given sequence of `moves`, given as a string of characters `'L'` for turning 90 degrees left while standing in place (just in case we are making a turn backwards, in case you spotted some glad rags or some out-of-town palooka looking to be taken for a ride), `'R'` for turning 90 degrees right (ditto), and `'F'` for moving one block forward in the current heading. This function should return the final position of the taxicab on this infinitely spanning Manhattan that has no borders. (As someone hypothesized, the rest of the world simulates it with mirrors.)

`moves`	Expected result
`'RFRL'`	`(1, 0)`
`'LFFLF'`	`(-2, -1)`
`'LLFLFLRLFR'`	`(1, 0)`
`'FR' * 1729`	`(0, 1)`
`'FFLLLFRLFLRFRLRRL'`	`(3, 2)`

As an aside, why do these problems always seem to take place in Manhattan and evoke nostalgic visuals of Jackie Mason or that Woodsy Allen fellow with the grumpy immigrant cabbie and various colourful bystander characters, instead of being set in, say, the mile high city of Denver whose street grid is cleverly rotated 45 degrees from the main compass axes to equalize the amount of daily sunlight on streets in both orientations? That ought to make for an interesting variation to many problems of this spirit. Unfortunately, diagonal moves always maintain the total **parity** of the coordinates, which makes it impossible to reach any coordinate pair of opposite parity in this manner, as in that old joke with the punchline "Gee… I don't think that you can get there from here."

12. Exact change only

```
def give_change(amount, coins):
```

Given the `amount` of money, expressed as the total number of kopecks of Latveria, Ruritania, Montenegro, or some other one of those vaguely Slavic fictional countries that Tintin and similar hearty fellows like to visit for a wacky chase to grab the current year's MacGuffin, followed by the list of available denominations of `coins` also expressed as kopecks, this function should create and return a list of coins that add up to the `amount` using the **greedy approach** where you use as many of the highest denomination coins as possible before moving on to the next lower denomination. The list of coin denominations is guaranteed to be given in descending sorted order, as your returned result should also be.

`amount`	`coins`	Expected result
`64`	`[50, 25, 10, 5, 1]`	`[50, 10, 1, 1, 1, 1]`
`123`	`[100, 25, 10, 5, 1]`	`[100, 10, 10, 1, 1, 1]`
`100`	`[42, 17, 11, 6, 1]`	`[42, 42, 11, 1, 1, 1, 1, 1]`

With countless variations, this problem is a computer science classic when modified to minimize the total number of returned coins. The above greedy approach then no longer produces the optimal result for all possible coin denominations. For example, using the simple coin denominations of `[50, 30, 1]` and the amount of sixty kopecks to be exchanged, the greedy solution `[50, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]` ends up using eleven coins due to the unfortunate first choice that prevents it from using any of the 30-kopeck coins. Those 30-kopeck coins sure would come in handy, since the optimal solution `[30, 30]` needs only two such coins! A more advanced **recursive** algorithm examines both sides of the "take it or leave it" decision for each coin and chooses the option that ultimately yields a better outcome. Memoize the intermediate results of this recursion to avoid the exponential blow-up in running time.

13. Rooks on a rampage

```
def safe_squares_rooks(n, rooks):
```

A parliament of rooks has invaded a generalized n-by-n chessboard, with each rook represented as a two-tuple (row, column) indicating the square it occupies. Since we are computer programmers rather than chess players and other normal folks, we number rows and columns from 0 to n – 1. A chess rook covers all squares in its row and column. Given the board size `n` and the list of `rooks` on that board, count the number of empty squares that are safe, that is, are not covered by any rook.

To achieve this within both reasonable time and memory, you should count separately how many rows and columns on the board are safe from any rook. Because **permuting** the rows and columns does not change the answer to this question, you can imagine all these safe rows and columns to have been permuted to form an empty rectangle at the top-left corner of the board. The area of that safe rectangle is then obviously the product of its known width and height.

n	rooks	Expected result
10	[]	100
4	[(2, 3), (0, 1)]	4
8	[(1, 1), (3, 5), (7, 0), (7, 6)]	20
2	[(1, 1)]	1
6	[(r, r) for r in range(6)]	0
100	[(r, (r*(r-1))%100) for r in range(0, 100, 2)]	3900
10**6	[(r, r) for r in range(10**6)]	0

14. Words with given shape

```
def words_with_given_shape(words, shape):
```

The **shape** of the given word of length *n* is a list of *n* – 1 integers, each one either –1, 0 or +1 to indicate whether the next letter following the letter in that position comes later (+1), is the same (0) or comes earlier (–1) in the alphabetical order of English letters. For example, the shape of the word `'hello'` is `[-1, +1, 0, +1]`, whereas the shape of `'world'` is `[-1, +1, -1, -1]`. Find and return a list of all `words` that have that particular `shape`, listed in alphabetical order.

Note that your function, the same as all the other functions specified in this document that operate on lists of words, **should not try to read** the wordlist file `words_sorted.txt`, even though, yes, we agree, Python makes that yawningly easy with a couple of lines of code. The tester script already reads in the entire wordlist and builds the list of words to be given to your function. Your function should use this given list of `words` without even caring which particular file it came from.

`shape`	Expected result (using wordlist `words_sorted.txt`)
`[1, 0, -1, 0]`	`['cooee', 'essee']`
`[1, -1, -1, -1, 0, -1]`	`['congeed', 'nutseed', 'outfeed', 'strolld']`
`[1, -1, -1, 0, -1, 1]`	`['axseeds', 'brogger', 'cheddar', 'coiffes', 'crommel', 'djibbah', 'droller', 'fligger', 'frigger', 'frogger', 'griffes', 'grogger', 'grommet', 'prigger', 'proffer', 'progger', 'proller', 'quokkas', 'stiffen', 'stiffer', 'stollen', 'swigger', 'swollen', 'twiggen', 'twigger']`
`[0, 1, -1, 1]`	`['aargh', 'eeler', 'eemis', 'eeten', 'oopak', 'oozes', 'sstor']`
`[1, 1, 1, 1, 1, 1, 1]`	`['aegilops']`

Curious students can take on the challenge of finding the shape of length n – 1 that matches the most words, for *n* from 3 to 20. Alternatively, try to count how many possible shapes of length *n* – 1 do not match any words of length *n* at all. What is the shortest possible shape that does not match any of the words in our giant wordlist?

15. Chirality

```
def is_left_handed(pips):
```

Although this doesn't affect fairness, the pips from one to six are painted on the dice so the pips on opposite faces always add up to seven. (This convention makes it easier to tell when someone tries to use crooked dice where certain undesirable pip values have been replaced with more amenable opposite values.) In each of the $2^3 = 8$ corners of the cube, exactly one value from each pair of forbidden opposites 1–6, 2–5 and 3–4 meets two values chosen from the other two pairs of forbidden opposites. You can twist and turn the die to face any corner, yet no two opposite sides ever appear together.

This discipline still allows for two distinct ways to paint the pips. If the numbers in the corner shared by the faces 1, 2, and 3 read **clockwise** as 1–2–3, that die is **left-handed**, whereas if they read as 1–3–2, that die is **right-handed**. Analogous to a pair of shoes made separately for the left and right foot, the left- and right-handed dice are in one sense perfectly identical, and yet again, no matter how you twist and turn either yourself or the shoe, you can't seriously put either shoe on the other foot than the one it was designed for. (At least, not without taking that three-dimensional pancake "Through the Looking-Glass" by flipping it around in the fourth dimension!)

The three numbers around any other corner determine the three numbers on the unseen opposite sides, and therefore also the die's handedness just as firmly. Given the three-element list of `pips` read clockwise around some corner, determine whether that die is left-handed. There are only $2^3 \times 3! = 8 \times 6 = 48$ possible pip combinations to test for, so feel free to exploit these four two-fold symmetries to simplify your code. (The first four test cases below strongly hint at the nature of these symmetries, `<milhouse>`hint hint`</milhouse>`.)

`pips`	Expected result
`[1, 2, 3]`	`True`
`[1, 5, 3]`	`False`
`[5, 3, 1]`	`False`
`[5, 1, 3]`	`True`
`[6, 5, 4]`	`False`

This problem would certainly make for an interesting code-golf exercise, the propeller-beanie discipline we otherwise frown upon in this course as the least economical coding. Also, after solving that one, imagine that our physical space had k dimensions, instead of merely just the familiar three. How would dice even be *cast* (in both senses of this word) in k dimensions? How would you generalize your function to find the chirality of an arbitrary k-dimensional die?

16. The card that wins the trick

```
def winning_card(cards, trump=None):
```

Playing cards are represented as tuples of `(rank, suit)` as in the cardproblems.py lecture example program. In **trick-taking** games such as **whist** or **bridge**, each one of the four players plays one card from their hands into the trick. Starting with the player who leads the trick by playing the first card, the remaining players play their cards sequentially, so each player plays after seeing the cards the previous players played in that trick. The following rules determine which one of the four cards played to the trick wins the trick:

1. If one or more trump cards have been played to the trick, the highest-ranking trump card wins, regardless of the other cards played.
2. If no trump cards have been played to the trick, the highest card of the suit of the first card played wins. Cards of any other suit, regardless of rank, cannot win that trick.
3. Ace is the highest card in each suit.

This function should return the winning card for the list of `cards` played to the trick. The order in which cards are played to the trick greatly affects the outcome, since the first card dictates which suits can win after that lead.

`cards`	`trump`	Expected result
`[('three', 'spades'), ('ace', 'diamonds'), ('jack', 'spades'), ('eight', 'spades')]`	`None`	`('jack', 'spades')`
`[('ace', 'diamonds'), ('ace', 'hearts'), ('ace', 'spades'), ('two', 'clubs')]`	`'clubs'`	`('two', 'clubs')`
`[('two', 'clubs'), ('ace', 'diamonds'), ('ace', 'hearts'), ('ace', 'spades')]`	`None`	`('two', 'clubs')`
`[('six', 'hearts'), ('ace', 'spades'), ('three', 'hearts'), ('jack', 'hearts')]`	`'hearts'`	`('jack', 'hearts')`
`[('jack', 'spades'), ('queen', 'spades'), ('eight', 'spades'), ('two', 'diamonds')]`	`'diamonds'`	`('two', 'diamonds')`
`[('jack', 'spades'), ('queen', 'spades'), ('eight', 'spades'), ('two', 'diamonds')]`	`'clubs'`	`('queen', 'spades')`

17. Do you reach many, do you reach one?

`def knight_jump(knight, start, end):`

An ordinary chess knight on a two-dimensional board of squares can make an "L–move" into up to eight possible neighbours. However, as has so often been depicted in various works of space opera, higher-than-us hairless apes, and therefore more logical beings, can generalize the chessboard to k dimensions from the puny two dimensions more suitable for us hairless apes, who have barely evolved to come down from swinging from the trees. A natural generalization of the knight's move, while maintaining its spirit, is to define the possible moves as a k-tuple of **strictly decreasing,** nonnegative integer offsets. Each one of these k offsets must be used for exactly one dimension of your choice during the move, either as a positive or a negative, to determine the square where the k-knight will teleport as the unseen hand of the player lifts it there through the dimension $k + 1$.

Given the `start` and the `end` positions as k-tuples of integer coordinates, determine whether the `knight` could legally move from `start` to `end` in a single teleporting jump.

knight	start	end	Expected result
`(2, 1)`	`(12, 10)`	`(11, 12)`	`True`
`(7, 5, 1)`	`(15, 11, 16)`	`(8, 12, 11)`	`True`
`(3, 2, 1)`	`(5, 2, 1)`	`(7, 1, 3)`	`False`
`(9, 7, 6, 5, 1)`	`(19, 12, 14, 11, 20)`	`(24, 3, 20, 11, 13)`	`False`

A quick combinatorial calculation shows that exactly $k! \times 2k$ neighbours are reachable in a single move, except for moves that would jump outside the board. In this notation, the ordinary chess knight is a `(2,1)`–knight that reaches up to $2! \times 2^2 = 8$ neighbours in one jump. A 6-dimensional knight could reach a whopping $6! \times 2^6 = 46080$ different neighbours in one jump! Since the number of possible moves from each position grows exponentially with k, almost everything ends up close to almost everything else in high-dimensional spaces, giving predators too much of an edge for life ever to have a prayer in such spaces. Three dimensions feel just right for life to be comfortable and balanced, as all things should be.

18. Sevens rule, zeros drool

```
def seven_zero(n):
```

Seven is considered a lucky number in Western cultures, whereas zero is what nobody wants to be. Let's bring these two opposites together briefly, without letting it become some kind of emergency, by looking at positive integers that consist of a solid sequence of sevens, followed by a (possibly empty) solid sequence of zeros. Examples of integers of this form are 7, 7700, 77777, 77777700, and 70000000000000. A surprising theorem shows that for any positive integer `n`, an integer of this seven-zero form exists that is divisible by `n`. Therefore, infinitely many exist. (DUCY?) This function should find the smallest such seven-zero integer.

Even though discrete math and number theory always help, this exercise is not about coming up with a clever symbolic formula and proving its correctness. This problem is about iterating through the numbers of this constrained form of sevens and zeros efficiently in strictly ascending order, so that the function can mechanistically find the smallest working number of this form. However, to speed up the search, we accept that whenever `n` is not divisible by 2 or 5, the smallest such number will always consist of a solid sequence of sevens with no zeros after them. This can speed up your search by an order of magnitude for such friendly values of `n`.

This logic is best written as a generator that yields such numbers. The body of this generator consists of two nested loops. The outer loop iterates through the number of digits `d` in the current number. For each `d`, the inner loop iterates through all possible `k` from 1 to `d` to create a number that begins with `k` sevens, followed by `d-k` zeros. Most of the work is done inside that generator; the `seven_zero` function itself will be short and sweet.

`n`	Expected result
`70`	`70`
`17`	`7777777777777777`
`42`	`7770`
`77700`	`77700`
`2**50`	`70000000000000000000000000000000000000000000000000`
`12345`	(a behemoth that consists of 822 sevens, followed by a single zero)

This problem is adapted from the excellent MIT OpenCourseWare online textbook "*Mathematics for Computer Science*" (PDF link to the 2018 version for anybody interested) that, like so many other **non-constructive** combinatorial proofs, uses the pigeonhole principle to prove that *some* solution *must* exist for any integer `n`, but provides no clue of where actually to find that solution.

19. Fulcrum

```
def can_balance(items):
```

Each element in `items` is a positive integer; for this problem, treat it as a physical weight. Your task is to find a **fulcrum** position in this list of weights so that when balanced on that position, the total torque of the items to the left of that position equals the total torque of the items to the right of that position. Assume the item on the fulcrum itself is centred symmetrically on the fulcrum and does not contribute to the torque calculation.

In physics, an item's torque with respect to the fulcrum equals its weight times its distance from the fulcrum. If a fulcrum position exists, return that position. Otherwise, return –1 to artificially indicate that the given `items` cannot be balanced, at least without rearranging them.

`items`	Expected result
`[6, 1, 10, 5, 4]`	`2`
`[10, 3, 3, 2, 1]`	`1`
`[7, 3, 4, 2, 9, 7, 4]`	`-1`
`[42]`	`0`

Finding the fulcrum position when rearranging elements is allowed would be an interesting but more advanced problem, suitable for motivated students in a third-year computer science course. However, this algorithm can already be built based on what we have learned so far in an *effective* (although not as *efficient*) **brute-force** fashion around this function by using the generator `permutations` in the Python standard library module `itertools` to try out all possible permutations in an outer loop until the inner loop finds one permutation that balances.

(In fact, quite a few problems of this style can be solved with this "**generate and test**" approach without having to resort to the general **backtracking** algorithm taught in the third year.)

20. Fail while daring greatly

```
def josephus(n, k):
```

The ancient world of swords and sandals "🎶 *back when men were made of iron, and their ships were made of wood* 🎶" could occasionally be an entertainingly ribald and violent place, at least according to popular historical docudramas such as "300", "Spartacus: Blood and Sand" and "300: Rise of an Empire". During one particularly memorable incident, a group of zealots (yes, Lana, *literally*) found themselves surrounded by overwhelming Roman forces. To avoid capture and humiliating death by crucifixion, in their righteous zeal, these men committed themselves to mass suicide in a scheme that ensured each man's unwavering commitment to their final shared fate. These zealots arranged themselves in a circle, used lots to choose a step size `k`, and then repeatedly counted `k` men ahead, killed that man, removed his corpse from this grim circle, and kept going.

As normal people rather than computer scientists, they play this deadly, one-based game of eeny-meeny-miney-moe, and it continues until the last man standing is expected to fall on his own sword to complete the dance. Having unfortunately fallen in with this gang, Josephus would much rather be that last man, since he has other ideas for surviving. Help him survive with a function that, given `n` and `k`, returns the execution-order list, so Josephus knows which position lets him survive and walk away from this grim circle. A cute mathematical solution instantly determines the survivor when $k = 2$. Unfortunately, k can get arbitrarily large, even far exceeding the current number of men… if only to briefly excite us cold and timid souls, hollow men without chests, the rictus of our black lips gaped in a grimace that sneers at the strong men who once stumbled.

`n`	`k`	Expected result
`4`	`1`	`[1, 2, 3, 4]`
`4`	`2`	`[2, 4, 3, 1]`
`10`	`3`	`[3, 6, 9, 2, 7, 1, 8, 5, 10, 4]`
`8`	`7`	`[7, 6, 8, 2, 5, 1, 3, 4]`
`30`	`4`	`[4, 8, 12, 16, 20, 24, 28, 2, 7, 13, 18, 23, 29, 5, 11, 19, 26, 3, 14, 22, 1, 15, 27, 10, 30, 21, 17, 25, 9, 6]`
`10`	`10**100`	`[10, 1, 9, 5, 2, 8, 7, 3, 6, 4]`

(To lighten this hammy lament, delivered in earnest while milking the invisible cow, we can also note the adorable feline generalization of this problem for beings with several lives, now included in this same author's *Third Python Problem Collection*.)

21. All your bases are belong to us

`def group_and_skip(n, out, ins):`

On a table, there are `n` identical coins, to which the following operation will be applied as many times as possible. Arrange the coins still on the table into groups of exactly `out` coins per group, where `out` is a positive integer greater than one. For example, if `n==13` and `out==3`, this would use twelve of these thirteen coins to create four groups of three coins each. The `n%out` leftover coins (in this example, one leftover coin) that did not form a complete group are set aside and recorded. After this, from each complete group, exactly `ins` coins are placed back on the table. In this example, if `ins==2`, it puts eight coins back on the table: two coins from each of the four groups, with three coins.

Repeat this operation until the table has zero coins remaining, which must eventually happen whenever `out>ins`. Return a list that says how many coins were taken aside in each move.

n	out	ins	Expected result
13	3	2	[1, 2, 1, 2]
123456789	10	1	[9, 8, 7, 6, 5, 4, 3, 2, 1]
987654321	1000	1	[321, 654, 987]
255	2	1	[1, 1, 1, 1, 1, 1, 1, 1]
81	5	3	[1, 3, 2, 0, 4, 3]
10**9	13	3	[12, 1, 2, 0, 7, 9, 8, 11, 6, 8, 10, 5, 8, 3]

As you can see in rows from the second to the fourth, this method produces the digits of the nonnegative integer `n` written in base `out` in reverse order. So this setup is a cleverly disguised algorithm for constructing the base-`out` representation of `n`. However, an improvement over the usual base conversion algorithm is that this version works not only for integer bases, but also for any fraction `out/ins` that satisfies `out>ins` and `gcd(out, ins)==1`. For example, the famous integer 42 would be written as 323122 in base 4/3.

Yes, fractional bases for integers are an actual thing. Take a deep breath and think about the implications (hopefully more than just winning a bar bet), then imagine trying to do basic real-world arithmetic in such a system. That certainly would have been some "New Math" for frustrated parents in the swinging sixties, when merely balancing their cheque books in familiar base ten was already an exasperating ordeal!

22. Count the balls off the brass monkey

```
def pyramid_blocks(n, m, h):
```

Mysteries of the pyramids have fascinated humanity throughout the ages. Instead of packing your machete and pith helmet to trek through deserts and jungles to raid the hidden treasures of the ancients like some Indiana Croft, or by gaining visions of enlightenment by intensely meditating under the apex set over a square base like some Deepak Veidt, this problem deals with something a bit more mundane: truncated brass monkeys of layers of discrete uniform spheres, in the spirit of that spherical cow running in a vacuum.

Given that the **top** layer of the truncated brass monkey consists of `n` rows and `m` columns of spheres, and each solid layer immediately below the one above it always contains one more row and one more column, how many spheres in total make up this truncated brass monkey that has `h` layers?

You can solve this problem with straightforward brute force by tallying the spheres across the layers. However, the only reward for naughty boys and girls who take such a blunt approach is watching the automated tester take roughly a minute to terminate! Some creative use of discrete math and summation formulas gives an **analytical closed-form formula** that spits out the answer faster than you can snap your fingers by plugging in *n*, *m*, and *h*.

n	m	h	Expected result
2	3	1	6
2	3	10	570
10	11	12	3212
100	100	100	2318350
10**6	10**6	10**6	2333331833333500000

As an unrelated note, as nice as Python can be for casual coding by liberating us from all that low-level nitty-gritty, Wolfram is another great language with great online documentation. You can play around with this language for free on Wolfram Cloud to try out not just all the cool one-liners from the tutorials and documentation pages, but also evaluate arbitrary mathematical expressions of your own making. For example, `Sum[(n+i)(m+i), {i, 0, h-1}]`, all in a fully symbolic fashion.

23. Count growlers

```
def count_growlers(animals):
```

Let the strings `'cat'` and `'dog'` denote that kind of animal facing left, and `'tac'` and `'god'` denote that same kind of animal facing right. Since in this day and age this whole setup sounds like some kind of a meme or one of those TikTok videos that you kids are always glued to these days anyway, let us somewhat unrealistically assume that each animal, regardless of its own species, growls if it sees **strictly more dogs than cats** in the direction that the animal is facing. Given a list of such `animals`, return the count of how many of these animals are growling. In the examples below, the growling animals are highlighted in green.

`animals`	Expected result
`['cat', 'dog']`	`0`
`['god', 'cat', 'cat', 'tac', 'tac', 'dog', 'cat', 'god']`	`2`
`['dog', 'cat', 'dog', 'god', 'dog', 'god', 'dog', 'god', 'dog', 'dog', 'god', 'god', 'cat', 'dog', 'god', 'cat', 'tac']`	`11`
`['god', 'tac', 'tac', 'tac', 'tac', 'dog', 'dog', 'tac', 'cat', 'dog', 'god', 'cat', 'dog', 'cat', 'cat', 'tac']`	`0`

I will be the first to admit that I was high as a kite when I first thought up this problem. Like cops and robbers in some gritty Netflix action romp, problems and their solutions can be hard to tell apart, since that distinction depends on the angle you view the whole mess.

24. Bulgarian solitaire

`def bulgarian_solitaire(piles, k):`

In front of you are `k*(k+1)//2` identical pebbles, given as `piles`. This weird number of pebbles equals the k:th **triangular number**, the **arithmetic sum** of the positive integers from `1` to `k`, for every budding little Gauss out there to speed up their calculations. Bored out of your mind, you lazily arrange the pebbles. Eventually, you make a game out of it: seeing how many times you can arrange these pebbles into neat rows in a day, then trying to break that record.

An apt metaphor for the bleak daily life behind the Iron Curtain: in this solitaire, all pebbles are identical, and you have no choice in your moves. Each move picks up exactly one pebble from every pile (making piles with only one pebble vanish), and creates a new pile from this handful. For example, the first move from `[7, 4, 2, 1, 1]` turns into `[6, 3, 1, 5]`. The next move turns into `[5, 2, 4, 4]`, which then turns into `[4, 1, 3, 3, 4]`, and so on.

This function should count the number of moves from the initial `piles` to a **steady state** where each number from `1` to `k` appears as the size of **exactly one pile**, and return that count. These numbers from `1` to `k` may appear in any order, not necessarily sorted. (Applying the move to this steady state simply leads right back to that same steady state, hence the name.)

`piles`	`k`	Expected result
`[1, 4, 3, 2]`	`4`	`0`
`[8, 3, 3, 1]`	`5`	`9`
`[10, 10, 10, 10, 10, 5]`	`10`	`74`
`[3000, 2050]`	`100`	`7325`
`[2*i-1 for i in range(171, 0, -2)]`	`171`	`28418`

This problem comes from the Martin Gardner column "*Bulgarian Solitaire and Other Seemingly Endless Tasks,*" where it was used as an example of a while loop where it is not immediately obvious that this loop will eventually reach its goal and terminate, analogous to the unpredictable behaviour of the **Collatz 3*x* + 1 problem** seen in `mathproblems.py`. However, unlike that still-annoyingly open problem, Bulgarian solitaire can be proven never to get stuck, and to reach the steady state from any starting configuration of triangular numbers in at most $k(k-1)/2$ steps.

25. Scylla or Charybdis?

```
def scylla_or_charybdis(moves, n):
```

Thanks to your recent cunning stunts, your nemesis in life finds herself trapped in a devious game where, for a refreshing change, you play the Final Boss. (Everyone is the hero in their own story until they become a villain in someone else's.) Her showdown is a one-dimensional 8-bit video game platform that extends $n - 1$ discrete steps in both directions from its center, with monotonic *beep-boop* sound effects to complete the mood of that era. At the ends of this platform, each exactly `n` steps from the center, your henchmen Scylla and Charybdis lick their lips in anticipation of a tasty morsel falling over the edge.

Your nemesis starts at the center of this platform and must begin by immediately committing to her entire sequence of future `moves` as a string of `'+'` ("🎶 just a step to the ri-i-i-i-ight 🎶") and `'-'` (move one step to the left). Your task is to find and return a positive integer k so that executing every k:th step of `moves` (so this subsequence starts from position $k - 1$ and includes every k:th element from then on) makes your nemesis fall into either one of the two possible dooms that are waiting `n` steps away from her starting point at the center.

If several values of `k` do the job, return the smallest value of `k` among those that minimize the number of steps to the downfall. Otherwise, those freaking Tolkien eagles will again swoop down to carry your nemesis back to safety. She is then guaranteed to return in a third-year course on analysis of algorithms as an **adversary** to put your functions through a wringer of wit, where she gets to wield the same second-mover advantage you enjoyed this time. So, no half measures!

`moves`	`n`	Expected result
`'-++--++-++++'`	2	3
`'--+++++---+-++-++++++++++'`	5	5
`'+-++---+-+-++-+++----+++-++-+---+--++++++++++'`	5	7
`'+--++---+-++-+++------+++++---++-+--+++-+--++-++-++-+-+++++++++++++++++'`	9	1

This problem was inspired by the article "A Magical Answer to the 80-Year-Old Puzzle" in Quanta Magazine. No less a mathematical maven than Paul Erdős himself asked whether for some sufficiently large platform width n, your nemesis could stay on that platform forever with a tactically chosen infinite sequence of `moves` so that no matter what step size k you chose to thwart her plan, she would always nimbly sidestep her doom. Internet mathematician Terry Tao recently answered this question in the negative. For any chosen infinite sequence of moves, you can choose a suitable k to thwart her moves and force your nemesis as far as you want from the starting point.

26. Longest arithmetic progression

```
def arithmetic_progression(items):
```

An **arithmetic progression** is an integer sequence in which the difference between consecutive elements is constant throughout the sequence. For example, `[4,7,10,13,16]` is an arithmetic progression of length 5, starting from 4 with the difference of 3. For example, `[5,4,3]` is another arithmetic progression of length 3, starting at 5 with a difference of –1.

Given a non-empty list of positive integer `items` in strictly ascending order, find and return the longest arithmetic progression whose values all exist inside that sequence. Return your answer as a tuple `(start, difference, n)` for the three components to define an arithmetic progression. To ensure unique results to facilitate automated testing, whenever several progressions of equal length exist, this function should return the one with the lowest `start`. If several progressions of equal length emanate from the same `start`, return the one with the smallest `difference`.

`items`	Expected result
`[42]`	`(42, 0, 1)`
`[2, 4, 6, 7, 8, 12, 17]`	`(2, 2, 4)`
`[1, 2, 36, 49, 50, 70, 75, 98, 104, 138, 146, 148, 172, 206, 221, 240, 274, 294, 367, 440]`	`(2, 34, 9)`
`[2, 3, 7, 20, 25, 26, 28, 30, 32, 34, 36, 41, 53, 57, 73, 89, 94, 103, 105, 121, 137, 181, 186, 268, 278, 355, 370, 442, 462, 529, 554, 616, 646, 703]`	`(7, 87, 9)`
`range(1000000)`	`(0, 1, 1000000)`

Bridge players distinguish between “best result possible” and “best possible result”, which are not the same thing! In the same spirit, do you see the difference between two deceptively similar concepts of “leftmost of longest” and “longest of leftmost”?

27. Best one out of three

```
def tukeys_ninthers(items):
```

Back in the day, when computers were far slower, and our programs had far less RAM to burrow into, we needed special techniques to do things that are trivial today with a couple of lines of code. In this spirit, "Tukey's ninther" is an eponymous **approximation algorithm** that quickly finds a value "reasonably close" to the **median** of the given unsorted `items`. For this problem, the median of the list is defined to be the element that would end up in the middle position after sorting that entire list. This definition makes the median unambiguous regardless of the elements and their multiplicities. Note that this function is not tasked with finding the true median, which would be a trivial Python one-liner by sorting the `items`, but with finding the element that the original Tukey's ninther algorithm returns for those `items`.

Tukey's algorithm first splits the `items` into triplets and finds the median of each triplet. It collects these medians-of-three into a new list, then repeats the operation until only one element remains. For simplicity, your function may assume that the length of `items` is always some power of three. In the following table, each row contains the result produced by applying a single round of Tukey's algorithm to the list immediately below it.

`items`	Expected result
`[15]`	`15`
`[42, 7, 15]`	`15`
`[99, 42, 17, 7, 1, 9, 12, 77, 15]`	`15`
`[55, 99, 131, 42, 88, 11, 17, 16, 104, 2, 8, 7, 0, 1, 69, 8, 93, 9, 12, 11, 16, 1, 77, 90, 15, 4, 123]`	`15`

Tukey's algorithm is extremely **robust**. You can see this by giving it a bunch of randomly shuffled lists of the same distinct numbers to operate on, and admiring how tightly the histogram of results hugs the actual median. For example, the median of the last example list in the above table is really 15, pinky swear for grownup realsies. These distinct numbers can even come from arbitrary distributions over arbitrarily wide scales, since this **purely comparison-based** algorithm never performs any arithmetic between elements. Even better, if all `items` are distinct and the length of the list is some power of three, the returned guess for median can never come from the true top or bottom third of the sorted elements (discrete math side quest: prove this), thus eliminating all risk of using some funky outlier as the approximate median.

28. Collecting numbers

```
def collect_numbers(perm):
```

This problem is adapted from the problem "Collecting Numbers" in the CSES Problem Set. Your Nemesis has shrunk you to a microscopic scale and trapped you inside the computer RAM chip. You are currently standing at the first position of `perm`, some **permutation** of integers from 0 to $n - 1$, each such number appearing in this list exactly once. The rules of this game allow you to move only right, advancing one element at a time, but you may never step left or jump over any elements. This list is treated as **cyclic**, so whenever you step past the last element, you return to the beginning, having completed another round around the carousel.

To get out, you must tell your Nemesis as soon as possible how many rounds it would take you to collect the elements from 0 to n – 1 in exact ascending order. Following the movement rules, you would keep going right until you find the element 0. After that, you would keep going right until you find the next element, 1. When you reach the end, start a new round from the beginning. Eventually, you will collect all the numbers in order and be done. For example, given the permutation `[2, 0, 4, 3, 1]`, you collect 0 and 1 during the first round. In the second round, you collect both 2 and 3. In the third round, you collect the remaining 4 and call it a day, for a total of three rounds.

The key lesson is that a function that simulates another system as a black box doesn't have to follow the system's rules; it can use any computational shortcut, as long as the final answer is correct. Therefore, beat your Nemesis with flair by first constructing the **inverse permutation** of `perm`. This inverse permutation `inv` is another permutation of integers from 0 to $n - 1$ that satisfies `inv[i]==j` whenever `perm[j]==i`. For example, the inverse of the previous permutation is `[1, 4, 0, 3, 2]`. The `inv` list lets you quickly look up the location of any particular element in the original `perm`. Once your function has constructed `inv`, you will be able to compute the required number of rounds with a single for-loop over `range(n)` whose body does not even glance at the original `perm`, since it gets all the information that it needs from the `inv` list alone.

perm	Expected result
`[0, 1, 2, 3, 4, 5]`	`1`
`[2, 0, 1]`	`2`
`[0, 4, 3, 5, 2, 1]`	`4`
`[0, 2, 4, 1, 3, 5]`	`3`
`[8, 6, 9, 5, 4, 11, 2, 0, 3, 10, 12, 1, 7]`	`7`
`list(range(10**6, -1, -1))`	`1000001`

29. Between the soft and the NP–hard place

```
def verify_betweenness(perm, constraints):
```

Following the previous problem of collecting numbers, here is another problem about permutations, where computing the inverse permutation first makes the actual result a breeze. Define a permutation of integers from 0 to *n* – 1 without duplicated elements as satisfying the **betweenness** constraint given as a three-tuple `(a, b, c)` if the middle element `b` lies in that permutation somewhere between the boundary elements `a` and `c`. The elements `a` and `c` can appear in either order in the permutation, but `b` has to be between them either way.

For example, the permutation `[2, 1, 3, 0]` satisfies the betweenness constraints `(1, 3, 0)` and `(0, 1, 2)`, but violates the betweenness constraints `(1, 3, 2)` and `(3, 0, 2)`. This function should determine whether the given `permutation` simultaneously satisfies all of the given betweenness `constraints`.

perm	constraints	Expected result
`[2, 0, 3, 1]`	`[(0, 2, 1), (1, 3, 0), (1, 3, 2)]`	`False`
`[0, 2, 3, 5, 4, 1]`	`[(0, 3, 1), (4, 2, 0)]`	`True`
`[5, 2, 0, 3, 1, 4]`	`[(4, 3, 5), (2, 0, 4), (2, 1, 4), (1, 2, 5), (4, 0, 5)]`	`True`
`[6, 1, 0, 3, 2, 4, 5]`	`[(2, 3, 4), (4, 0, 3)]`	`False`

This problem is far more subtle than it initially appears. Even though we can verify relatively quickly whether a given permutation satisfies the constraints, the inverse problem, determining whether the constraints can be satisfied by at least one permutation, any one permutation you choose, turns out to be NP-complete! As with all other NP-complete problems, the worst-case running time of all known algorithms guaranteed to find one such permutation whenever one exists grows exponentially with the permutation's length. The fundamental asymmetry between how easy it is to verify that the solution that some hinky guy in a dark alley offers to sell you satisfies the desired constraints, versus the exponential difficulty of constructing a working solution for those same constraints, seems to be built into the "physics of the abstract" of our very shared reality.

30. Count Troikanoff, I presume

```
def count_troikas(items):
```

Define three positions $i < j < k$ in the `items` list to constitute a **troika** if the elements in all three positions are equal, and the positions themselves are spaced apart in an **arithmetic progression**. More succinctly, these positions satisfy `items[i]==items[j]==items[k]` and `j-i==k-j`. This function should count the number of troikas in the given list of `items`. For example, the list `[42, 17, 42, 42, 42, 99, 42]` contains four troikas (count 'em), each using the value `42`.

You could solve this inefficiently by iterating through all equally spaced position triples (i, j, k). (You only need to loop through i and j, since those nail down k.) However, you can do better by building a `dict`ionary whose keys are the elements that appear in `items`. Each value is a sorted list of positions where that particular key appears in `items`. For the previous example list, this dictionary would be `{42: [0, 2, 3, 4, 6], 17: [1], 99: [5]}`.

Then, loop through the keys of this dictionary. For each key, loop through every position pair $i < j$ in its value list (the decorator `itertools.combinations` again goes *brrrrr*, ha ha), and compute the assumed third position with the formula $k = j + (j - i)$. If `items[k]==items[i]`, increment your troika counter. Searching for troikas this way typically takes an order of magnitude less time than the "Shlemiel" brute-force method. The more distinct the values in `items` are, the sharper this method's edge becomes.

items	Expected result
`[5, 8, 5, 5]`	`0`
`[-8, -8, -8, -18, -8, 13, -8]`	`3`
`[3, 6, 3, 6, 3, 6, 6, 6, 3]`	`5`
`[23, 23, 23, 23, 23, 23, 16, 23]`	`8`
`[21, 21, 21, 21, 21, 21, 21, -41, 21, 76, 21, -71, 47]`	`15`
`[42 for _ in range(100)]`	`2450`

31. Crack the crag

```
def crag_score(dice):
```

Crag is an old dice game similar to the more popular Yahtzee and Poker dice in style and spirit, but with simpler combinatorics for roll-value calculation because it uses only three dice. Players repeatedly roll three dice and assign the resulting patterns to **scoring categories**; once a roll is assigned to a category, that category is spent and cannot be used again on a future roll. These tactical choices between safety and risk-taking give the game a more tactical flair, rather than relying solely on Lady Luck when rolling the bones. See the Wikipedia page for the scoring table used in this problem.

Given the pips of the three dice on the first roll, this function should return the highest possible score when all categories in the scoring table are still available, so all that matters is greedily maximizing the score for this first roll. Note that the examples on the Wikipedia page show the score that some dice would score **in that particular category**, which is not necessarily even close to the maximum score in principle attainable with that roll. For example, the roll `[1, 1, 1]`, used inefficiently in the category "Ones," scores only three points, whereas the same roll scores a whopping 25 points in the category "Three of a kind." (Problem 107 in this collection, "Optimal crag score", has you distribute a series of such rolls into distinct categories to maximize the overall score, not merely the score for the first roll.)

This problem ought to be a straightforward exercise on if-else ladders combined with simple sequence management. Your function should be swift and sure to return the correct answer for every one of the $6^3 = 216$ possible pip values of three dice. However, you will surely design your conditional statements to handle entire **equivalence classes** of pips in a single step, so that your entire ladder consists of *far* fewer than 216 separate steps.

`dice`	Expected result
`[1, 2, 3]`	`20`
`[4, 5, 1]`	`5`
`[3, 3, 3]`	`25`
`[4, 5, 4]`	`50`
`[1, 1, 1]`	`25`
`[1, 1, 2]`	`2`

32. Three summers ago

```
def three_summers(items, goal):
```

Given a list of positive integer `items` sorted in ascending order, determine whether there exist **precisely three** separate elements in `items` that together add up exactly to the given `goal`.

Sure, you could solve this problem with three nested loops to check every possible triple of elements from items to see whether they add up to the `goal`. However, iterating over all triples gets pretty slow as the list length increases, since the number of triples grows proportionally to the cube of the list length. Of course, the automated tester will make the lists given to your function large enough to make such solutions reveal themselves by their glacial running time.

Since `the items` are sorted, a better technique will find the answer much faster. See the function `two_summers` in the class example program `listproblems.py` to quickly find two elements in the given sorted list that together add up to the given `goal`. You can use this function as a subroutine to speed up your search for three summing elements, once you realize that the list contains three elements that add up to `goal` if and only if it contains some element `x` so that the remaining list after `x` contains two elements that add up to `goal-x`.

items	goal	Expected result
[10, 11, 16, 18, 19]	40	True
[10, 11, 16, 18, 19]	41	False
[1, 2, 3]	6	True

For the general **subset sum problem** that was used as an example of inherently **exponential** branching recursion in that lecture, the question of whether the given list of integers contains some subset of k elements that together add up to the given `goal` can be determined by trying each element `x` in turn as the highest element of this subset, and then recursively determining whether the remaining elements before `x` contain some subset of elements that add up to `goal-x`.

33. Sum of two squares

`def sum_of_two_squares(n):`

Many positive integers can be expressed as a sum of exactly two squares of positive integers, both possibly equal. For example, 74 = 49 + 25 = 7^2 + 5^2. This function should find and return a tuple of two positive integers whose squares add up to `n`, or return `None` if the integer `n` cannot be expressed as a sum of two squares.

The returned tuple must present the larger of its two numbers first. Furthermore, if some integer can be expressed as a sum of two squares in several ways, return the breakdown that maximizes the larger number. For example, the integer 85 has two such representations, 7^2 + 6^2 and 9^2 + 2^2, so this function must return `(9,2)`. (Perhaps numbers like this could be named "pedicab numbers", a kid brother of their more famous big brother of taxicab numbers.)

The classic coding technique of **two pointers**, as previously seen in the function `two_summers` in the `listproblems.py` example program, also directly works on this problem. The two pointers start at both ends of the sequence and move toward each other, ensuring neither skips over a solution. This process must eventually reach a definite outcome: either a working solution is found, or the pointers meet before finding one. In **binary search**, one of these indices jumps halfway toward the other each round, making the execution time logarithmic in `n`. However, we are not in such a lucky situation; the two pointers must inch toward each other one position at a time.

n	Expected result
`1`	`None`
`2`	`(1, 1)`
`50`	`(7, 1)`
`8`	`(2, 2)`
`11`	`None`
`123**2 + 456**2`	`(456, 123)`
`55555**2 + 66666**2`	`(77235, 39566)`

("Hey, I like that. One pointer goes one way, the other pointer goes the other way, and the elements in the middle are like, what do you want from me?")

34. Carry on Pythonista

```
def count_carries(a, b):
```

You can add two positive integers, `a` and `b`, using the usual column-wise addition algorithm we learned as wee little children, so early that most people don't even think of that mechanistic operation as an "algorithm". Instead of the sum `a+b` that the language would already compute for you anyway, this problem asks you to count how many times there will be a **carry** of one into the next column during this mechanistic column-wise addition. Your function should be efficient even for behemoths with thousands of digits.

To extract the lowest digit of a positive integer `n`, use the expression `n%10`. To extract all other digits except the lowest one, use the expression `n//10`. You can use these simple integer arithmetic operations to traverse the steps of the **column-wise integer addition** where this time you don't care about the actual result of the addition, but only tally up the carries produced in each column as **proof of work** of having laboured through the steps of the column-wise addition.

a	b	Expected result
`0`	`0`	`0`
`99999`	`1`	`5`
`11111111111`	`2222222222`	`0`
`123456789`	`987654321`	`9`
`2**100`	`2**100 - 1`	`13`
`10**1000 - 123**456`	`123**456`	`1000`

35. As below, so above

```
def leibniz(heads, positions):
```

Regardless of your stance on the famous Pascal's wager, betting that some rows of the famous Pascal's triangle will be generated during the first two years of a computer science undergraduate program should be as safe as houses. Even without considering the interesting combinatorial sequences inside this infinite table, merely tabulating its entries is a good exercise in nested loops.

This problem looks at **Leibniz triangles**, a curious "upside-down" variation of Pascal's triangle where each entry equals the sum of the two entries immediately **below** it, instead of above it, as in Pascal's triangle. The immediate problem with such upside-down recursion is that it has no base cases to terminate the recursion. Instead of Pascal's base case of the unity bit at the top whose union with the zero bit of the nonexistent void outside the triangle fills in the rest of the triangle as an ordered universe full of lively mathematical truths and combinatorial entities, a Leibniz triangle branches out into a cacophonous pyramid of turtles all the way down to Dante's ten circles, whence the Opposing Adversary can fill it in uncountably (yes, Lana, *literally*) many ways, each of these more diabolical than the next.

Fortunately for us, clouds of dust dancing in the great wind that carries us all, merely defining the `heads`, the leading elements of every row, sufficiently constrains this recursion to rein in the rest of the triangle. For example, one such possible triangle, perhaps even the best of all possible such triangles, is the original Leibniz harmonic triangle that uses consecutive **unit fractions** as `heads`. If a dispute about any particular value arises, then let us simply take out our mechanistic step reckoners and calculate, without further ado, to see who is right.

This function should immanentize as many rows of the Leibniz triangle as there are `heads` given, using two nested loops to compute these entries. However, this function needs to compute only the entries in the given `positions` of the final generated row. Having verified those to be correct, we can safely assume your intermediate rows have also been correct, without actually forcing you to "show your work".

`heads`	`positions`	Expected result
`[3, 2]`	`range(2)`	`[2, 1]`
`[1, -1, 1, -1]`	`range(4)`	`[-1, 2, -4, 8]`
`range(6)`	`range(6)`	`[5, -1, 0, 0, 0, 0]`

(Extra gold stars are awarded for solutions that Ol' Leiby himself would have grokked. And this guy sure grokked a lot about most of the things known at his time.)

36. Expand positive integer intervals

```
def expand_intervals(intervals):
```

An **interval** of consecutive positive integers can be succinctly described as a string containing its first and last values, inclusive, separated by a minus sign. (This problem is intentionally restricted to positive integers so that there will be no ambiguity between the minus sign character used as a separator and an actual unary minus sign tacked in front of a digit sequence.) For example, the interval containing the integers 5, 6, 7, 8, 9 can be more concisely described as `'5-9'`. Multiple intervals can be described together by separating their descriptions with commas. A **singleton** interval consisting of one value is given as that value.

Given a string of such comma-separated `intervals`, guaranteed to be in sorted ascending order and never overlap or be contiguous with each other, this function should create and return the list that contains all the integers contained inside these intervals. As with any problem, don't reinvent the wheel; first check whether the string objects offer any useful methods that make your job easier.

`intervals`	Expected result
`''`	`[]`
`'42'`	`[42]`
`'4-5'`	`[4, 5]`
`'4-6,10-12,16'`	`[4, 5, 6, 10, 11, 12, 16]`
`'1,3-9,12-14,9999'`	`[1, 3, 4, 5, 6, 7, 8, 9, 12, 13, 14, 9999]`

For purely aesthetic reasons, we restrict the problem to positive numbers. The above notation would still be unambiguous even if the same minus sign character served two masters, acting as both the element separator and the unary minus sign. As with parses of more complicated languages such as Python, where commas and asterisks can mean multiple things depending on the preceding context, the parser's internal state always decides what each character means at its current position.

We can also note that, for some still-unexplained reason, different-length dashes and quotation marks pointing in different directions tend to go unused in computer text, even though these characters are Unicode symbols like any old-timey ASCII symbol. (Perhaps ASCII stands for “ass-kiss” here; go figure.)

37. Collapse positive integer intervals

```
def collapse_intervals(items):
```

This function is the inverse of the previous problem of expanding positive integer intervals. Given a nonempty list of positive integer `items` guaranteed to be in sorted ascending order, create and return the unique description string where every **maximal** sublist of consecutive integers has been condensed to the notation `first-last`. This encoding doesn't save characters when the first and last values differ by only one, but uniformity matters more than prettiness. We have space to splurge, so don't worry about it. As a general principle, uniform, consistent data encoding lets you process that data uniformly across tools down the line, an advantage well worth paying a few extra bytes of storage for.

If a maximal sublist consists of a single integer, it must be included in the result string by itself without the minus sign separating it from the redundant `last` number. Make sure that the string returned by your function does not contain any whitespace characters, and that it does not have a redundant comma hanging at the end.

items	Expected result
`[1, 2, 4, 6, 7, 8, 9, 10, 12, 13]`	`'1-2,4,6-10,12-13'`
`[42]`	`'42'`
`[3, 5, 6, 7, 9, 11, 12, 13]`	`'3,5-7,9,11-13'`
`[]`	`''`
`range(1, 1000001)`	`'1-1000000'`

As you have come this far in solving the problems in this collection, surely your function will process the given `items` in pure sequential order so that your function works for arbitrarily long lazy sequences such as `range(10**10)` without running out of memory during the churn, yes?

38. Prominently featured

```
def prominences(height):
```

The one-dimensional silhouette of an island is given as a `height` list of raw **elevation** values at each position, measured from the sea-level baseline. To simplify things, this rocky, volcanic island is guaranteed not to contain any plateaus, consecutive positions with equal elevation. Positions outside the `height` list are assumed to lie at sea level at zero height.

This function should return the list of **peaks** on this island, that is, positions whose immediate left and right neighbours lie at a lower elevation. Each peak is represented as a three-element tuple `(position, elevation, prominence)`. Take the position and elevation of each peak directly from the `height` list. As explained on the Wikipedia page "Topographical prominence", a peak's prominence is the minimum vertical descent needed to reach some strictly higher peak, where you can freely choose the higher peak and the route to minimize this descent. (In technical terms, you must do **maximin optimization** to choose the best route and *max*imize the height of its *min*imum point.) The lowest valley along this best route is the **key col** of the original peak, and the next strictly higher peak reached through the key col is its **parent peak**. Since the island's highest peak has no key col, its prominence is defined to equal its raw elevation.

This problem will soon get tricky for two-dimensional height fields with all the ensuing freedom to go around things. Fortunately, in our far simpler one-dimensional version, you can first construct a list of all peaks and valleys with one linear pass through the height, since only these entries matter for calculating prominences. For each peak, proceed to the left until you reach some higher peak, keeping track of the lowest elevation seen along the way. Then do the same thing, going right from that same starting peak. The higher of these two lowest valleys is the key col that determines the prominence of that peak.

`height`	Expected result
`[42]`	`[(0, 42, 42)]`
`[1, 3]`	`[(1, 3, 3]`
`[1, 3, 2, 5, 1]`	`[(1, 3, 1), (3, 5, 5)]`
`[4, 2, 100, 99, 101, 2]`	`[(0, 4, 2), (2, 100, 1), (4, 101, 101)]`
`[1, 10, 8, 10, 5, 11]`	`[(1, 10, 5), (3, 10, 5), (5, 11, 11)]`
`[3, 5, 9, 12, 4, 3, 6, 11, 2]`	`[(3, 12, 12), (7, 11, 8)]`

39. Like a kid in a candy store, except without money

```
def candy_share(candies):
```

A group of kindergarteners sit in a circle, with some candies in front of each child. To synchronize them to move as a group in a logically simultaneous way, the teacher rings a bell to mark each round. At each ring, each child who presently has at least two pieces of candy must pass one piece to the left and one piece to the right. Children who have zero or one piece at that moment simply sit out that round, waiting for candies to reach their position so they can join in.

As in other chip-firing games of this kind, the total number of candies remains fixed throughout the process, assuming some little fatsenheimer won't fail the marshmallow test implicitly going on here. As shown in "Mathematics Galore", James Tanton's delightful collection of recreational mathematical problems, from which this problem was adapted, whenever the initial state contains strictly fewer candies than there are children, this process will eventually spread the candies evenly around the circle and reach a unique stable state where no child has more than one piece of candy. Given the initial distribution of `candies`, this function should return the number of rounds needed to reach this stable state.

`candies`	Expected result
`[1, 0, 1, 1, 0, 1]`	`0`
`[3, 0, 0, 0]`	`1`
`[4, 0, 0, 0, 0, 1]`	`6`
`[5, 1, 0, 0, 0, 0, 0, 1, 0]`	`10`
`[0, 0, 0, 10, 0, 0, 0, 0, 0, 0, 0, 0, 0]`	`21`

With more candies than children, the simple pigeonhole principle shows why this process can never terminate. However, interesting chaos breaks loose in the borderline case where there are exactly as many candies as children! Some states, such as `[0,3,0]`, reach a stable state, while others, such as `[2,1,0]`, are destined to race forever around a **limit cycle**, passing the same candy around the circle. It remains an unsolved mathematical problem to classify starting states significantly faster than mechanistically iterating the process to see what happens! (Falling into the limit cycle can be detected efficiently with the famous tortoise and hare algorithm, as you will see later with the "Bulgarian cycle" problem in this collection.)

40. Dibs to dubs

```
def duplicate_digit_bonus(n):
```

Some of us ascribe deep significance to numerical coincidences, so consecutive repeated digits or other low-description-length patterns, such as a digital clock blinking 11:11, seem special and personally meaningful to us. These people often find numbers with repeated or ascending digits more fascinating than ordinary numbers with no obvious patterns. For example, getting into a taxicab flashing an exciting number such as 1234 or 6969 would be more instatokkable than getting into a taxicab adorned with some more pedestrian number such as 1729.

Assume that some such person assigns a meaningfulness score to every positive integer so that every maximal block of `k` consecutive digits with `k>1` scores `10**(k-2)` points for that block. A block of two digits thus scores one point, three digits score ten points, four digits score a hundred points, and so on. However, to make this more interesting, a special rule applies: whenever a block of digits appears at the lowest end of the number, it scores twice as many points as it would in any other position. This function should compute the meaningfulness score of the given positive integer `n` as the sum of its individual block scores.

n	Expected result
`43333`	`200`
`2223`	`10`
`777777777`	`20000000`
`3888882277777731`	`11001`
`2111111747111117777700`	`12002`
`99999977777774444488872222`	`21210`
`1234**5678`	`15418`

41. Nearest smaller element

```
def nearest_smaller(items):
```

Given a list of integer `items`, create and return a new list of equal length so that each element has been replaced with the nearest element in the original list whose value is smaller. If no smaller elements exist because that element was the minimum of the original list, that element should remain as it is in the result list. If two smaller elements exist equidistant in both directions, this function should resolve this ambiguity by always using the smaller of these two elements.

`items`	Expected result
`[42, 42, 42]`	`[42, 42, 42]`
`[42, 1, 17]`	`[1, 1, 1]`
`[42, 17, 1]`	`[17, 1, 1]`
`[6, 9, 3, 2]`	`[3, 3, 2, 2]`
`[5, 2, 10, 1, 13, 15, 14, 5, 11, 19, 22]`	`[2, 1, 1, 1, 1, 13, 5, 1, 5, 11, 19]`
`[1, 3, 5, 7, 9, 11, 10, 8, 6, 4, 2]`	`[1, 1, 3, 5, 7, 9, 8, 6, 4, 2, 1]`

Side mission for any combinatorics enthusiasts reading this: starting from a random permutation of *n* distinct elements, what is the expected number of rounds that the above algorithm needs to be repeatedly iterated for all elements in the list to become equal?

42. Iterated ordinal transform

```
def ordinal_transform(seed, i):
```

The **ordinal transform** of a `sequence` of elements is an integer sequence of the same length, whose element at each position i equals the count of how many original elements, up to and including position i, equal the element at position i. For example, the ordinal transform of the list `[2, 42, 1, 1, 42]` equals `[1, 1, 1, 2, 2]`. (Note how every element of the ordinal transform will be positive.) The ordinal transform can be computed with one loop through the sequence, using a Python dictionary to store each element and its occurrence count. This lets us compute each element of the ordinal transform in constant time, instead of looping through all preceding elements to count that element's occurrences.

The **iterated** ordinal transform repeatedly concatenates the current sequence with its ordinal transform, doubling the sequence's length each time. For example, starting with `[42, 99]`, whose ordinal transform is `[1, 1]`, the first round produces `[42, 99, 1, 1]`. Since the ordinal transform of this new sequence is `[1, 1, 1, 2]`, concatenating these two sequences in the second round produces `[42, 99, 1, 1, 1, 1, 1, 2]`. Since each step contains the previous step's sequence as a prefix, the infinite ordinal transform makes sense as the limit of repeating this operation forever. This function should iterate over the given `seed` sequence until the sequence contains the (zero-based) position `i`, and return the element at that position.

`seed`	`i`	Expected result
`[1]`	`1`	`1`
`[4, 2]`	`6`	`1`
`[1, 1, 6, 4]`	`30`	`2`
`[3, 1, 3, 10, 13, 5, 15, 10, 15, 14]`	`184`	`12`
`[2, 10, 10, 8, 10, 10, 8, 10, 8]`	`196`	`17`

The iterated ordinal transform of the simplest possible seed `[1]` has a nice **self-similar** fractal nature, qualifying it for inclusion in the Online Encyclopedia of Integer Sequences, as this tweet from Amazing Graphs shows. The reader might enjoy comparing the evolution of that sequence with seeding it with either mustard seed `[2]` or `[42]`. How far would you have to generate the two sequences that start from one million and one million plus one to see the rest of these sequences finally diverge from each other?

43. Interesting, intersecting

`def squares_intersect(s1, s2):`

An **axis-aligned** square on the two-dimensional plane can be defined as a tuple `(x, y, r)`, where (x, y) are the coordinates of its bottom-left corner and r is the side length. Given two squares as tuples `(x1, y1, r1)` and `(x2, y2, r2)`, this function should determine whether these two squares **intersect** by having at least one point in common, even if that one point is the shared corner point of two squares placed kitty-corner. This function **should not contain any loops or list comprehensions of any kind**, but should compute the result using only integer comparisons and conditional statements.

This problem illustrates a common approach to problems of this nature: it's far easier to determine that the two axis-aligned squares do not intersect, then negate that for the final answer! Two squares do not intersect if one ends horizontally before the other begins, or if the same thing happens vertically. This technique generalizes from rectangles lying on the flat two-dimensional plane to not only three-dimensional cuboids, but to hyper-boxes of arbitrarily high dimensions. (Hyper, schmyper. As far as this author is concerned, there can never be too many dimensions.)

s1	s2	Expected result
`(2, 2, 3)`	`(5, 5, 2)`	`True`
`(3, 6, 1)`	`(8, 3, 5)`	`False`
`(8, 3, 3)`	`(9, 6, 8)`	`True`
`(5, 4, 8)`	`(3, 5, 5)`	`True`
`(10, 6, 2)`	`(3, 10, 7)`	`False`
`(3000, 6000, 1000)`	`(8000, 3000, 5000)`	`False`
`(5*10**6, 4*10**6, 8*10**6)`	`(3*10**6, 5*10**6, 5*10**6)`	`True`

44. So shall you sow

```
def oware_move(board, house):
```

Oware, an African board game, is one of the most popular Mancala variations. After appreciating the simple general structure of **sowing games** that allow us to toy with **emergent complexity** with minimal equipment of holes and stones that are amply available in all places on this planet where the human hand has ever set down its foot, we first display some mathematical gusto by generalizing the game mechanics for an arbitrary number of *k* houses. The complete **minimax algorithm** for finding the best move in the given situation will have to wait until the course CCPS 721 *Artificial Intelligence*. However, we already have a small but essential part of this algorithm: the **move executor**, which plays out the chosen move in the given situation.

The `board` is a list with 2*k* elements: the first *k* represent the current player's houses, and the last k represent the opponent's. (The stores that keep track of the captured stones do not appear anywhere in this list.) This function should return the outcome of picking up the seed from the given `house` on your side (numbering of houses starting from zero) and **sowing** these seeds counterclockwise around the `board`. Skip the original `house` during sowing, even if it had enough seed to complete a full lap.

After sowing, the **capturing** stage begins at the house where the last seed was sown. Capturing continues backwards from that house as long as the current house is on the opponent's side and contains two or three seeds. As soon as the current house contains some other number of seeds, the capturing ends even if some opponent houses after that contain two or three seeds. For simplicity, we ignore edge cases stemming from the **grand slam rule** and the gentlemanly meta-rule that at least one seed be left for the opponent so he can move.

`board`	`house`	Expected result
`[0, 2, 1, 2]`	`1`	`[0, 0, 0, 0]`
`[2, 0, 4, 1, 5, 3]`	`0`	`[0, 1, 5, 1, 5, 3]`
`[4, 4, 4, 4, 4, 4, 4, 4]`	`2`	`[4, 4, 0, 5, 5, 5, 5, 4]`
`[10, 10, 10, 10]`	`0`	`[0, 14, 13, 13]`
`[4, 10, 4, 1, 1, 1, 4, 4]`	`1`	`[5, 0, 6, 3, 0, 2, 5, 5]`
`[4, 5, 1000, 1, 2, 3]`	`2`	`[204, 205, 0, 201, 202, 203]`
`[0, 0, 0, 6, 1, 1, 2, 1, 2, 1]`	`3`	`[0, 0, 0, 0, 2, 0, 0, 0, 0, 0]`

45. That's enough of you!

```
def remove_after_kth(items, k=1):
```

Given an unsorted list of `items`, some of which may be duplicated, this function should create and return a new list that is otherwise the same as `items`, but only up to `k` occurrences of each element are kept, and all occurrences of that element after the first `k` are discarded.

To achieve this, your function should loop through the `items`, maintaining a dictionary that remembers how many times you have already seen each element. Update this count as you see new elements, and append each element to the `result` list only if its count is still at most `k`.

items	k	Expected result
`[42, 42, 42, 42, 42, 42, 42]`	`3`	`[42, 42, 42]`
`['tom', 42, 'bob', 'bob', 99, 'bob', 'tom', 'tom', 99]`	`2`	`['tom', 42, 'bob', 'bob', 99, 'tom', 99]`
`[1, 2, 3, 4, 5, 4, 3, 2, 1, 2, 3, 4, 5, 4, 3, 2, 1]`	`1`	`[1, 2, 3, 4, 5]`
`[1, 2, 3, 4, 5, 4, 3, 2, 1, 2, 3, 4, 5, 4, 3, 2, 1, 2, 3, 4, 5]`	`3`	`[1, 2, 3, 4, 5, 4, 3, 2, 1, 2, 3, 4, 5, 1, 5]`
`[42, 42, 42, 99, 99, 17]`	`0`	`[]`

Note the counterintuitive and yet completely legitimate edge case of `k==0` that has the well-defined and unambiguously correct answer of an empty list. Once again, an often-missed but very important part of becoming a programmer is learning to see zero as a number.

46. Brussel's choice

```
def brussels_choice_step(n, mink, maxk):
```

This problem is adapted from another jovial video, "The Brussel's Choice" on Numberphile, a site so British you just know a Trevor or a Colin is lurking somewhere. You should watch the first five minutes to get an idea of what is going on, even if the mathematical properties of such numbers are not necessary for you to complete the coding. (The nice thing about not just computing, but all machines, is that understanding how they work from an outsider's point of view doesn't require knowing *why* they work on the inside.) This function should compute the list of all numbers that the positive integer `n` can be converted to by treating `n` as a string and replacing some substring `m` of its digits with the new substring of either `2*m` or `m/2`, the latter substitution allowed only when `m` is even so that dividing it by two produces an integer.

This function should return the list of numbers that can be produced from `n` in a single step. To keep the results manageable, we also impose an additional constraint: the chosen substring m must have between `mink` and `maxk` digits, inclusive. The returned list must contain these numbers in ascending sorted order.

`n`	`mink`	`maxk`	Expected result
`42`	`1`	`2`	`[21, 22, 41, 44, 82, 84]`
`1234`	`1`	`1`	`[1134, 1232, 1238, 1264, 1434, 2234]`
`1234`	`2`	`3`	`[634, 1117, 1217, 1268, 1464, 1468, 2434, 2464]`
`88224`	`2`	`3`	`[44124, 44224, 84114, 84124, 88112, 88114, 88212, 88248, 88444, 88448, 176224, 176424, 816424,816444]`
`123456789`	`7`	`9`	`[61728399, 111728399, 126913578, 146913569, 146913578, 246913489, 246913569, 246913578]`
`123456789`	`1`	`9`	(a list with 65 elements, the last three of which are `[1234567169, 1234567178, 1234567818]`)

47. Cornered cases

```
def count_corners(points):
```

On a two-dimensional integer grid, a **corner** is three points of the form (x, y), $(x, y + h)$ and $(x + h, y)$ for some $h > 0$. Such points form a sort of **triangle** pointing southwest so that the point (x, y) serves as the **tip** and $(x, y + h)$ and $(x + h, y)$ define its axis-aligned **wings** of equal length. For example, the points (2, 3), (2, 7), and (6, 3) form a corner with $x = 2$, $y = 3$, and $h = 4$. On the other hand, the points (2, 3), (2, 1) and (0, 3) do not form a corner, since their group orientation, well-intentioned but unfortunately facing the wrong way, would give a match with $h = -2$. Each point can simultaneously belong to many corners, in all three roles.

Given a list of `points` sorted by their x-coordinates, ties broken by y-coordinates, this function should count how many corners altogether exist among these `points`. Again, you could solve this in "Shlemiel" fashion by brute-forcing all three-element subsets of points, either the manly way of using three levels of nested for-loops or the more elegant `itertools.combinations` sequence decorator for a good time. Instead, iterate through the individual points (x, y) to determine whether they can serve as a chevron tip. Since the upwing's x-coordinate matches the tip's, the inner while loop can check points sequentially from the current point. For each candidate point $(x, y + h)$ to try as this upwing, you need to rapidly determine whether the right wing $(x + h, y)$ that completes this corner is among the remaining `points`.

`points`	Expected result
`[(2, 2), (2, 5), (5, 2)]`	`1`
`[(0, 3), (0, 8), (2, 2), (2, 5), (5, 2), (5, 3)]`	`2`
`[(1, 3), (1, 4), (1, 5), (1, 6), (2, 3), (3, 3), (4, 3)]`	`3`
`[(0, y) for y in range(1000)] +` `[(x, 999-x) for x in range(1, 1000)]`	`999`
`[(x, y) for x in range(200) for y in range(200)]`	`2646700`

As for the corners themselves and how they relate to each other, a curious theorem sets a hard upper bound on how many points you can choose from an n-by-n integer grid without creating any corners. Try this out for yourself with small n, if only to get the feel of the tight wiggle room that will soon see you painted in yet another corner in the twisty maze of possibilities. Conversely, how many corners can you create by placing n points on the plane?

48. McCulloch's second machine

```
def mcculloch(digits):
```

Esoteric programming languages deserve appreciation as works of conceptual performance art. These tar pits aren't designed as practical programming tools, but to point out, in a ha-ha-only-serious manner, how a deceptively simple mechanism secretly resembles an ancient demon sword that unexpectedly grants its wielder the superpowers of **universal computation**.

Introduced by the late Raymond Smullyan in one of his clandestine brain teasers on logic and computability, McCulloch's second machine is a **string rewriting** system for a string of `digits` between one and nine. The rewrite rule applied to the rest of the `digits` (denoted below by `X`) depends on the first digit. Notice how the rule for the leading digit 2 is applied to `X` itself, whereas rules for the leading digits 3 to 5 are applied to `Y` computed from the call `Y=mcculloch(X)`. This function should return `None` whenever either `X` or `Y` is `None`.

Form of `digits`	Formula for the expected result	(computational operation)
`2X`	`X`	quoting
`3X`	`Y + '2' + Y`	concatenation with separator
`4X`	`Y[::-1]`	reversal
`5X`	`Y + Y`	concatenation
anything else	`None`	N/A

Recursive application of these rules produces the following example results:

`digits`	Expected result
`'329'`	`'929'`
`'53231'`	`'3123131231'`
`'4524938'`	`'83948394'`
`'343424859355'`	`'485935524859355248593552485935524859355'`
`'433342717866'`	`'71786627178662717866271786627178662717866271786627178662717866'`

49. That's enough for you!

```
def first_preceded_by_smaller(items, k=1):
```

Find and return the first element of the given list of `items` that is preceded by at least `k` smaller elements in the list. These required `k` smaller elements can be positioned anywhere before the current element, not necessarily immediately before it. If no element satisfying this requirement exists anywhere in the list, this function should return `None`.

Since the only operation performed on individual items is order comparison and no arithmetic occurs during execution, this function works for lists of any element type, as long as the elements are pairwise order-comparable.

items	k	Expected result
`[4, 4, 5, 6]`	2	5
`[42, 99, 16, 55, 7, 32, 17, 18, 73]`	3	18
`[42, 99, 16, 55, 7, 32, 17, 18, 73]`	8	None
`['bob', 'carol', 'tina', 'alex', 'jack', 'emmy', 'tammy', 'sam', 'ted']`	4	'tammy'
`[9, 8, 7, 6, 5, 4, 3, 2, 1, 10]`	1	10
`[42, 99, 17, 3, 12]`	2	None

50. Crab bucket list

```
def eliminate_neighbours(items):
```

Given a list of integer `items` guaranteed to be some **permutation** of positive integers from 1 to `n` where `n` is the length of the list, keep performing the following step until the largest number in the original list gets eliminated: find the smallest number still in the list, and remove from this list both that smallest number and the larger one of its current immediate left and right neighbours. (At the edges, you have no choice which neighbour to remove.) Return the number of steps needed to remove the largest element `n` from the list. For example, given the list `[5, 2, 1, 4, 6, 3]`, start by removing 1 and its current larger neighbour, 4, resulting in `[5, 2, 6, 3]`. The next step removes 2 and its larger neighbour, 6, reaching the goal in two steps.

Removing an element from the middle of the list is expensive and will surely become a bottleneck in the straightforward solution. However, since the `items` are known to be the integers 1 to `n`, we can **simulate** the effect of these expensive removals without ever actually mutating `items`! Define two auxiliary lists, `left` and `right`, to keep track of the current **left neighbour** and the current **right neighbour** of each element. These two lists can be easily initialized with a single loop through the positions of the original `items`.

To remove `i`, make its left and right neighbours, `left[i]` and `right[i]`, figuratively join hands with two assignments `right[left[i]]=right[i]` and `left[right[i]]=left[i]`, as in "Joe, meet Moe; Moe, meet Joe". This noble law of hatchet, axe and saw keeps the elimination times equal for all trees, regardless of their original standing in the forest. (No tree, no problem; all will ultimately be equal in the mulch from the wood chipper's equal bite.)

`items`	Expected result
`[1, 6, 4, 2, 5, 3]`	`1`
`[8, 3, 4, 1, 7, 2, 6, 5]`	`3`
`[8, 5, 3, 1, 7, 2, 6, 4]`	`4`
`[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`	`5`
`range(1, 10001)`	`5000`
`[1000] + list(range(1, 1000))`	`1`

51. What do you hear, what do you say?

`def count_and_say(digits):`

Given a string of digits that is guaranteed to contain only **digit characters** from `'0123456789'`, read that string "out loud" by saying how many times each digit occurs consecutively in the current bunch of digits, and then return the string of digits that you just said out loud. For example, the digits `'222274444499966'` would read out loud as "four twos, one seven, five fours, three nines, two sixes", combined to produce the result `'4217543926'`.

digits	Expected result
`'333388822211177'`	`'4338323127'`
`'11221122'`	`'21222122'`
`'123456789'`	`'111213141516171819'`
`'777777777777777'`	`'157'`
`''`	`''`
`'1'`	`'11'`

As silly and straightforward as this "count-and-say sequence" problem might initially seem, it required the genius of a mathematician of no lesser calibre than the late great John Conway himself not only to notice the tremendous complexity ready to burst out from just below the surface when this operation is repeatedly iterated to produce an infinite sequence of such digit strings, but also to capture that whole mess into a single polynomial equation, as the man himself explains in this Numberphile video. Interested students can also check out the related construct of the infinitely long and yet perfectly self-describing Kolakoski sequence, where only the lengths of each consecutive block of digits are written into the result string, not the actual digits.

52. Bishops on a binge

```
def safe_squares_bishops(n, bishops):
```

The generalized *n*-by-*n* chessboard has been taken over by an army of bishops, each bishop represented as a two-tuple `(row, col)` of the coordinates of the square that the bishop stands on. As in the earlier version of this problem with rampaging rooks, rows and columns are numbered from 0 to *n* – 1. Unlike a chess rook, whose moves are axis-aligned, a chess bishop covers all squares on the same diagonal, arbitrarily far in any of the four diagonal directions. Given the board size `n` and the list of `bishops` on that board, count the number of safe squares that no bishop covers.

To check whether two squares `(r1, c1)` and `(r2, c2)` are reachable from each other in a single bishop move, the expression `abs(r1-r2)==abs(c1-c2)` checks that the horizontal distance between those squares equals their vertical distance, which is both necessary and sufficient for those squares to lie on the same diagonal. This way, you don't have to rewrite the same logic four times; one test handles all four diagonals at once.

n	bishops	Expected result
10	`[]`	100
4	`[(2, 3), (0, 1)]`	11
8	`[(1, 1), (3, 5), (7, 0), (7, 6)]`	29
2	`[(1, 1)]`	2
6	`[(0, 0), (1, 1), (2, 2), (3, 3), (4, 4), (5, 5)]`	18
100	`[(row, (row*row) % 100) for row in range(100)]`	6666

53. Dem's some mighty tall words, pardner

```
def word_height(words, word):
```

The **length** of a `word` is easy enough to define by tallying up its characters. Taking the road less travelled, we define a `word`'s **height** recursively, based on the heights of the two words that make it up. First, any character string that is not one of the actual `words` automatically has zero height. Second, an actual word that cannot be broken into a concatenation of two nonempty actual words has height 1. Otherwise, the height of an actual `word` equals one plus the **larger** of the heights of the two actual words whose combined concatenation it can be expressed as. To make these heights unambiguous for words that can be split into two non-empty subwords in multiple ways, this splitting is done in the best way that produces the tallest final height.

Since the list of `words` is known to be sorted, you can use **binary search** (available as the function `bisect_left` in the `bisect` module) to determine whether some subword is an actual word quickly. Be mindful of the return value of that function whenever the parameter string is not an actual word, and also the edge case of looking for subwords that start with two copies of the letter `z`. (Weird things will happen to people caught driving down the Zzyzx Road.)

`word`	Expected result (using `words_sorted.txt`)
`'hxllo'`	`0`
`'chukker'`	`1`
`'asteroid'`	`7`
`'pedicured'`	`2`
`'enterprise'`	`6`
`'antidisestablishmentarianism'`	`11`
`'noncharacteristically'`	`13`

In the giant wordlist `words_sorted.txt`, every individual letter seems to be a word, which makes these word heights much larger than we expect. Narrowing the dictionary to cover only everyday words would make most of these word heights topple. Finally, words such as `'mankind'` steadfastly thumb their noses at humanity by splitting into nonsense like `'mank'` and `'ind'`. The meaning of those words is a mystery, and that's why so is mankind…

54. Up for the count

```
def counting_series(n):
```

The Champernowne word `12345678910111213141516171819202122 23`…, also known as the **counting series**, is an infinitely long string of digits made up of all positive integers written out in ascending order without any **separators**. This function should return the digit at position `n` of the Champernowne word. For us budding computer scientists, position counting starts from zero.

Of course, the automated tester will throw at your function values of `n` huge enough that the poor souls who construct the Champernowne word as an explicit string will run out of both time and space long before receiving the answer. Instead, this infinite sequence starts with nine single-digit numbers, followed by ninety two-digit numbers, followed by nine hundred three-digit numbers, and so on. This observation gives you a pair of seven-league boots that let their wearer skip prefixes of this series in exponential leaps and bounds, instead of crawling to the desired position one step at a time with the rest of us. Once you reach the block of *k*-digit numbers that contains the position `n`, the digit in that position is best determined with integer arithmetic, perhaps aided by a simple `str` conversion for access to digit positions.

`n`	Expected result
`0`	`1`
`10`	`0`
`100`	`5`
`10000`	`7`
`10**100`	`6`

This favourite problem of many past students leads to several fascinating side quests that may be more difficult but also more rewarding. For example, see the bonus problem 121, “Count your lucky sevens”. Meanwhile, for additional food for thought, let us define “punctual” integers as those that appear inside the Champernowne word for the first time at their first “official appearance”, whereas “early bird” integers such as 9202122 and 456123 (DUCY?) make earlier appearances in the digit sequence as Frankennumbers stitched together from pieces of lesser numbers. Can you think up a rule to determine at a glance whether a given integer is an early bird?

55. Revorse the vewels

```
def reverse_vowels(text):
```

Given a `text` string, create and return a new string constructed by finding all its **vowels** and reversing their order, while keeping all other characters exactly as they were in their original positions. To make the result more presentable, keep the capitalization at each position the same as in the original `text`. For example, reversing the vowels of `'Ilkka'` should produce `'Alkki'` instead of `'alkkI'`. For this problem, vowels are the usual `'aeiouAEIOU'`.

One straightforward way to reverse the vowels is to collect the `text`'s vowels into a separate list and initialize the result as an empty string. Then, loop through all characters of the original `text`. Whenever the current character is a vowel, `pop` one vowel from the end of this list of vowels. Convert that vowel to either uppercase or lowercase depending on the case of the current character, and add the converted vowel to the `result`. Otherwise, add the character from the original `text` to the `result` as it is.

`text`	Expected result
`'Bengt Hilgursson'`	`'Bongt Hulgirssen'`
`'Why do you laugh? I chose death.'`	`'Why da yee loigh? U chasu dooth.'`
`'These are the people you protect with your pain!'`	`'Thisa uro thi peoplu yoe protect weth year peen!'`
`'We had to sacrifice a couple of miners to free Bolivia.'`	`'Wa hid ti socrefeco e ciople uf monars te frii Balovae.'`
`"Who's the leader of the club that's made for you and me? T-R-I-C-K-Y M-O-U-S-E! Tricky Mouse! TRICKY MOUSE! Tricky Mouse! TRICKY MOUSE! Forever let us hold our Hammers high! High! High! High!"`	`"Whi's thi liider af thu clob thot's mude fer yeo end mu? T-R-O-C-K-Y M-I-E-S-U! Trocky Miesu! TROCKY MIESU! Trocky Miesu! TROCKY MIESA! Furovor let as hald uer Hommers hagh! Hegh! Hegh! Hogh!"`

As the examples above show, applying this operation to everyday English sentences often produces pig Latin imitations of other languages. As another idle idea, perhaps a Vietnamese-speaking reader could extend this algorithm to recognize and reverse all diacritical versions of the vowels in that language, if only to see whether this transformation yields unintentional comedy or revelations, divine or mundane, when applied to everyday Vietnamese sentences.

56. Everybody on the floor, do the Scrooge Shuffle

```
def spread_the_coins(coins, left, right):
```

Some positions on the integer line $\mathbb{Z}$ initially contain gold coins, as given in the `coins` list starting from the origin position zero. For example, if `coins` equals `[3, 0, 1, 7]`, there are three coins at position 0, zero coins at position 1, one coin at position 2, and seven coins at position 3. With these example `coins`, all other positions on the integer line $\mathbb{Z}$, both positive and negative, are initially empty and have zero coins.

Any position that contains at least `left+right` coins is **unstable**. For example, if `left==3` and `right==2` in the previous example piles, only position 3 with its seven coins would be unstable. Given the starting configuration of `coins`, your function should repeatedly find some unstable position *i*; it doesn't matter which one. Move exactly `left` coins from position *i* to position *i* – 1, and move exactly right coins to position *i* + 1. Performing this spill in the previous configuration at position 3 produces the new configuration `[3, 0, 4, 2, 2]`. This configuration is stable everywhere since no position contains five or more coins, and this process terminates.

The terminal coin configuration of this back-and-forth dance is unique and is reached in a finite number of steps, regardless of the order you process unstable positions. This function should return a tuple `(start, coins)` where `start` is the leftmost position containing at least one coin. The second component lists the coins that end up in each position from the start to the last position that contains at least one coin. For example, the expected answer from this function for the previous example would be `(0, [3, 0, 4, 2, 2])`. Some initial configurations will end up spilling coins to the negative part of the integer line $\mathbb{Z}$, so your function needs to be able to handle this somehow.

coins	left	right	Expected result
`[0, 0, 2]`	`1`	`1`	`(1, [1, 0, 1])`
`[20]`	`3`	`2`	`(-4, [3, 1, 4, 2, 4, 2, 4])`
`[8, 4]`	`3`	`2`	`(-2, [3, 1, 3, 3, 2])`
`[111, 12, 12]`	`19`	`6`	`(-6, [19, 13, 13, 13, 13, 13, 10, 23, 18])`
`[101]`	`1`	`1`	(a two-tuple whose first element equals `-50` and the second element is a list of 101 copies of `1`)

Discrete math enthusiasts can try proving that the resulting stable position is always unique, regardless of how the unstable positions choose to spread the wealth at each step.

57. Rational lines of action

`def calkin_wilf(n)`

The nodes of the Calkin–Wilf tree, when read in **level order** so that the elements in each level are read from left to right, produce the linear sequence of all possible **positive rational numbers**. Almost as if by magic, this construction guarantees that every positive integer fraction appears exactly once in this sequence. Even more delightfully, this construction makes every rational number appear in its lowest reduced form! To perform the following calculations, you should `import` the data types `Fraction` and `deque` from the `fractions` and `collections` modules.

Your function should return the `n-th` element of this sequence. First, create a new instance of `deque` and `append` the first fraction `1/1` to prime the pump, so to speak, to initiate the production of the values of this sequence. Repeat the following procedure `n` times: pop the fraction currently in front of the queue using the `deque` method `popleft`, extract its numerator and denominator, `p` and `q`, and push the two new fractions `p/(p+q)` and `(p+q)/q` to the back of the queue, in this order. Return the fraction object that was popped in the final round. Your function must return the solution as a `Fraction` object, not as a Python string.

n	Expected result (as `Fraction`)
3	2
10	3/5
10000	11/39
100000	127/713

Once you reach the position `n//2+1`, the queue already contains the result you need, so you can save a hefty chunk of time and space by not actually pushing in any new values. The linked Wikipedia page and other sources provide additional optional shortcuts to jump into the given position faster than sloughing all the way there the hard way, one element at a time.

58. *Verbos regulares*

```
def conjugate_regular(verb, subject, tense):
```

After watching what was clearly one too many episodes of *Narcos* to pass the pandemic lockdown, this author decided to finally learn some basic Spanish, one of the most meme-worthy languages devised for men and their machines. Conjugating verbs *en español* is significantly more complex than in English, where the same information is conveyed with an explicit subject and additional words around that *pinche* infinitive. Frustrated students can only exclaim "*¡Dios mío!"* or "*¡No me gusta!"* and diligently memorize all these mechanistic forms.

Fortunately, regular Spanish verbs whose infinitives end in -ar, -er, or -ir follow systematic rules that make for a good exercise in Python string processing and multiway logic. Given the infinitive of the `verb`, the `subject` as one of *yo*, *tú*, *él*, *ella*, *usted*, *nosotros*, *nosotras*, *vosotros*, *vosotras*, *ellos*, *ellas*, and *ustedes*, and the `tense` as one of four simple indicative tenses: *presente*, *pretérito*, *imperfecto*, and *futuro,* to keep this problem manageable, this function should return the conjugate of the `verb`. Since it is no longer the year 1985 when every ASCII character was stored in a single byte, enough for our daily uphill journey both ways rain or shine (and that is how we liked it, without complaining!), Unicode strings in Python 3 treat accented characters no differently from any other characters, appropriate for the globalized cyberpunk utopia barreling at all of us just around the corner. Therefore, these accents should also be correct in the returned answer.

verb	subject	tense	Expected result
`'añadir'`	`'ellos'`	`'presente'`	`'añaden'`
`'cantar'`	`'yo'`	`'presente'`	`'canto'`
`'bailar'`	`'tú'`	`'pretérito'`	`'bailaste'`
`'decidir'`	`'usted'`	`'futuro'`	`'decidirá'`
`'meter'`	`'ustedes'`	`'imperfecto'`	`'metían'`
`'romper'`	`'ella'`	`'pretérito'`	`'rompió'`
`'escribir'`	`'nosotros'`	`'imperfecto'`	`'escribíamos'`

Este problema es, básicamente, un gran árbol de decisiones. Sin embargo, la regularidad de las reglas de conjugación del español te permite agrupar los casos equivalentes y manejarlos con la misma lógica. Su escala de decisiones debe tener menos de 120 escalones. El patrón no le gusta cuando se desperdician recursos, ¡cabrón!

59. Hippity hoppity, abolish loopity

```
def frog_collision_time(frog1, frog2):
```

A frog hopping along on the infinite two-dimensional grid of integers is represented as a 4-tuple of the form `(sx,sy,dx,dy)` so that `(sx,sy)` is the frog's **starting position** at time $t = 0$, and `(dx,dy)` is its constant **direction vector** for each hop. Time advances in discrete integer steps so that each frog makes one hop at every tick of the clock. At time `t`, the position of that frog is given by the formula `(sx+t*dx,sy+t*dy)`, which can be nimbly evaluated for any `t`.

Given two frogs, `frog1` and `frog2`, guaranteed to stand on different squares initially, return the time when both frogs hop into the same square. If these two frogs never hop into the same square at the same time, this function should return `None`.

This function should not contain any loops; compute the result using conditional statements and integer arithmetic. First, solve the simpler version of this problem of one-dimensional frogs restricted to hop along the one-dimensional line of integers. Once you get that function working correctly, including all its possible edge cases such as both frogs jumping at the same speed, or one or both frogs staying put in the same place with zero speed, use your one-dimensional method as a subroutine to solve for `t` separately for the *x*- and *y*-dimensions in the original problem. Combine these two one-dimensional answers to get the final answer.

frog1	frog2	Expected result
`(0, 0, 0, 2)`	`(0, 10, 0, 1)`	`10`
`(10, 10, -1, 0)`	`(0, 1, 0, 1)`	`None`
`(0, -7, 1, -1)`	`(-9, -16, 4, 2)`	`3`
`(-28, 9, 9, -4)`	`(-26, -5, 8, -2)`	`None`
`(-28, -6, 5, 1)`	`(-56, -55, 9, 8)`	`7`
`(620775675217287, -1862327025651882, -3, 9)`	`(413850450144856, 2069252250724307, -2, -10)`	`206925225072431`

60. In space, no one can hear you bounce

```
def reach_corner(x, y, n, m, aliens):
```

A lone chess bishop stands on square (*x*, *y*) of a curious *n*-by-*m* chessboard floating in outer space, covered with completely frictionless magic ice created by Jack Frost, who snuck by your room while you slept and pinched you in the belly. (True story. Yeah, and once again, zero-based indexing.) This board is not necessarily square, but can be any rectangle of integer dimensions. The board is surrounded by bouncy walls, like an air hockey table. Some squares on the board contain `aliens` that use both their big mouths and little mouths to make it game over, game over, man, for anyone who enters.

Rapid motion knife tricks won't help the bishop here. Instead, he must reach any one of the four exits at the board's corners. Like patiently watching the screensaver on an old DVD player, the bishop must reach any one of these corners in a single move. The bishop can initially propel himself to any of the four diagonal directions. After this initial impulse, the bishop can no longer control its movement on the frictionless ice. Still, it will keep sliding toward whatever end the current direction vector and wall bounces take it. The bishop can even repeat the same cycle of squares forever. (Your function, mindlessly doing an analogous dance, just on the far more complex "surface" of computer state configurations, should not follow suit, but nuke the whole computation from the orbit, just to be sure.)

This function should determine whether at least one starting direction leads the bishop to the safety of any one of the four corners, avoiding all the `aliens` along the way.

x	y	n	m	aliens	Expected result
0	2	5	5	[]	False
4	4	9	9	[(0, 0), (0, 8), (8, 0), (8, 8)]	False
1	1	1000	2	[(0, 0), (0, 1), (999, 0)]	True
1	1	1000	2	[(0, 0), (0, 1), (999, 1)]	False
3	2	4	4	[(1, 2), (0, 1)]	False
3	2	5	4	[(2, 2), (1, 4)]	True

61. Nearest polygonal number

```
def nearest_polygonal_number(n, s):
```

Any positive integer $s > 2$ defines an infinite sequence of ***s*–gonal numbers** whose i:th element is given by the formula $((s - 2) \times i^2 - (s - 4) \times i)/2$, as explained on the Wikipedia page "Polygonal Number". In this formula, which was devised not by a computer scientist but by a more ordinary person, positions start at 1, not 0. We also rewrote the formula using the letter i to denote the position, since n already has a different meaning in this problem. (As variable names, letters n and m should always denote unknown fixed integers anyway, whereas position indices that will move back and forth are better denoted by letters i and j.)

For example, the sequence of "octagonal numbers" that springs forth from $s = 8$ shoots towards infinity as 1, 8, 21, 40, 65, 96, 133, 176, … Given the number of sides denoted by s and an arbitrary positive integer n, this function should return the s–gonal integer closest to n. If n falls exactly halfway between two s–gonal numbers, return the smaller one of those two numbers.

`n`	`s`	Expected result
`5`	`3`	`6`
`27`	`4`	`25`
`450`	`9`	`474`
`10**10`	`42`	`9999861561`
`10**100`	`91`	`100000000000000000000000000000000000000000000000000416332753518329478897755794704334003003544212420356`

As you can see from the last row of the expected results table, this function must be efficient even for gargantuan values of n. Harness **repeated halving** to pull your wagon to this promised land with a clever application of **binary search**. Start with two integers `a` and `b` wide enough that they satisfy `a<=i<=b` for the currently unknown position `i` that the nearest polygonal number is stored in. (Just initialize these as `a=1` and `b=2`, and keep squaring `b` until the s–gonal number in that position gets too big. It's not like these initial bounds need to be super accurate.) From there, compute the **midpoint** position `(a+b)//2`, and look at the element in that position. Depending on how that midpoint element compares to `n`, bring either `b` or `a` to the midpoint position. Continue this until the gap has become small enough so that `b-a<2`, at which point one final comparison tells you the correct answer.

62. Don’t worry, we will fix it in the post

```
def postfix_evaluate(items):
```

When arithmetic expressions are given in the familiar **infix** notation 2 + 3 × 4, parentheses change the evaluation order from the usual PEMDAS order determined by each operator's precedence and associativity. The alternative **postfix** notation (also known as Reverse Polish Notation, for all you “simple Poles on a complex plane”) may first look weird to people accustomed to the conventional infix. However, postfix notation is easier for machines because it lets you encode any intended evaluation order without parentheses!

A postfix expression is given as a list of `items` that can be either individual integers or one of the strings `'+'`, `'-'`, `'*'`, and `'/'` to denote the four basic arithmetic operators. To evaluate a postfix expression using a simple linear loop, use an ordinary list as an initially empty **stack**. Loop through the `items` one by one, from left to right. Whenever the current item is an integer, just `append` it to the end of the list. Whenever the current item is one of the four arithmetic operations, `pop` two items from the end of the list to perform that operation on, and `append` the result back to the list. Assuming `items` is a legal postfix expression (as guaranteed in this problem), you don’t need to perform any error detection or recovery; once all items have been processed, the lone number left in the stack is the final answer.

To avoid the intricacies of floating-point arithmetic, you should perform the division operation using the Python integer division operator `//` that truncates the result to the integer part. Also, to avoid a crash from dividing by zero, this problem includes an artificial (yet mathematically sound) rule: division by zero returns 0 instead of crashing.

`items`	(equivalent infix)	Expected result
`[2, 3, '+', 4, '*']`	(2 + 3) × 4	`20`
`[2, 3, 4, '*', '+']`	2 + (3 × 4)	`14`
`[3, 3, 3, '-', '/', 42, '+']`	3 / (3 – 3) + 42	`42`
`[7, 3, '/']`	7 / 3	`2`
`[1, 2, 3, 4, 5, '*', '*', '*', '*']`	1 × 2 × 3 × 4 × 5	`120`

By adding more operators and another auxiliary stack, you can build a Turing-complete programming language on top of postfix evaluation. Interested students can sally forth to the Wikipedia page “Forth” to learn more about this ingenious **concatenative programming** language.

63. Fractran interpreter

```
def fractran(n, prog, giveup=1000):
```

The late great John Conway is best known for The Game of Life, not to be confused with the earlier family board game of the same name. Since that achievement eclipsed basically all Conway's other wacky, original creations, we ought to rectify this by giving these less-appreciated creations an occasional spin in the paparazzi lights and red-carpet fame.

This function interprets the esoteric programming language FRACTRAN, a pun combining "fraction" with FORTRAN. (Things used to be all uppercase back when real scientists wore horn-rimmed glasses and came to work driving cars equipped with tail fins, with occasional dollar signs interspersed as visual separators to keep the perfumed princes of the military-industrial complex happy.) A program written in such an esoteric form consists of a list of positive integer fractions, given in this problem as tuples of numerator and denominator. Of course, you should use the `Fraction` data type in the `fractions` module to perform these computations exactly.

Given a positive integer `n` as its start state, the next state is the product `n*f` for the first fraction listed in `prog` for which `n*f` is an exact integer. That integer then becomes the new state for the next round. Once `n*f` is not an integer for any of the fractions `f` listed in `prog`, the execution terminates. Your function should compute the sequence of integers produced by the given FRACTRAN program, with a forced termina… ("*Oi, m8, Ol' Connie was British!*"), Sorry, forced **halting** takes place after `giveup` steps, should the execution not have halted by itself by then. (Any **Turing-complete** language that is capable of arbitrary computation is also capable of infinite loops.)

n	prog	giveup	Expected result
3	`[(94, 51)]`	10	`[3]`
7	`[(7, 3), (12, 7)]`	10	`[7, 12, 28, 48, 112, 192, 448, 768, 1792, 3072, 7168]`
10	`[(3, 6), (9, 11), (4, 5), (6, 2), (7, 1)]`	10	`[10, 5, 4, 2, 1, 3, 9, 27, 81, 243, 729]`
5	`[(108, 125), (90, 45), (122, 57), (75, 158)]`	10	`[5, 10, 20, 40, 80, 160, 320, 640, 1280, 2560, 5120]`

64. Permutation cycles

```
def permutation_cycles(perm):
```

In this problem, a **permutation** is a list of n elements that contains every integer from 0 to $n - 1$ exactly once. For example, `[5, 2, 0, 1, 4, 3]` is one of the 6! = 720 permutations for $n = 6$. Each permutation can be viewed as a **bijective** function that maps each position from 0 to $n - 1$ to a unique position. For example, the previous permutation maps position 0 to 5, position 1 to 2, 2 to 0, and so on.

As explained in the YouTube video "Cycle Notation of Permutations" by Socratica, a cycle is a list of positions where each position maps to the next, with the last element looping back to the first. Positions inside a cycle are treated in a cyclic "necklace" fashion so that `[1, 2, 3]`, `[2, 3, 1]` and `[3, 1, 2]` are **rotationally equivalent** ways to represent the same three-cycle, distinct from another possible cycle of these elements representable as either `[1, 3, 2]`, `[3, 2, 1]` or `[2, 1, 3]`. To construct the cycle that position i belongs to, start at $j = i$ and keep moving from the current position j to its successor, `perm[j]`, until you return to the original position `i`. The cycles of the above permutation are `[0, 5, 3, 1, 2]` and `[4]`, the second "unicycle" being a **singleton** since this permutation maps position 4 back to itself.

This function should list each cycle starting from its highest element, and the cycles must be listed in increasing order of their starting positions. For example, the previous example cycles `[4]` and `[5, 3, 1, 2, 0]` must be listed in this order, since 4 < 5. Furthermore, instead of returning a list of cycles, encode these cycles as a flat list with no separators. (Amazingly, this run-on **standard representation** of the permutation still allows unique reconstruction of the original permutation and its cycles!)

perm	Expected result
`[0, 1, 2, 3]`	`[0, 1, 2, 3]`
`[3, 2, 1, 0]`	`[2, 1, 3, 0]`
`[5, 2, 0, 1, 4, 3]`	`[4, 5, 3, 1, 2, 0]`
`[2, 4, 6, 5, 3, 1, 0]`	`[5, 1, 4, 3, 6, 0, 2]`
`[5, 3, 0, 1, 4, 2, 6]`	`[3, 1, 4, 5, 2, 0, 6]`
`[0, 9, 7, 4, 2, 3, 8, 5, 1, 6]`	`[0, 7, 5, 3, 4, 2, 9, 6, 8, 1]`

65. Whoever must play, cannot play

```
def subtract_square(queries):
```

Two players play a game of "Subtract a square" starting from a positive integer. On his turn to play, each player must subtract some **square number** (1, 4, 9, 16, 25, 36, 49, …) from the current number so that the result does not become negative. Under the **normal-play convention**, the player who moves to zero wins, as their opponent cannot move. This game is impartial because the same moves are available to the player whose turn it is, unlike partisan games such as chess, where players can move only pieces of their colour.

This and similar combinatorial games can be solved with recursive equations. A state is **hot** (or "winning") if at least one legal move from that state leads to a **cold** (or "losing") state. Yes, that's the entire definition, including the base case, even if you have to squint a bit to see it.

Since the heat value of each state `n` is determined by the heat values of states lower than `n`, we might as well combine all these heat computations of states into a single **batch job**. This function should return a list of results for the given `queries`, with `True` meaning hot and `False` meaning cold. You should compute the heat values for all states as a single list, built up from left to right. The heat value computation at each state can then read the previously computed heat values of the lower-numbered states to which there is a legal move from the current state.

`queries`	Expected result
`[7, 12, 17, 24]`	`[False, False, False, True]`
`[2, 3, 6, 10, 14, 20, 29, 39, 57, 83, 111, 149, 220, 304, 455, 681]`	`[False, True, True, False, True, False, True, False, False, True, True, True, True, True, True, True]`
`[7, 12, 17, 23, 32, 45, 61, 84, 120, 172, 251, 345, 510, 722, 966, 1301, 1766, 2582, 3523, 5095, 7812, 10784, 14426, 20741]`	`[False, False, False, True, True, True, True, True, True, True, True, True, False, False, True, True, True, True, True, True, True, True, True, True]`

(In the **misère** version of the game, which has otherwise identical rules but where you win by losing the original game by reaching zero, these definitions would be adjusted accordingly. However, remember that in this type of situation, reversed stupidity is not intelligence.)

66. ztalloc ecneuqes

```
def ztalloc(shape):
```

The famous Collatz sequence was used in the lectures as an example of a situation where a `while`-loop is necessary, since it is impossible to predict in advance how many steps are needed to reach the goal from the given starting value. (In fact, no finite upper bound is known, as any such bound, however loose or pessimistic, would immediately settle the still-open Collatz conjecture in the affirmative.) The answer is the list of integers the sequence visits before terminating at its goal.

However, the Collatz sequence can also be viewed in a binary fashion depending on whether each value steps **up** ($3x+1$) or **down** ($x/2$) from the previous value, denoting these steps with `'u'` and `'d'`, respectively. Starting from `n=12`, the sequence `[12, 6, 3, 10, 5, 16, 8, 4, 2, 1]` would have the step shape `'ddududddd'`. This function should, given the step `shape` as a string made of letters `u` and `d`, determine which starting value for the Collatz sequence produces that exact `shape` when run to the end of that sequence. Unlike an ordinary Collatz sequence, which must stop when it reaches 1, our sequences are allowed to visit that value multiple times. For example, the sequence `[4, 2, 1, 4, 2, 1]` would have the step shape `'ddudd'`.

However, this function must also recognize that some shape strings are impossible as entailed by the Collatz transition rules, and correctly return `None` for all such shapes. You should start from the goal state 1, and perform the given transitions in reverse. Along the way, you have to ensure that your function does not accept moves that would be illegal in the original forward-going Collatz sequence. In particular, you can apply the $3x+1$ rule only to odd values of x, no matter how tempting the siren song of the successor $3x+1$ might be.

shape	Expected result
`'ddudd'`	4
`'udddd'`	5
`'dudududdudddudddd'`	14
`'uduuudddd'`	None
`'d'`	2
`'uudddddddd'`	None
`'duuudddddddd'`	None

67. The solution solution

```
def balanced_centrifuge(n, k):
```

Another Numberphile video inspired this problem, this time “The Centrifuge Problem.” Alternatively, students whose eyes read faster than their ears can check out Matt Baker’s post, “The Balanced Centrifuge Problem.” A **centrifuge** has `n` identical slots, each big enough to fit one test tube. To prevent the centrifuge from wobbling, place `k` identical tubes in these slots so their combined **center of gravity** lies precisely at the center of the centrifuge.

Balancing `k` test tubes into `n` slots is possible if and only if both `k` and `n-k` can be expressed as sums of **prime factors** of `n`, with repetitions allowed. For example, when `n` equals 6, whose prime factors are 2 and 3, the centrifuge can hold 0, 2, 3, 4 (= 2 + 2) or 6 (= 3 + 3) test tubes. However, there is no way to balance 1 or 5 test tubes in six slots. Although 5 = 2 + 3 satisfies the first part of the rule, there is no way to balance the remaining empty slot, which is required despite the counterintuitive fact that empty slots weigh nothing, and there somehow ought to be nothing to balance!

This function does not need to construct a balanced configuration of `k` test tubes, but merely determine whether at least one balanced configuration exists for the given `n` and `k`.

`n`	`k`	Expected result
`6`	`3`	`True`
`7`	`0`	`True`
`15`	`8`	`False`
`222`	`107`	`True`
`1234`	`43`	`False`

68. Reverse ascending sublists

```
def reverse_ascending_sublists(items):
```

Create and return a new list that contains the same elements as the `items` list argument, but where the order of the elements inside every **maximal strictly ascending** sublist has been reversed. Note the modifier "strictly" used in the previous sentence to require each element to be greater than the previous element, not merely equal to it.

In the table below, different colours highlight the maximal strictly ascending sublists for readability, and are not part of the actual argument object given to the Python function.

`items`	Expected result
`[1, 2, 3, 4, 5]`	`[5, 4, 3, 2, 1]`
`[5, 7, 10, 4, 2, 7, 8, 1, 3]`	`[10, 7, 5, 4, 8, 7, 2, 3, 1]`
`[5, 4, 3, 2, 1]`	`[5, 4, 3, 2, 1]`
`[5, 5, 5, 5, 5]`	`[5, 5, 5, 5, 5]`
`[1, 2, 2, 3]`	`[2, 1, 3, 2]`

69. Brangelin–o–matic for the people

```
def brangelina(first, second):
```

Combining the first names of beloved celebrity couples into a catchy shorthand for mass media consumption is easy to automate. Start by counting how many maximal groups of consecutive vowels (*aeiou*; to keep this problem simple, the letter *y* is always a consonant) there are in the first name. For example, `'brad'` and `'jean'` have one vowel group, `'jeanie'` and `'britain'` have two, and `'angelina'` and `'alexander'` have four. Note that a vowel group can contain more than one vowel, such as the word `'queueing'` with an entire fiver.

If the `first` name has only one vowel group, keep only the consonants before that group and lose everything else. For example, `'ben'` becomes `'b'`, and `'brad'` becomes `'br'`. Otherwise, if the `first` word has $n > 1$ vowel groups, keep everything before the **second-to-last** vowel group $n - 1$. For example, `'angelina'` becomes `'angel'` and `'alexander'` becomes `'alex'`. Concatenate that string with the string that you get by removing all consonants from the beginning of the `second` name. All `first` and `second` names given to this function are guaranteed to consist of the 26 lowercase English letters, and each name will have at least one vowel and one consonant somewhere.

first	second	Expected result
`'brad'`	`'angelina'`	`'brangelina'`
`'angelina'`	`'brad'`	`'angelad'`
`'sheldon'`	`'amy'`	`'shamy'`
`'amy'`	`'sheldon'`	`'eldon'`
`'frank'`	`'ava'`	`'frava'`
`'britain'`	`'exit'`	`'brexit'`

These rules don't always produce the best result. For example, `'ross'` and `'rachel'` combine into `'rachel'` instead of the more informative `'rochel'`. The reader can later develop more advanced rules that cover a wider range of name combinations and special cases. (A truly advanced set of rules, perhaps trained on a real-world news corpus with **deep learning** techniques using connectionist neural networks that learn to recognize the implicit **semantic** content, might then combine `'donald'` and `'hillary'` into either `'dollary'` or `'dillary'`, depending on the intended tone and the target audience.)

70. Line with the most points

```
def line_with_most_points(points):
```

A point on the two-dimensional integer grid $\mathbb{Z}^2$ is given as a two-tuple of x- and y-coordinates, for example, `(2,5)` or `(10,1)`. As Euclid originally postulated, any two distinct points on the plane define exactly one line through both points. (In differently shaped spaces made of finger-quotes "points" and "lines", different rules and their consequences apply.) Of course, this unique line, being shamelessly infinite in both directions, will also pass through an infinite number of other points on that same plane. However, that claim might not be as easy to prove from first principles.

Given a list of `points` on the grid of integers, find the line that contains the largest number of points from this list. To ensure a unique result for pseudorandom fuzz testing, this function should not return the line itself, but only the count of points on that line. The list of `points` is guaranteed to contain at least two points, and all points are distinct, but they are not given in any specific order.

To get started, consult the example program `geometry.py` for the **cross product** function that can be used to quickly determine whether three given points on the plane are **collinear**. Using this operation as a subroutine, the rest of the algorithm can operate at a higher level of abstraction.

`points`	Expected result
`[(42, 1), (7, 5)]`	`2`
`[(1, 4), (2, 6), (3, 2), (4, 10)]`	`3`
`[(x, y) for x in range(10) for y in range(10)]`	`10`
`[(3, 5), (1, 4), (2, 6), (7, 7), (3, 8)]`	`3`
`[(5, 6), (7, 3), (7, 1), (2, 1), (7, 4), (2, 6), (7, 7)]`	`4`

This problem could always be inefficiently **brute-forced** with three nested loops, but the point (heh) of this problem is not to do too much more work than you really need to. With this function, you could next numerically investigate the problem in the middle panel of this XKCD comic. (When generating each random non-self-intersecting path, **rejection sampling** would be the easiest way to ensure that your choices among all possible such paths are unbiased.)

71. Om nom nom

```
def cookie(piles):
```

The beloved Cookie Monster from *Sesame Street* has stumbled upon a table with `piles` of cookies, each pile a positive integer. However, the Count's monomaniacal obsessiveness has recently escalated to a whole new level of severity. The Count insists that these cookies must be eaten in the smallest possible number of moves. Each move chooses one of the remaining pile sizes `p`, and removes `p` cookies from every pile that contains at least `p` cookies (thus eradicating all piles with exactly `p` cookies), and leaves all smaller piles as they were.

Since the Count also has an unhealthy obsession with order and hierarchies, he expects these moves to be done in decreasing order of `p`, as shown in the third column of the table below. This function should return the fewest number of moves needed for Cookie Monster to scarf down these cookies. For the given `piles`, your function should loop through its possible moves and recursively solve the problem for the new piles that result from applying each move to the `piles`. The optimal solution for `piles` is then simply one plus the optimal subproblem solution.

`piles`	Expected result	(optimal moves)
`[1, 2, 3, 4, 5, 6]`	`3`	`[4, 2, 1]`
`[2, 3, 5, 8, 13, 21, 34, 55, 89]`	`5`	`[55, 21, 8, 3, 2]`
`[1, 10, 17, 34, 43, 46]`	`5`	`[34, 9, 8, 3, 1]`
`[11, 26, 37, 44, 49, 52, 68, 75, 87, 102]`	`6`	`[37, 31, 15, 12, 7, 4]`
`[2**n for n in range(10)]`	`10`	`[512, 256, 128, 64, 32, 16, 8, 4, 2, 1]`

The above version of the Cookie Monster problem has been streamlined a bit to keep this problem simple enough to solve here. The more general formulation lets Cookie Monster choose any subset of remaining piles and remove the same number of cookies from each pile in that subset. Interested students can check out the article "On the Cookie Monster Problem" about the subtle complexities of the general problem formulation.

72. Autocorrect for sausage fingers

```
def autocorrect_word(word, words, df):
```

In this day and age, all you whippersnappers are surely familiar with **autocorrect** that replaces a non-word with the “closest" real word in the dictionary. Many techniques exist to guess more accurately what the user intended to write. Many older people, like your instructor, who are already baffled by the technological doodads and thingamabobs in your TikTok videos, often inadvertently press a virtual key near the intended one. For example, when the non-existent word is `'cst'`, the intended word is more likely to be `'cat'` than `'cut'`, assuming the text was entered on a standard QWERTY keyboard where `'a'` and `'s'` are adjacent.

Given a `word`, a list of `words`, and a **distance** function `df` that tells how “far” the first key is from the second (for example, `df('a','s')==1` and `df('a','a')==0`), return the word in the list of `words` that has the same number of letters and minimizes the total distance, measured as the sum of the distances between the keys in the same positions. If several words are equidistant from the `word`, return the first of these words in the dictionary order.

`word`	Expected result (on QWERTY keyboard)
`'qrc'`	`'arc'`
`'jqmbo'`	`'jambo'`
`'hello'`	`'hello'`
`'interokay'`	`'interplay'`

More advanced autocorrect techniques use surrounding context to suggest the best replacement, since not all words are equally likely given the rest of the sentence. For example, the misspelling `'urc'` should be corrected very differently in the sentence “The gateway was an urc made of granite blocks” than in “The brave little hobbit swung his sword at the smelly urc.” Similarly, even though `'lobe'` is a perfectly valid word by itself, “I lobe you!” is most likely a typo; and whoever wrote just one word `'hellonthere'` probably meant to write two but missed the space bar at the bottom row… or it could even be three words, assuming that the space bar detection was somehow broken.

73. Uambcsrlne the wrod

```
def unscramble(words, word):
```

Smdboeoy nteoicd a few yreas ago taht the lretets isndie Eisgnlh wdors can be ronmaldy slmecbrad wouhtit antifecfg tiehr rlaibdiatey too mcuh, piovredd that you keep the frsit and the last lteters as tehy were. Gevin a lsit of `wdors` gtuaraened to be soterd, and one serlcmbad `wrod`, tihs fctounin shulod rterun the list of wrdos taht cloud hvae been the orgiianl word taht got seambclrd, and of csorue retrun taht lsit wtih its wdros in apcabihaetll oerdr to enrsue the uaigntbmuiy for atematoud testing. For the vast maitjory of ceass, this list wlil catnoin only one wrod.

`wrod`	Etecxepd rssuelt (unsig the wrosldit `wdors_sroted.txt`)
`'tartnaas'`	`['tantaras', 'tarantas', 'tartanas']`
`'aierotsd'`	`['asteroid']`
`'ksmrseah'`	`['kersmash']`
`'bttilele'`	`['belittle', 'billette']`

Writing the filter to transform the given plaintext to the above scrambled form is also a good little programming exercise in Python. This leads us to a useful estimation exercise: how long would the plaintext have to be for you to write such a filter yourself instead of scrambling the plaintext by hand the way you would if you needed to scramble just one word?

In general, how many times do you think you need to solve a particular problem until it becomes more efficient to design, write and debug a Python script to do it? Would your answer change if it turned out that millions of people around the world have the same problem, silently shaking their fists at the heavens while hoping their Strong Man saviour will ride in on his mighty horse to scratch that itch by automating this repetitive, error-prone task? Could the very reader of this sentence be the One the ancient prophecies have only dared to hint at, coming to solve all our computational problems?

74. Substitution words

```
def substitution_words(pattern, words):
```

Given a list of `words` (again, guaranteed to consist of the 26 lowercase English letters only) and a `pattern` that consists of uppercase English characters (possibly repeated), this function should return a list of precisely those words that match the `pattern` in the sense that there exists a substitution from uppercase letters to lowercase letters that turns the `pattern` into that word. To keep the result lists reasonably short, we require this substitution to be **injective**, so that no two different uppercase letters in the `pattern` map to the same lowercase letter.

For example, the word `'akin'` matches the pattern `'ABCD'` with the substitutions `A`→`a`, `B`→`k`, `C`→`i` and `D`→`n`. However, the word `'area'` would not match that pattern, since both pattern characters `A` and `D` would have to map to the same letter `a`, violating the requirement for an injective mapping.

`pattern`	Expected result (using the wordlist `words_sorted.txt`)
`'ABBA'`	`['abba', 'acca', 'adda', 'affa', 'akka', 'amma', 'anna', 'atta', 'boob', 'deed', 'ecce', 'elle', 'esse', 'goog', 'immi', 'keek', 'kook', 'maam', 'noon', 'otto', 'peep', 'poop', 'sees', 'teet', 'toot']`
`'ABABA'`	`['ajaja', 'alala', 'anana', 'arara', 'ululu']`
`'CEECEE'`	`['beebee', 'booboo', 'muumuu', 'seesee', 'soosoo', 'teetee', 'weewee', 'zoozoo']`
`'ABCDCBA'`	`['adinida', 'deified', 'hagigah', 'murdrum', 'reifier', 'repaper', 'reviver', 'rotator', 'senones']`
`'DEFDEF'`	76 words, the first three of which are `['akeake', 'atlatl', 'barbar']`

(`K`, `NM`, `AVG`, `GLEN`, `FURAN`, `DOPANT`, `MOLDING`, `NODULIZE`, `UNWITCHED`, `DITHYRAMBS`, `SULPHOAMIDE`, `NEUROBLASTIC`, `SULPHOCYANIDE`, `PRODUCIBLENESS`, `ETHNOBIOLOGICAL`, `INTELLECTUALIZED`, `INTERNATIONALIZES`, `TERRITORIALIZATION`, `ELECTROCARDIOGRAPHS`, `UNCONTROVERTIBLENESS`. Yep. Apparently, repeating characters doesn't pay off until you are forced to. I should have thought about this before I created an algorithm to verify it.)

75. Manhattan skyline

```
def manhattan_skyline(towers):
```

Computational geometry is essentially geometry that uses only integer arithmetic without any of those nasty transcendental functions. (Not only is that an actual place, but I have come here to lead my people into that promised land. God created the integers; the rest is man's work.) The **Manhattan skyline problem**, a classic chestnut of computational geometry, is best illustrated with pictures and animations, as those on the page "The Skyline problem". Given a list of rectangular `towers` as tuples `(s,e,h)` where `s` and `e` are the start and end *x*-coordinates with `e>s` so that tenement is not fit only for immigrants fresh off the Flatland plane (surely Rodgers and Hammerstein would have turned this backstory into a delightful little ditty) and `h` is the height of that tower, compute the total visible area of the `towers`. Be careful not to **double-count** the area of partially overlapping towers. All `towers` share the same flat ground baseline.

The classic solution uses a **sweep-line technique** that starts by creating a list of the x-coordinates where something relevant to the problem occurs. In this problem, the relevant *x*-coordinates are where a tower starts or ends. Next, loop through these `towers` in ascending order, updating your computation for the interval between the current relevant *x*-coordinate and the previous one. In this particular problem, you need to maintain a **list of active towers** so that tower `(s,e,h)` becomes active when `x==s`, and becomes inactive again when `x==e`. Within each interval, add only the tallest active tower to the area summation.

towers	Expected result
`[(0, 5, 2), (1, 3, 4)]`	`14`
`[(3, 4, 1), (1, 6, 3), (2, 5, 2)]`	`15`
`[(-10, 10, 4), (-8, -6, 10), (1, 4, 5)]`	`95`
`[(s, 1000-s, s) for s in range(500)]`	`249500`
`[(s, s+(s*(s-2))%61+1, max(1,s*(s%42))) for s in range(100000)]`	`202121725069`

The automated tester script will produce random start and end coordinates from an exponentially increasing scale. This puts the kibosh on solving this problem with the inferior method that builds up the list of all positions from zero up to the maximum end coordinate in `towers`, loops through the `towers` to update the tallest tower height at each position with an inner loop, and finally sums up these position height strips for the total area. As in your calculus class, symbolic summation beats Riemann summation any day, even when the latter technique happens to produce the correct result in the domain of discrete calculus.

76. Count overlapping disks

```
def count_overlapping_disks(disks):
```

Right on the heels of the previous Manhattan skyline problem, here is another classic problem in a similar spirit, solved efficiently with a sweep-line **algorithm**. Given a list of `disks` on the two-dimensional plane represented as tuples `(x,y,r)` so that `(x,y)` is the **center point** and `r` is the **radius** of that disk, count how many pairs of disks **intersect** so that their areas have at least one point in common. Two disks `(x1,y1,r1)` and `(x2,y2,r2)` intersect if and only if they satisfy the **Pythagorean** inequality `(x2-x1)**2+(y2-y1)**2<=(r1+r2)**2`.

Note how this precise formula runs on pure integer arithmetic whenever its arguments are integers, so that no square roots or any other irrational numbers will gum up the works with all that decimal noise. (This formula also uses the operator `<=` instead of `<` to count two **kissing** disks as an intersecting pair. One is not zero, *ja*?)

For this problem, crudely looping through all possible pairs of disks would work, but it would also become horrendously inefficient for large lists. However, a sweep line algorithm can solve this problem not just **effectively** but also **efficiently** (a crucial but often overlooked "eff-ing" distinction) by examining far fewer pairs of disks. Again, sweep through the space from left to right over all relevant x-coordinate values, maintaining the set of **active** disks at each step. Each disk `(x,y,r)` enters the active set when the vertical sweep line reaches `x-r` and leaves when it reaches `x+r`. At the time when a disk enters the active set, you need to look for its intersections only in this active set.

`disks`	Expected result
`[(0, 0, 3), (6, 0, 3), (6, 6, 3), (0, 6, 3)]`	`4`
`[(4, -1, 3), (-3, 3, 2), (-3, 4, 2), (3, 1, 4)]`	`2`
`[(-10, 6, 2), (6, -4, 5), (6, 3, 5), (-9, -8, 1), (1, -5, 3)]`	`2`
`[(x, x, x//2) for x in range(2, 101)]`	`2563`

77. Ordinary cardinals

```
def sort_by_digit_count(items):
```

We can sort meaningfully by arbitrary comparison criteria, as long as those criteria satisfy the mathematical requirements of a **total ordering** relation. To play around with this concept and grok it, let us define a wacky ordering relation on positive integers where, for any two integers, the one that contains the digit 9 more times is considered larger, regardless of magnitude or other digits. For example, 99 > 12345678987654321 > 10^{100000} in this ordering. If both integers contain the digit 9 the same number of times, the comparison proceeds to the next lower digit, 8, and so on until it finds the first distinguishing digit. If both integers contain every digit from 9 to 0 the same number of times, the ordinary integer order comparison determines their mutual ordering.

`items`	Expected result
`[9876, 19, 4321, 99, 73, 241, 111111, 563, 33]`	`[111111, 33, 241, 4321, 563, 73, 19, 9876, 99]`
`[111, 19, 919, 1199, 911, 999]`	`[111, 19, 911, 919, 1199, 999]`
`[1234, 4321, 3214, 2413]`	`[1234, 2413, 3214, 4321]`
`list(range(100000))`	(a list of 100,000 elements whose first five elements are `[0, 1, 10, 100, 1000]` and the last five are `[98999, 99899, 99989, 99998, 99999]`)

As explained in your friendly local discrete math course, the above relation is transitive and antisymmetric, and therefore an ordering relation for integers. Outside its absurdist entertainment value, it is useless in The Real World, though. The ordinary ordering of ordinals that we tend to grant the status of being almost some sort of law of nature is useful because it plays along nicely with useful arithmetic operations such as addition (for example, $a + c < b + c$ for any c whenever $a < b$) to make these operations more powerful in this symbiotic relationship.

78. Count divisibles in range

```
def count_divisibles_in_range(start, end, n):
```

Let us take a breather with a problem simple enough that its solution requires only a couple of conditional statements and some integer arithmetic, no loops or anything fancier. The difficulty is finding conditions that cover all cases correctly, including tricky edge and corner cases, without being off by one. Given three integers `start`, `end`, and `n` so that `start<=end`, count how many integers between `start` and `end`, inclusive, are divisible by `n`. Sure, the distinguished gentleman *could* solve this problem as a one-liner with the lazy generator comprehension

```
return sum(1 for x in range(start, end+1) if x % n == 0)
```

But of course, the author's thumb once again tactically pushes the automated tester's scale, ensuring that anyone trying to solve this problem with such a blunt, unimaginative tool will soon find themselves running out of time! Your code should have **no loops at all**, but use only integer arithmetic and conditional statements to root out the truth. Beware of edge cases and off-by-one ticks lurking in the bushes. Note that either `start` or `end` can be negative or zero, but `n` is guaranteed to be greater than zero. You should also be aware of the mathematically incorrect behaviour of Python's remainder operator `%` when applied to negative numbers.

start	end	n	Expected result
7	28	4	6
-77	19	10	9
-19	-13	10	0
1	10**12 - 1	5	199999999999
0	10**12 - 1	5	200000000000
0	10**12	5	200000000001
-10**12	10**12	12345	162008911

79. Bridge hand shape

`def bridge_hand_shape(hand):`

In the card game of bridge, each player receives a hand of exactly thirteen cards. The hand **shape** is the distribution of these cards into the four suits, in the order **spades, hearts, diamonds, and clubs**; these words in English in descending alphabetical order. Given a bridge `hand` encoded as in the example script `cardproblems.py`, return the list of these four numbers. For example, if a hand contains five spades, no hearts, five diamonds, and three clubs, this function should return `[5, 0, 5, 3]`.

Note that the cards in `hand` can be given to your function in any order, since in this question, the player has not yet manually sorted his hand. Your answer still must list all four suits in their canonical order, so that other players will also know what you are talking about.

hand	Expected result
`[('eight', 'spades'), ('king', 'diamonds'), ('ten', 'diamonds'), ('three', 'diamonds'), ('seven', 'spades'), ('five', 'diamonds'), ('two', 'hearts'), ('king', 'spades'), ('jack', 'spades'), ('ten', 'clubs'), ('ace', 'clubs'), ('six', 'diamonds'), ('three', 'hearts')]`	`[4, 2, 5, 2]`
`[('ace', 'spades'), ('six', 'hearts'), ('nine', 'spades'), ('nine', 'diamonds'), ('ace', 'diamonds'), ('three', 'diamonds'), ('five', 'spades'), ('four', 'hearts'), ('three', 'spades'), ('seven', 'diamonds'), ('jack', 'diamonds'), ('queen', 'spades'), ('king', 'diamonds')]`	`[5, 2, 6, 0]`

Yes, this was a milk run problem as a breather for everyone who's come this far, more suitable for the level of the first thirty problems or so. The later problems in this collection dealing with bridge hands will get a bit trickier (heh).

80. Milton Work point count

```
def milton_work_point_count(hand, trump='notrump'):
```

Playing cards are represented as tuples `(rank, suit)` as in our cardproblems.py example program. The Milton-Work point count estimates the trick-taking power of a bridge hand, and we will implement a version simple enough for beginners in either Python or the game of bridge. For a thirteen-card bridge hand, start with 4 points for each ace, 3 for each king, 2 for each queen, and 1 for each jack. This **raw point count** total is then adjusted with the following rules:

- If the hand contains one four-card suit and three three-card suits, subtract one point for being **flat**. (Flat hands rarely play as well as non-flat hands of equal point count.)
- Add 1 point for every suit with five cards, 2 points for every suit with six cards, and 3 points for every suit with seven or more cards. (Shape is power for the declarer.)
- If the `trump` suit is anything other than `'notrump'`, add 5 points for every **void** (that is, a suit without any cards in it) and 3 points for every **singleton** (that is, a suit with exactly one card), both of these for any suit other than the `trump` suit. (Voids and singletons are great when you are playing a suit contract, but very bad in a no-trump contract. Being void or a singleton in the trump suit is, of course, extremely bad!)

`hand` (each hand below has been sorted by suits for readability, but the tester can and will give these cards to your function in any order)	`trump`	Expected result
`[('four', 'spades'), ('five', 'spades'), ('ten', 'hearts'), ('six', 'hearts'), ('queen', 'hearts'), ('jack', 'hearts'), ('four', 'hearts'), ('two', 'hearts'), ('three', 'diamonds'), ('seven', 'diamonds'), ('four', 'diamonds'), ('two', 'diamonds'), ('four', 'clubs')]`	`'diamonds'`	`8`
`[('three', 'spades'), ('queen', 'hearts'), ('jack', 'hearts'), ('eight', 'hearts'), ('six', 'diamonds'), ('nine', 'diamonds'), ('jack', 'diamonds'), ('ace', 'diamonds'), ('nine', 'clubs'), ('king', 'clubs'), ('jack', 'clubs'), ('five', 'clubs'), ('ace', 'clubs')]`	`'clubs'`	`20`
`[('three', 'spades'), ('seven', 'spades'), ('two', 'spades'), ('three', 'hearts'), ('queen', 'hearts'), ('nine', 'hearts'), ('ten', 'diamonds'), ('six', 'diamonds'), ('queen', 'diamonds'), ('ace', 'diamonds'), ('nine', 'clubs'), ('four', 'clubs'), ('five', 'clubs')]`	`'notrump'`	`7`

81. Never the twain shall meet

```
def hitting_integer_powers(a, b, tolerance=100):
```

Integer powers grow rapidly and quickly become too large to be useful in daily life. Outside time and space, alone with no human minds to watch their journey, these sequences probe the space of positive integers with exponentially increasing gaps, paradoxically both infinite and instantaneous, and eventually contain not just entire universes but all their possible timelines encoded within them. Fortunately, Python allows us to play around with integer powers of millions of digits. Its mechanical decisions will reliably and unerringly dissect these slumbering Eldritch abominations that our mortal minds could not begin to visualize, even as our logic tickles their bellies in a way that hopefully feels sufficiently pleasant to amuse, but not to anger them.

Except when the prime factors of `a` and `b` already cooperate, the iron hand of the Fundamental Theorem of Arithmetic dictates that the integer powers `a**pa` and `b**pb` can never be equal for any two positive integer exponents `pa` and `pb`. However, in the jovial spirit of "close enough for government work", we define two such powers to "hit" if their difference `abs(a**pa-b**pb)` multiplied by the `tolerance` is at most equal to the smaller of those powers. (This definition intentionally avoids division to keep it both fast and accurate for arbitrarily large integers.) For example, `tolerance=100` expects `a**pa` and `b**pb` to be within 1%. For given positive integers `a` and `b`, return the smallest integer exponent pair `(pa, pb)` that satisfies the `tolerance` requirement.

a	b	tolerance	Expected result
9	10	5	(1, 1)
2	4	100	(2, 1)
2	7	100	(73, 26)
3	6	100	(137, 84)
4	5	1000	(916, 789)
10	11	1000	(1107, 1063)
42	99	100000	(33896, 27571)

This problem was inspired by The Riddler's column "Can you get another haircut already?", which is generally an excellent source of puzzles in this spirit.

82. Bridge hand shorthand form

`def bridge_hand_shorthand(hand):`

In **contract bridge** literature, hands are often abbreviated, making their key features easier to visualize at a glance. In this shorthand form, suits are always listed in the order spades, hearts, diamonds, **and clubs**, so no special symbols are needed to show which suit is which. Each suit lists ranks as letters from `'AKQJ'` for aces and face cards, and all spot cards lower than a Jack are written as the same letter 'x' to indicate their exact spot value is irrelevant to the hand's play mechanics. These letters must be listed in descending order of ranks `AKQJx`. If a suit is **void** (the hand contains no cards of that suit), abbreviate it with a minus sign character `'-'`. The shorthand forms for the individual suits are separated using single spaces, with no trailing whitespace.

`hand` (each hand below is sorted by suits for readability, but your function can receive these 13 cards from the tester in any order)	Expected result
`[('four', 'spades'), ('five', 'spades'), ('ten', 'hearts'), ('six', 'hearts'), ('queen', 'hearts'), ('jack', 'hearts'), ('four', 'hearts'), ('two', 'hearts'), ('three', 'diamonds'), ('seven', 'diamonds'), ('four', 'diamonds'), ('two', 'diamonds'), ('four', 'clubs')]`	`'xx QJxxxx xxxx x'`
`[('three', 'spades'), ('queen', 'hearts'), ('jack', 'hearts'), ('eight', 'hearts'), ('six', 'diamonds'), ('nine', 'diamonds'), ('jack', 'diamonds'), ('ace', 'diamonds'), ('nine', 'clubs'), ('king', 'clubs'), ('jack', 'clubs'), ('five', 'clubs'), ('ace', 'clubs')]`	`'x QJx AJxx AKJxx'`
`[('three', 'spades'), ('seven', 'spades'), ('two', 'spades'), ('three', 'hearts'), ('queen', 'hearts'), ('nine', 'hearts'), ('ten', 'diamonds'), ('six', 'diamonds'), ('queen', 'diamonds'), ('ace', 'diamonds'), ('nine', 'clubs'), ('four', 'clubs'), ('five', 'clubs')]`	`'xxx Qxx AQxx xxx'`
`[('ace', 'spades'), ('king', 'spades'), ('queen', 'spades'), ('jack', 'spades'), ('ten', 'spades'), ('nine', 'spades'), ('eight', 'spades'), ('seven', 'spades'), ('six', 'spades'), ('five', 'spades'), ('four', 'spades'), ('three', 'spades'), ('two', 'diamonds')]`	`'AKQJxxxxxxxx - x -'`

(Then again, the author did once see a play problem where a freaking *eight* was a relevant rank…)

83. Points, schmoints

`def losing_trick_count(hand):`

The Milton Work point count in an earlier problem is only the first baby step in estimating the bridge hand's playing power. Once the partnership finds a good trump fit, hand evaluation proceeds more accurately using some form of losing-trick count. For example, a small slam in spades with `'AKxxxxx - Kxxx xx'` facing `'xxxx xxxxx AQx -'` is a lock despite possessing only 16 of the 40 high card points in the deck, assuming that the opponents, looking at their own massive shapes, let you play that slam instead of making a great sacrifice by bidding seven of their suit. (Advanced partnerships can judge such things to astonishing precision merely from what has, and especially what has *not*, been bid!) On the other hand (heh), any slam is dead in the water with the `'QJxxx xx AKx QJx'` facing `'AKxxx QJ QJx AKx'` regardless of the lead, thanks to the horrendous duplication of the useful high cards. ("If it quacks like a duck…")

In this problem, we compute the basic losing trick count as given in step 1 of the "Methodology" section of the Wikipedia page "Losing Trick Count" without any finer refinements. Keep in mind that no suit can have more losers than it has cards, and never more than three losers, even if the hand has ten cards of that suit! The following dictionary (composed by student Shu Zhu Su during the Fall 2018 semester) might also come in handy for the combinations whose losing trick count differs from the string length, once you convert each `J` of the shorthand form into an `x` :

```
{'-':0,'A':0,'x':1,'Q':1,'K':1,'AK':0,'AQ':1,'Ax':1,'KQ':1,'Kx':1,'Qx':2,'xx':2,
'AKQ':0, 'AKx':1, 'AQx':1,'Axx':2, 'Kxx':2, 'KQx':1, 'Qxx':2, 'xxx':3}
```

hand	Expected result
`[('ten', 'clubs'), ('two', 'clubs'), ('five', 'clubs'), ('queen', 'hearts'), ('four', 'spades'), ('three', 'spades'), ('ten', 'diamonds'), ('king', 'spades'), ('five', 'diamonds'), ('nine', 'hearts'), ('ace', 'spades'), ('queen', 'spades'), ('six', 'spades')]`	7
`[('eight', 'hearts'), ('queen', 'spades'), ('jack', 'hearts'), ('queen', 'hearts'), ('six', 'spades'), ('ten', 'hearts'), ('five', 'clubs'), ('jack', 'spades'), ('five', 'diamonds'), ('queen', 'diamonds'), ('six', 'diamonds'), ('three', 'spades'), ('nine', 'clubs')]`	8

84. Bulls and cows

```
def bulls_and_cows(guesses):
```

In the two-player game of "Bulls and Cows", reincarnated after the seventies Rural Purge under the more dramatic, urban title "Mastermind" in colourful plastic, the first player thinks up a four-digit secret number with all different digits, such as 8723 or 9425. (For simplicity, the digit zero is not used in our version of this problem.) The second player tries to pinpoint this secret number by repeatedly guessing some four-digit numbers. After each guess, the first player reveals how many **bulls** (the right digit in the right position) and **cows** (a right digit but in the wrong position) that particular guess contains. For example, if the secret number is 1729, the guess 5791 contains one bull (the digit 7) and two cows (the digits 9 and 1). The guess 4385, on the other hand, contains no bulls or cows… thus neatly pruning all four digits from all future guesses!

This function should list all numbers that could still be the secret number according to the results of the list of `guesses` completed so far. Each guess is given as a tuple `(guess, bulls, cows)`. This function should return the list of such consistent four-digit integers in ascending order. Note that it is very much possible for this result list to be empty if the results of the `guesses` are inherently contradictory. (Good functions ought to be **robust** so that they do the right thing even when given syntactically correct but semantically meaningless argument values.)

Start by creating a list of all four-digit numbers that do not contain any repeated digits. Loop through the individual `guesses`, and for each guess, use a list comprehension to create a list of numbers that were in the previous list and are still consistent with the current guess. After you have done all that, oi jolly well then mate, "*When you have eliminated all which is impossible, then whatever remains, however improbable, must be the truth.*" —Sherlock Holmes.

`guesses`	Expected result
`[(1234, 2, 2)]`	`[1243, 1324, 1432, 2134, 3214, 4231]`
`[(8765, 1, 0), (1234, 2, 1)]`	`[1245, 1263, 1364, 1435, 1724, 1732, 2734, 3264, 4235, 8134, 8214, 8231]`
`[(1234, 2, 2), (4321, 1, 3)]`	`[]`
`[(3127, 0, 1), (5723, 1, 0), (7361, 0, 2), (1236, 1, 0)]`	`[4786, 4796, 8746, 8796, 9746, 9786]`

This problem and its myriad generalizations to exponentially more possibilities (for example, the current craze of Wordle or its predecessor Jotto, which played with English words in the same manner) can be solved more cleverly and efficiently than the brute-force enumeration above. However, we will leave that for a later course on advanced algorithms.

85. Frequency sort

```
def frequency_sort(items):
```

Sort the given list of integer `items` so its elements appear in order of decreasing frequency, that is, the number of times they appear in `items`. Ties where two elements occur with the same frequency should be broken up by sorting such elements in **ascending** order of their element values with respect to each other.

The best way to solve this problem is to "let George do it," or whatever you want to call passing the buck to a hapless underling, by assigning the role of George to the Python `sort` function. We are going to tell the `sort` function to perform sorting according to a custom sorting criterion, as in the lecture example script `countries.py`.

Start by creating a dictionary to keep track of how many times each element occurs inside the array. Then use the counts stored in that dictionary as the **sorting key** for the actual array elements, breaking ties by the element value. (If you also remember that the order comparison between Python tuples is **lexicographic**, you don't even have to burden yourself with the work needed to break these ties between two equally frequent values…)

`items`	Expected result
`[4, 6, 2, 2, 6, 4, 4, 4]`	`[4, 4, 4, 4, 2, 2, 6, 6]`
`[4, 6, 1, 2, 2, 1, 1, 6, 1, 1, 6, 4, 4, 1]`	`[1, 1, 1, 1, 1, 1, 4, 4, 4, 6, 6, 6, 2, 2]`
`[17, 99, 42]`	`[17, 42, 99]`
`['bob','bob','carl','alex','bob']`	`['bob','bob','bob','alex','carl']`

86. Calling all units, B-and-E in progress

```
def is_perfect_power(n):
```

A positive integer `n` is a perfect power if it can be expressed as the power `b**e` for some two integers `b` and `e` that are both **greater than one**. (Any positive integer `n` can always be expressed as the trivial power `n**1`, so we don't care about those.) For example, 32, 125, and 441 are perfect powers since they equal 2**5, 5**3, and `21**2`, respectively.

This function should determine whether the positive integer `n` is a perfect power. Your function needs to somehow iterate through a sufficient number of possible combinations of `b` and `e` that could work, returning `True` right away when you find some `b` and `e` that satisfy `b**e == n`, and returning `False` when all relevant possibilities for `b` and `e` have been tried and found wanting. Since `n` can get pretty large, your function should not examine too many combinations of `b` and `e` above and beyond those that are both necessary and sufficient to determine the answer reliably. This efficiency is the central educational point of this problem.

`n`	Expected result
`42`	`False`
`441`	`True`
`469097433`	`True`
`12**34`	`True`
`12**34 - 1`	`False`

A tester for this problem can be built on Catalan's conjecture, now a proven theorem, which says that, except for the special case of the two consecutive perfect powers 8 and 9, if n is a perfect power, $n - 1$ is never a perfect power. This theorem makes it easy to generate pseudorandom test cases with known positive and negative answers. For example, we don't have to slog through all possible ways to express the number as an integer power to know from the get-go that $12345^{67890} - 1$ isn't a perfect power. This also illustrates the common asymmetry between computing in opposite directions. Given a big chungus integer such as 492223524295202670403711324312200806 4, but not the formula that produced it, it is not easy to tell whether that integer is a perfect power, or a perfect power plus or minus one.

87. Lunatic multiplication

```
def lunar_multiply(a, b):
```

This problem was inspired by another jovial Numberphile video, "Primes on the Moon" by Neil Sloane (as in), that you should watch first to get an idea of how this wacky "lunar arithmetic" works. Formerly known as "dismal arithmetic", addition and multiplication of natural numbers are redefined so that **adding** two digits means taking their **maximum**, whereas **multiplying** two digits means taking their **minimum**. For example, 2 + 7 = 7 + 2 = 7 and 2 × 7 = 7 × 2 = 2. Unlike ordinary addition, there can never be a carry to the next column of digits, no matter how many individual digits are added together in that column. For numbers with several digits, addition and multiplication work exactly as you learned in grade school, except that the shifted digit columns from lunar multiplication of the individual digits are added in the same lunatic fashion.

These wacky operators define an algebraic ring where useful and important algebraic identities such as $a \times b = b \times a$, $(a + b) + c = a + (b + c)$, and $a \times (b + c) = a \times b + a \times c$ hold, along with everything that is then entailed by those identities. The **unit** element for addition is zero, the same as in ordinary arithmetic. However, if only to rouse the boomers in the peanut gallery, the unit for multiplication is not one but their beloved number nine, since $9n = n$ for any natural number n, as you can easily verify. This function should compute and return the lunar product of two nonnegative integers `a` and `b`.

a	b	Expected result
2	3	2
8	9	8
11	11	111
17	24	124
123	321	12321
357	64	3564
123456789	987654321	12345678987654321

88. Distribution of abstract bridge hand shapes

```
def hand_shape_distribution(hands):
```

This continues the earlier problem of computing the shape of a given bridge hand. In that problem, we considered the shapes `[6, 3, 2, 2]` and `[2, 3, 6, 2]` different, as they would be in actual bidding and play. However, in this combinatorial generalization, we consider two hand shapes the same abstract shape if they match when we look only at the sorted suit counts, regardless of which suits they are.

Given a list of bridge `hands`, each hand represented as a list of 13 cards encoded the same way as in all previous card problems, create and return a Python dictionary that contains all abstract shapes that occur in the `hands`, each shape mapped to its count of occurrences. Note that Python dictionary keys cannot be lists, so you need to represent the abstract hand shapes as immutable **tuples** that can be used as keys inside your dictionary. (Since Python lists are mutable, changing the contents of a dictionary key would break the internal ordering of the dictionary.)

As tabulated in "Suit distributions" in "Durango Bill's Bridge Probabilities and Combinatorics", there are precisely 39 possible abstract shapes of thirteen cards of four suits, the most common of which is 4–4–3–2, followed by 5–3–3–2. Contrary to intuition, the most balanced possible hand shape, 4–3–3–3, is surprisingly unlikely, trailing even far less balanced shapes like 5–4–3–1 and 5–4–2–2, which one might expect to be less frequent. (Understanding why randomness tends to produce variance rather than converge to complete uniformity helps explain many other counterintuitive truths about the behaviour of random processes in computer science and mathematics.)

If it were somehow possible to give your function the list of all 635,013,559,600 possible bridge hands without running out of the heap memory in the Python virtual machine, the returned dictionary would contain 39 entries for the 39 possible abstract hand shapes, two examples of these entries being `(4,3,3,3):66905856160` and `(6,5,1,1):4478821776`. Our automated tester will try out your function with a much smaller list of pseudo-randomly generated bridge hands. Still, for the common hand types you might expect to see every day at the local bridge club's daily duplicate, the percentages really shouldn't differ much from the exact answers if measured over a sufficiently large number of random hands.

89. Fibonacci sum

```
def fibonacci_sum(n):
```

Fibonacci numbers are a cliché in introductory computer science, especially in teaching recursion, where this famous combinatorial series is mainly used to reinforce the belief that recursion is silly. However, all clichés became clichés because they were so darn good that every Tom, Dick, and Harry kept using them! Let us therefore traipse around Zeckendorf's theorem, the more amazing property of these famous numbers: **every positive integer can be expressed exactly one way as a sum of distinct non-consecutive Fibonacci numbers**.

The simple **greedy algorithm** can be proven to always produce the desired breakdown of `n` into a sum of distinct non-consecutive Fibonacci numbers: always append to the list the largest possible Fibonacci number `f` that is less than or equal to `n`. Then convert the rest of the number `n-f` in this same manner, until nothing remains to be converted. To make the results unique for automated testing, this list of Fibonacci numbers that sum to n must be returned in descending order.

`n`	Expected result
`10`	`[8, 2]`
`100`	`[89, 8, 3]`
`1234567`	`[832040, 317811, 75025, 6765, 2584, 233, 89, 13, 5, 2]`
`10**10`	`[7778742049, 1836311903, 267914296, 102334155, 9227465, 3524578, 1346269, 514229, 75025, 6765, 2584, 610, 55, 13, 3, 1]`
`10**100`	(a list of 137 terms, the largest of which starts with digits `921684571`)

This greedy construction guarantees the chosen Fibonacci numbers are distinct and cannot include two consecutive Fibonacci numbers F_i and F_{i+1}. Otherwise, their sum F_{i+2} would also have fit inside n and been used instead of F_{i+1}, and thus would have prevented the use of F_{i+1} with the same argument involving the next higher Fibonacci number F_{i+3} …

90. Wythoff array

```
def wythoff_array(n):
```

Wythoff array (see the Wikipedia article for illustration) is an infinite two-dimensional grid of integers that is seeded with 1 and 2 to start the first row. In each row, each element equals the sum of the previous two elements, so the first row contains precisely the Fibonacci numbers.

The first element of each row after the first is **the smallest integer `c` that does not appear anywhere in the previous rows**. Since each row is strictly ascending and grows exponentially fast, you can find this by looking at relatively short finite prefixes of the rows. To determine the second element of that row, let `a` and `b` be the first two elements of the previous row. If the difference `c-a` equals two, the second element of that row equals `b+3`, and otherwise that element equals `b+5`. This construction guarantees that the Wythoff array intersperses positive integers; every positive integer appears exactly once in the infinite grid, with no gaps or duplicates. (This result also nicely highlights the deeper combinatorial importance of the deceptively simple Fibonacci numbers as potential building blocks of integers and their sequences.)

The difficulty in this problem is determining the first two elements of each row, since the rest of the row can then be generated trivially. This function should return the position of `n` in the Wythoff array as a two-tuple `(row, col)`, with rows and columns both starting from zero.

`n`	Expected result
`21`	`(0, 6)`
`47`	`(1, 5)`
`1042`	`(8, 8)`
`424242`	`(9030, 6)`
`39088169`	`(0, 36)`
`39088170`	`(14930352, 0)`

91. Rooks with friends

```
def rooks_with_friends(n, friends, enemies):
```

Those dastardly rooks have gone on a rampage again on a generalized n-by-n chessboard, like in the earlier problem of counting how many squares kept their occupants from being pulverized into dust under these lumbering juggernauts. Each rook is again represented as a two-tuple `(row, col)` of the zero-based coordinates of the square it stands on. However, this version of the problem introduces a pinch of old-fashioned ethnic nepotism to the mix so that some of these rooks are your **friends** (same colour as you) while the others are your **enemies** (the opposite colour from you). Trapped in this surrealistic hellscape straight out of Hieronymus Bosch's medieval visions, you can only scurry along and hope to survive until the dust settles.

Friendly rooks protect the chess squares by standing between them and any enemy rooks that want to invade. An enemy rook can attack only those squares in the same row or column that do not enjoy the protection of such a friendly rook standing between them. Given the board size `n` and the lists of `friends` and `enemies` (these two lists are guaranteed to be disjoint so that no rook can act as a fence-sitter or a turncoat), count how many empty squares on the board are safe from the enemy rooks.

n	friends	enemies	Expected result
4	`[(2,2), (0,1), (3,1)]`	`[(3,0), (1,2), (2,3)]`	2
4	`[(3,0), (1,2), (2,3)]`	`[(2,2), (0,1), (3,1)]`	2
8	`[(3,3), (4,4)]`	`[(3,4), (4,3)]`	48
100	`[(r, (3*r+5) % 100) for r in range(1, 100, 2)]`	`[(r, (4*r+32) % 100) for r in range(0, 100, 2)]`	3811

92. Possible words in Hangman

```
def possible_words(words, pattern):
```

Given a list of possible `words`, and a `pattern` string that is guaranteed to contain only lowercase English letters `a` to `z` and asterisk characters `*`, create and return a sorted list of words that match the `pattern` in the sense of the famous pen-and-paper word guessing game of Hangman.

Each `pattern` can only match words of the same length. In positions where the `pattern` contains some letter, the word must contain that same letter. In positions where the `pattern` contains an asterisk, the word character in that position can be any letter except one of the letters that occur inside the `pattern`. In a real game of Hangman, all occurrences of that letter would have been revealed in an earlier round when that letter was the current guess.

For example, the words `'bridge'` and `'smudge'` both match the pattern `'***dg*'`. However, the words `'grudge'` and `'dredge'` would not match that same pattern, since the first asterisk may not be matched with either letter `'g'` or `'d'` that already appears inside the pattern.

`pattern`	Expected result (using wordlist `words_sorted.txt`)
`'***dg*'`	`['abedge', 'aridge', 'bludge', 'bridge', 'cledge', 'cledgy', 'cradge', 'fledge', 'fledgy', 'flidge', 'flodge', 'fridge', 'kludge', 'pledge', 'plodge', 'scodgy', 'skedge', 'sledge', 'slodge', 'sludge', 'sludgy', 'smidge', 'smudge', 'smudgy', 'snudge', 'soudge', 'soudgy', 'squdge', 'squdgy', 'stodge', 'stodgy', 'swedge', 'swidge', 'trudge', 'unedge']`
`'*t*t*t*'`	`['statute']`
`'a**s**a'`	`['acystia', 'acushla', 'anosmia']`
`'*ikk**'`	`['dikkop', 'likker', 'nikkud', 'tikker', 'tikkun']`

93. All branches lead to Rome

```
def lattice_paths(x, y, tabu):
```

You are standing at the point `(x,y)` in the **lattice grid** of pairs of natural numbers, and wish to make your way to the **origin** point `(0,0)`. At any point, you are allowed to move either one step left or one step down, and you are not allowed to step into any of the points in the `tabu` list. This function should count the number of distinct paths from the point `(x, y)` to the origin `(0, 0)` that satisfy these constraints. The goal `(0,0)` is never part of the `tabu` list.

This constrained variation of the classic combinatorial problem has a reasonably straightforward recursive solution. As a base case, the number of paths from the origin (0,0) to itself (0,0) equals one (the empty path). (Note the crucial difference between an empty path that *exists*, versus a *nonexistent* path!) If the point `(x,y)` is in the `tabu` list, the number of paths from that point `(x,y)` to the origin equals zero. The same holds for all points whose either coordinate `x` or `y` is negative. Otherwise, the number of paths from `(x,y)` to the origin (0,0) equals the sum of paths from the two neighbours `(x-1,y)` and `(x,y-1)`.

However, this simple recursion branches into an exponential number of possibilities, which is far too slow to execute. Therefore, you should either **memoize** the recursion with `lru_cache`, or even better, not use recursion at all but build up a two-dimensional list whose entries are the individual subproblem solutions. Fill in the correct values with two `for`-loops in some order that guarantees that when these loops arrive at position `[x][y]`, the results for positions `[x-1][y]` and `[x][y-1]` needed to compute `[x][y]` are already there.

x	y	tabu	Expected result
3	3	[]	20
3	4	[(2, 2)]	17
10	5	[(6, 1), (2, 3)]	2063
6	8	[(4, 3), (7, 3), (7, 7), (1, 5)]	1932
10	10	[(0, 1)]	92378
100	100	[(0, 1), (1, 0)]	0

This technique, using for-loops to fill memoization tables rather than a recursive formula, is called **dynamic programming**. This elegant technique is a skeleton key that opens thousands of doors Hercules himself could not pry open with brute force.

94. Be there or be square

```
def count_squares(points):
```

This problem is adapted from "Count the Number of Squares" at Wolfram Challenges, so you might want to first check out that page for illustrative visualizations of this problem.

Given a `set` of `points`, each point a tuple `(x,y)` where `x` and `y` are nonnegative integers, this function should count how many **squares** exist so that all four corners are members of `points`. Note that these squares are not required to be **axis-aligned**, so their sides would have to be either horizontal or vertical. For example, the points `(0,3)`, `(3,0)`, `(6,3)`, and `(3,6)` define a square, even if it may look like a lozenge from our axis-aligned vantage point. As long as all four sides have the same length, whether that length is an integer or the square root of a non-square integer, that makes us "square," at least in my book, pardner!

To identify four points that constitute a square, note how every square has **a bottom-left corner** point `(x,y)` and **direction vector** `(dx,dy)` towards its **upper-left corner** point that satisfies `dx>=0` and `dy>0`, so that the points `(x+dx,y+dy)`, `(x+dy,y-dx)` and `(x+dx+dy,y-dx+dy)` for the top left, bottom right and top right corners of that square, respectively, are also included in the `points`. You can therefore iterate through all possibilities for the bottom left point `(x,y)` and the direction vector `(dx,dy)` and be guaranteed to find all squares in the grid. But please do at least make an attempt to be economical in these loops, and make your function sweep up these squares more swiftly than Tom Swift himself ever could corner them.

`points`	Expected result
`[(0,0), (1,0), (2,0), (0,1), (1,1), (2,1), (0,2), (1,2), (2,2)]`	`6`
`[(4,3), (1,1), (5,3), (2,3), (3,2), (3,1), (4,2), (2,1), (3,3), (1,2), (5,2)]`	`3`
`[(x, y) for x in range(1, 10) for y in range(1, 10)]`	`540`
`[(3,4), (1,2), (3,2), (4,5), (4,2), (5,3), (4,1), (5,4), (3, 5), (2,4), (2,2), (1,1), (4,4), (2,5), (1,5), (2,1), (2,3), (4, 3)]`	`15`

Interested students can check out the Mathologer video "What does this prove?" about this general phenomenon, if only to admire a beautiful **infinite regress** proof of why squares are the only regular polygon whose corner points can lie exactly on points of an integer grid.

95. Flip of time

```
def hourglass_flips(glasses, t):
```

An hourglass is given as a tuple `(upper, lower)` representing the number of minutes of sand currently stored in its `upper` and `lower` chambers, both chambers large enough to hold all the sand in that hourglass. After `m` minutes have elapsed, the state of that particular hourglass will be `(upper-min(upper,m),lower+min(upper,m))`. The total amount of sand inside the hourglass never changes, and neither chamber will ever contain negative anti-sand.

Given a list of `glasses`, your task is to find an optimal sequence of moves to measure exactly `t` minutes, scored as the number of individual hourglass flips performed along the way. Each move consists of two stages. You must first wait for the hourglass that currently holds the lowest non-zero amount `m` of sand in its upper chamber to run out. When that happens, choose any subset of `glasses` and instantly flip that subset. You have no choice in the first stage, but in the second stage, you have an embarrassment of riches: $2^n - 1$ possible moves for n glasses! Continue such two-stage moves to measure the remaining `t-m` minutes best.

This function should return the minimum number of individual hourglass flips needed to measure exactly `t` minutes, or `None` if such a measurement is impossible. The base cases of recursion are when `t` equals zero or when exactly `t` minutes remain in some upper chamber (no flips are needed), or when `t` is smaller than the smallest time remaining in the upper chamber of any hourglass (no solution is possible). Otherwise, update all hourglasses to their new states after `m` minutes, and loop through all possible subsets of `glasses` to flip. For each such subset, recursively construct the optimal sequence of moves to measure the remaining `t-m` minutes, and combine these moves to the best solution.

glasses	t	Expected result
`[(7, 0), (11, 0)]`	15	2 (see here)
`[(4, 0), (6, 0)]`	11	None
`[(7, 1), (10, 4), (13, 1), (18, 4)]`	28	3
`[(13, 1), (14, 3)]`	22	16
`[(10, 1), (13, 4)]`	15	None

The generator `itertools.product([0,1], repeat=n)` yields all possible `n`-element tuples of `[0,1]` representing the 2^n possible subsets of the `n` individual `glasses` to be flipped.

96. Bulgarian cycle

```
def bulgarian_cycle(piles):
```

When the total number of pebbles in the piles is the *k*-th triangular number, the earlier Bulgarian solitaire iteration is guaranteed to reach a steady state whose pile sizes are the integers 1 to *k*, each exactly once. If you start with a different number of pebbles, this iteration will never reach such a steady state. Instead, the system will eventually re-enter a state it has already visited and then repeat the same **limit cycle** of states. For example, starting from `[2, 3, 2]`, the system goes through states `[1, 2, 1, 3]`, `[1, 2, 4]`, `[1, 3, 3]`, and `[2, 2, 3]`. Since this last state `[2, 2, 3]` is considered equal to the starting state `[2, 3, 2]`, this completes the limit cycle of length four. In general, the limit cycle need not return to the starting state, but it can loop back to a later state in this sequence.

This function should compute the length of the limit cycle starting from the given `piles`. The ingenious **tortoise-and-hare algorithm** invented by Robert Floyd detects when an iteration returns to a previously seen state without explicitly storing the entire sequence of previously seen states in memory. This algorithm only needs to track two current states, called the tortoise and the **hare**. Both start in the same pile configuration and follow the same sequence of states. However, the hare takes two steps for every step the tortoise takes.

Like two long-distance runners racing toward a stadium some unknown distance ahead, even the tortoise will eventually reach the stadium. As the hare runs around the stadium track, it will inevitably overtake the tortoise, and the point where they meet must be part of the limit cycle. Then have the hare count its steps around the stadium one more time while the tortoise naps to mark the spot, so the hare knows when to stop counting.

`piles`	Expected result
`[1, 3, 2]`	`1`
`[2, 3, 2]`	`4`
`[4, 4, 3, 3, 2, 1]`	`6`
`[17, 99, 42]`	`18`
`[2, 0, 0, 2, 6, 12, 20, 30, 42, 56]`	`19`
`[n*n for n in range(1, 51)]`	`293`

97. Spirit of the staircase

```
def staircase(digits):
```

The earlier problem “Beat the previous” asked you to greedily extract a strictly ascending sequence of integers from the given series of `digits`. For example, for `digits` equal to `'31415926'`, the list of integers returned should be `[3, 14, 15, 92]`, with the last original digit left unused.

Counterintuitively, tactically skipping digits may let the resulting list contain more elements than the list constructed by the previous greedy approach. With this extra freedom, the digit string `'31415926'` yields `[3, 4, 5, 9, 26]`, which has one more element than the greedy solution.

Your function should return the length of the longest list of ascending integers that can be extracted from the `digits`. Note that you are allowed to skip one or more `digits`, not just between two integers being extracted, but during the construction of each such integer.

digits	Expected result
`'100123'`	4
`'503715'`	4
`'0425494220946'`	6
`'04414952075836'`	7
`'1234567891011213'`	12
`'1011101110121013'`	6
`'007304517931223832'`	9
`'78219236133269344741'`	9

98. Square it out amongst yourselves

```
def cut_into_squares(w, h):
```

A rectangle is defined by its width w and height h, both of which are positive integers. We allow a rectangle to be cut into two smaller rectangles by a straight horizontal or vertical cut at any integer position. For example, one way to cut the rectangle `(5,8)` into two pieces is to cut it into `(2,8)` and `(3,8)`. Another way is to cut it into `(5,4)` and `(5,4)`. And many others, too many to list here. You can then cut the resulting smaller pieces further, as long as the side being cut is at least 2 units long. (Also, at the risk of disappointing all you Edward de Bono fanboys reading this, you may not cut multiple pieces in one motion by stacking pieces on top of each other.)

Your task is to keep cutting the given `w`-by-`h` rectangle into smaller pieces until each piece is a **square**, that is, the width of each piece equals its own height. This is always possible, since you could just keep cutting until each piece is 1-by-1. However, this function should return the minimum number of cuts needed to make each piece a square.

This problem is best solved with recursion. The base cases are when `w==h` (the piece is already a square), so you return 0, and when `min(w,h)==1`, so you can't cut the shorter side further; return `max(w,h)-1`. Otherwise, loop through the possible ways to cut this piece into two smaller pieces, recursively computing the best possible way to cut up these two pieces. Return the number of cuts produced by the optimal way of cutting this piece. Since this branching recursion would visit its subproblems exponentially many times, you will surely want to sprinkle some `@lru_cache` memoization magic on it to downgrade that exponential tangle into a mere quadratic one.

w	h	Expected result
4	4	0
4	2	1
7	6	4
91	31	11
149	139	19

99. Sum of distinct cubes

```
def sum_of_distinct_cubes(n):
```

Positive integers can be expressed as sums of **distinct** cubes of **positive** integers, sometimes in multiple ways. This function should find and return the list of distinct cubes whose sum equals the given positive integer `n`. If `n` allows several breakdowns into sums of distinct cubes, this function must return the **lexicographically highest** solution that starts with the largest possible first number `a`, followed by the lexicographically highest representation of the rest of the number `n-a*a*a`. For example, called with `n=1729`, this function should return `[12, 1]` instead of `[10, 9]`. If it is impossible to express `n` as a sum of distinct cubes, return `None`.

Unlike in the earlier, much simpler problem of expressing `n` as a sum of exactly two squares, the result can contain any number of distinct terms. This problem, like almost all such problems that cause an exponential blowup of two-way "take it or leave it" decisions, is usually best solved with recursion that explores both branches of each decision and returns the solution that yields a better outcome. The base case is when `n` equals zero. Otherwise, loop through the possible values that could serve as the first element in the result, denoted by `a`. For each such possible `a`, break down the rest, n- a*a*a, recursively into a sum of distinct cubes using only integers smaller than `a`. Again, to let your function efficiently gleam these cubes even for large `n`, it might be a good idea to prepare something that allows you to quickly find the largest integer whose cube is not larger than `n`.

`n`	Expected result
`8`	`[2]`
`11`	`None`
`855`	`[9, 5, 1]`
`sum(n*n*n for n in range(11))`	`[14, 6, 4, 1]`
`sum(n*n*n for n in range(1001))`	`[6303, 457, 75, 14, 9, 7, 5, 4]`

This problem is intentionally restricted to positive integers to keep the number of possibilities finite. Since the cube of a negative number is also negative, no finite upper bound for search would exist if negative numbers were allowed. Breaking a small, simple number into a sum of exactly three cubes can suddenly become a highly nontrivial problem…

100. Count maximal layers

```
def count_maximal_layers(points):
```

A point `(x1,y1)` on the two-dimensional plane **dominates** another point `(x2,y2)` if both inequalities `x1>x2` and `y1>y2` hold simultaneously. A point inside the given list of `points` is defined to be **maximal** if any other point in the list does not dominate it. Unlike in the boring old one dimension, where points have a total order, some lists of `points` may contain maximal points that do not dominate each other, since it is enough that no other point dominates them.

A list of points on the two-dimensional plane can contain any number of maximal points, which then form that list's **maximal layer**. For example, the points `(3,10)` and `(9,2)` form the first maximal layer for `[(1,5), (8,1), (3,10), (2,1), (9,2)]`. Removing these two points leaves `[(1, 5), (8,1), (2,1)]`, all of which are maximal and form the second maximal layer. Removing these points leaves the empty list.

Given a list of `points` from the upper right quadrant of the infinite two-dimensional plane, with all point coordinates guaranteed to be nonnegative, this function should count how many times you have to remove the points in the current maximal layer from the list for that list to become empty.

points	Expected result
`[(1, 3), (2, 2), (3, 1)]`	1
`[(1, 5), (3, 10), (2, 1), (9, 2)]`	2
`[(x, y) for x in range(10) for y in range(10)]`	10
`[(x, x**2) for x in range(100)]`	100
`[(x, x**2 % 91) for x in range(1000)]`	28
`[((x**3) % 891, (x**2) % 913) for x in range(10000)]`	124

This is again one of those problems whose point (heh) and motivation is to make this function fast enough to finish reasonably quickly, even for a large list of `points`. Try not to perform much more work than is necessary to identify and remove the current maximal points. Start by noting how each point can potentially be dominated only by those points whose distance from the origin `(0,0)` is strictly larger. It doesn't even really matter which particular distance metric you use…

101. Maximum checkers capture

```
def max_checkers_capture(n, x, y, pieces):
```

Again, we're on a generalized *n*-by-*n* chessboard, this time playing a variation of checkers where your lone **king** stands at `(x,y)`, and the parameter pieces is a set containing the positions of the opponent's pawns. This function should compute the maximum number of pieces that your king could potentially capture in a single move.

Some variants of checkers allow **leaping kings** that can capture pieces across arbitrary distances. For simplicity, in our problem the royalty is more subdued so that kings capture a piece only one step in the four diagonal directions (the list `[(-1, 1), (1, 1), (1, -1), (-1, -1)]` might come in handy), assuming that the square behind the opponent piece in that diagonal direction is vacant. Your king can then capture that piece by jumping over it into the vacant square, **immediately removing it from the board**. However, unlike in chess, where each move can capture at most one enemy piece, the chain of captures continues from the new square, potentially capturing all the `pieces` in one swoop.

The maximum number of pieces you can capture in a single move is best computed by looping locally through the four diagonal directions from the king's current position. For each direction that contains an opponent piece with a vacant space behind it, remove that opponent piece from the board and recursively compute the number of pieces that can be captured from the vacant square your king jumps into. Once that recursive call has returned, restore the captured piece on the board, and continue looping to the next diagonal direction. Return the number of captures in the direction that yields the best result from that square.

n	x	y	pieces	Expected result
5	0	2	`set([(1,1), (3,1), (1,3)]`	2
7	0	0	`set([(1,1), (1,3), (3,3), (2,4), (1,5)])`	3
8	7	0	`set([(x, y) for x in range(2, 8, 2) for y in range(1, 7, 2)])`	9

102. Collatzy distance

```
def collatzy_distance(start, goal):
```

From a positive integer `n`, you are allowed to move into either `3*n+1` or `n//2` as a single step. Although these formulas were obviously inspired by the wonderfully chaotic Collatz conjecture from the lecture on unlimited iteration, this problem lets you apply either formula to any integer n, regardless of parity. This function should determine the number of steps needed to get from `start` to `goal` by taking the shortest possible route. You can solve all such problems with breadth-first search, which we preview here for a later course on **graph traversal** algorithms. (So take this problem here in the same spirit as you would watch the post-credits teaser of some Marvel capeshit romp.)

The algorithm visits all integers reachable from the `start`, advancing in **layers**, so that each layer consists of integers at the same distance from the `start`. To begin, the zeroth layer is the singleton list `[start]`, all numbers at a distance of zero from `start`. (Not too many of those!) Then, each layer `k+1` consists of integers `3*n+1` and `n//2`, where n steps through the numbers in layer k, except that we ignore any numbers already seen in previous layers. For example, if `start=4`, the first four layers are `[4]`, `[13, 2]`, `[40, 6, 7, 1]` and `[121, 20, 19, 3, 22, 0]` for the reachable numbers at distances from zero to three from the `start`, respectively. As you can see here, typical search problems exhibit these layers growing exponentially with `k`. (It's a small word, after all.)

Keep generating layers until the `goal` first appears; then return the number of the current layer as your answer. In practical breadth-first search implementations, a single first-in, first-out queue stores the nodes the algorithm has discovered but not yet visited, rather than processing each layer in a separate inner loop.

`start`	`goal`	Expected result
`10`	`20`	`7`
`42`	`42`	`0`
`42`	`43`	`13`
`76`	`93`	`23`
`1000`	`10`	`9`

103. Van Eck sequence

```
def van_eck(n):
```

The Van Eck sequence problem was making rounds while your author was updating these lab specifications. After reading only half of the original problem description, it was already clear this would be one of the most entertaining integer-sequence problems ever posed. The first term, at position $i = 0$, equals zero. The term in any later position $i > 0$ is determined by the term x in the previous position $i - 1$:

- If the position $i - 1$ was the first appearance of that term x in the sequence so far, the term in the current position i equals zero.
- Otherwise, the term at position i equals $i - 1 - j$, where j is the position of the second most recent appearance of x before position i.

This sequence begins with 0, 0, 1, 0, 2, 0, 2, 2, 1, 6, 0, 5, 0, 2, 6, 5, 4, 0, 5, 3, 0, 3, 2, 9, … so you see how its terms start making repeated appearances right away, especially the zero that automatically follows the first appearance of every new term. Your function should return the term in position `n`.

To let the automated tester finish within the twenty-second time limit, this function should not repeatedly search the already generated sequence for the most recent occurrence of each term. You should instead use a Python dictionary to remember the terms seen in the sequence, mapping them to the positions where they last appeared. With the aid of this dictionary, you don't need to store the entire sequence explicitly, but only the unique terms and their most recent positions that are relevant for your future decisions.

`n`	Expected result
`0`	`0`
`10`	`0`
`1000`	`61`
`10**6`	`8199`
`10**8`	`5522779`

In general, neither you nor your programs should try to remember the entire past, since there is just too much of it to store, and most of it is irrelevant. (All that past wouldn't even fit inside the space available inside the infinitely slim present moment anyway.)

104. Tips and trips

```
def trips_fill(words3, pattern, tabu):
```

A string of lowercase letters is **trip-filled** if every 3-letter substring (here, a "trip") inside it is a three-letter word from `words3`, the list of three-letter words in sorted order. Examples include `'pacepad'`, `'baganami'` and `'udianaahunat'`, using some of the more obscure three-letter words from `words_sorted.txt`, such as `'cep'` and `'gan'`. Given a `pattern` made of lowercase letters and asterisks, this function should find and return **the lexicographically first trip-filled string** that can be constructed by replacing each asterisk of the `pattern` with any letter of your choice. Also, no individual trip may appear more than once in the solution**.** If no solution exists for the given `pattern`, return `None`.

This problem is best solved with recursion, similar to `decode_morse` in the `morse.py` example. This function should loop through all trips that fit into the first three characters of the `pattern`. (Let `bisect_left` be your friend in this time of need.) For each fitting trip, recursively fill in the rest of the `pattern` from the next position onwards, with your currently chosen trip stamped into the first three characters of that `pattern`. Because this function needs to return only the first complete solution rather than generate all of them, ordinary recursion is sufficient instead of writing an entire **recursive generator**. To enforce the uniqueness of your chosen trips, this recursion should `append` each trip to the `tabu` list on the way down, and then `pop` that trip out of the `tabu` list after returning. (In all top-level calls from the tester, `tabu` will be an empty list.)

`pattern`	Expected result (using the wordlist `words_sorted.txt`)
`'**q'`	`'aeq'`
`'*yo***'`	`'iyobaa'`
`'*m*gv**o'`	`None`
`'*ef***da**s'`	`'befloadabas'`
`'**l**a*l*ai*'`	`'ablobaalcaid'`
`'*e'*8`	`'lekereseyewemeve'`
`'*'*100`	`'aaahabaalabbloadabcdfthadcabdeaddtdryadebbsfmtgnubcfibabeamablschaeraboafbibobaccmlxxiseanaambdspace'`

105. Balanced ternary

```
def balanced_ternary(n):
```

The integer two and its powers abound all over computing. In contrast, the number three seems mostly absent, at least until the theory of computation and its NP-complete problems, where three suddenly becomes very different from two, not only quantitatively, but qualitatively. (Sometimes quantity has a surprising quality of its own, as someone famously noted.) Stepping from two to three often opens a fresh spring of computational complexity.

Integers are normally written out in base ten, each digit giving the coefficient of the power of ten in that position. Computers internally encode integers in the simpler (at least for machines) **binary** of base two. Both schemes need special tricks to represent negative numbers without an explicit negation sign. Balanced ternary representation uses base three instead of two, with signed coefficients from three possibilities: –1, 0, and +1. Every integer, positive, negative or zero, can be broken down into a sum of signed powers of three exactly one way. Furthermore, the balanced ternary representation is symmetric around zero; you can trivially construct the representation for –*n* by flipping the sign of each term in the representation of +*n*.

Given an integer *n*, return the list of signed powers of three that add up to *n*, listing these powers in the descending order of their absolute values. This problem can be solved in several ways. The page "Balanced ternary" shows one way to do this by first converting *n* to **unbalanced ternary** (this is just base three with coefficients 0, 1 and 2, analogous to binary) and taking it from there. Another way is first to find *p*, the largest power of three less than or equal to |*n*|. Then, depending on the value of |*n*| – *p*, you either take *p* into the result and convert |*n*| – *p* for the rest, or take 3*p* into the result and convert |*n*| – *3p* for the rest, flipping the signs as needed. All roads lead to Rome.

n	Expected result
5	[9, -3, -1]
-5	[-9, 3, 1]
42	[81, -27, -9, -3]
-42	[-81, 27, 9, 3]
100	[81, 27, -9, 1]
10**8	[129140163, -43046721, 14348907, -531441, 177147, -59049, -19683, -6561, -2187, -729, 243, -81, -9, 1]

106. Lords of Midnight

```
def midnight(dice):
```

Midnight is a dice game where the player rolls six dice and decides which to keep and which to re-roll to maximize their final score. The final roll must include at least one 1 and one 4 for the player to score anything. The score is the sum of the other four dice, with a maximum of 24 and a minimum of 4. However, all your hard work with the previous problems has now mysteriously rewarded you with the gift of perfect foresight (as some wily Frenchman might say, you might unknowingly be a descendant of Madame de Thèbes) that allows you to foresee what pip values each die will produce in its entire sequence of future rolls, expressed as a sequence such as `[2, 5, 5, 1, 6, 3]`. With this foresight, return the maximum total score you can achieve.

The argument `dice` is a list where each element is a sequence of the pip values a particular die can produce when rolled. Since the rules require you to keep at least one die in each roll, this game must necessarily end after at most six rolls. Note that, as demonstrated by the first row of the following table, because of the rule that you must keep at least one die in every roll, the trivial algorithm "First choose the two dice you use to get the 1 and 4, and add up the maximum pip values for the four remaining dice" does not work.

`dice`	Expected result
`[[3, 4, 6, 6, 6, 2], [3, 2, 6, 2, 3, 3], [2, 2, 2, 2, 2, 3], [6, 1, 4, 2, 2, 2], [2, 2, 2, 3, 2, 3], [2, 3, 3, 3, 3, 2]]`	`14`
`[[2, 6, 2, 5, 2, 5], [5, 3, 3, 2, 5, 3], [2, 2, 2, 2, 5, 2], [3, 6, 3, 2, 2, 5], [6, 2, 2, 6, 3, 2], [2, 2, 3, 2, 2, 2]]`	`0`
`[[2, 6, 2, 1, 3, 3], [2, 2, 2, 2, 2, 3], [2, 2, 4, 3, 6, 6], [4, 5, 6, 3, 2, 5], [2, 4, 2, 6, 5, 3], [2, 2, 2, 2, 2, 3]]`	`17`
`[[3, 4, 6, 6, 6, 2], [3, 2, 6, 2, 3, 3], [2, 2, 2, 2, 2, 3], [6, 1, 4, 2, 2, 2], [2, 2, 2, 3, 2, 3], [2, 3, 3, 3, 3, 2]]`	`14`
`[[2, 3, 5, 3, 2, 2], [1, 3, 2, 3, 6, 4], [3, 2, 3, 3, 3, 5], [3, 6, 4, 6, 2, 3], [2, 3, 3, 2, 3, 2], [3, 5, 3, 5, 1, 2]]`	`17`

To make the test cases more interesting, the tester is programmed to produce fewer ones, fours, and sixes than a perfectly fair die would. Tactically placing the thumb on the scales occasionally helps the raw randomness hit hidden edge cases more effectively.

107. Optimal crag score

```
def optimal_crag_score(rolls):
```

An earlier problem asked you to compute the best possible score for a single roll in the dice game of crag. In this problem, the game spans multiple `rolls`, again aided by the same perfect foresight as in the previous "Lords of Midnight" problem. Given the entire sequence of `rolls`, this function should return the highest possible score that can be achieved with those `rolls` under the constraint that **the same category cannot be used more than once**, as is dictated by the rules of the actual game of crag, the same way as done in the more famous dice game of Yahtzee.

The greedy algorithm "sort the rolls in descending order of their maximum possible individual score, and then use each roll for its highest-scoring remaining category" does not work here, since several rolls might fit in the same category. Yet these rolls are not equally good choices for the remaining categories. Therefore, you need to process the list of rolls recursively, with each recursive level assigning an unused category to the current roll. For each such choice of category, recursively compute the best possible way to assign unused categories to the remaining rolls. The best category for the current roll is the one that maximizes the sum of the current roll's value and the recursively computed optimal solution for the remaining rolls.

To speed up this recursion, you should keep track of the best solution that you have found so far. Once the current branch of the recursive search no longer has a chance of beating that best known solution, you can treat that current branch as having no solutions whatsoever and turn right back. There is no point wasting perfectly good processor cycles computing the exact value of something that is already certain not to affect the final answer.

`rolls`	Expected result
`[(1, 6, 6), (2, 5, 6), (4, 5, 6), (2, 3, 5)]`	`101`
`[(3, 1, 2), (1, 4, 2)]`	`24`
`[(5, 1, 1), (3, 5, 2), (2, 3, 2), (4, 3, 6), (6, 4, 6), (4, 5, 2), (6, 4, 5)]`	`74`
`[(3, 1, 2), (1, 4, 2), (5, 2, 3), (5, 5, 3), (2, 6, 3), (1, 1, 1), (5, 2, 5)]`	`118`
`[(1, 5, 1), (5, 5, 6), (3, 2, 4), (4, 6, 1), (4, 4, 1), (3, 2, 4), (3, 4, 5), (1, 2, 2)]`	`33`

108. Painted into a corner

```
def cut_corners(points):
```

As in the earlier problem, in a set of points in the two-dimensional integer grid, a **corner** is three points of the exact form (x, y), $(x, y + h)$ and $(x + h, y)$ for some $h > 0$ for its **tip** and its two **wings**. Given a list of `points` sorted by x-coordinates, ties resolved by y-coordinates, this function should return the minimum number of points whose removal from the `points` makes all corners disappear. You must remove at least one point from each corner. However, whenever several corners share a common point, removing that shared point makes all those corners disappear! This function should therefore choose which points to remove judiciously, since the greedy approach is not always optimal.

This function should start by creating a list of all corners among the given `points`, each corner represented as a tuple `(i,j,k)` of the **position indices** of the `points` that form its tip and its wings. Then, call the recursive function that determines the minimum number of points to remove to get rid of all corners. At each level of recursion, this function should choose any remaining corner whose three points are still intact. It makes no difference which corner you choose, since you must remove at least one point from every corner anyway, either now or later. As a general rule in life, when you have no choice, it never hurts to grab the bull by the horns and, as the famous advertising slogan puts it, just do it! Try removing each of these three points separately in a for-loop. For each removed point, recursively examine the best way to eliminate the remaining corners in that scenario. Out of those three, use the best way that works.

Use sets and dictionaries to speed up decision-making at all levels of this recursion. For example, if some corner contains only one point shared with another corner, there is little point (heh) in branching the search for that corner, since removing the shared point can never result in a worse global outcome than removing either of the other two points.

`points`	Expected result
`[(2, 2), (2, 5), (6, 2)]`	`0`
`[(2, 2), (2, 5), (5, 2)]`	`1`
`[(1, 0), (1, 3), (1, 4), (2, 0), (2, 2), (2, 3), (4, 0), (5, 0)]`	`3`
`[(x, y) for x in range(5) for y in range(5)]`	`8`

109. Operation Grand Slam: Infinite Fibonacci word

```
def fibonacci_word(k):
```

Fibonacci words are strings defined analogously to Fibonacci numbers. The recursive definition starts with two base cases S_0 = `'0'` and S_1 = `'01'`, followed by the recursive rule $S_n = S_{n-1}S_{n-2}$ that concatenates the two previous Fibonacci words. See the Wikipedia page for more examples. Crucially for this problem, each Fibonacci word has length equal to the corresponding Fibonacci number. Even though we cannot actually generate the entire Fibonacci word S_∞ because this limit of this construction would be infinitely long, we can construct the character at any particular position k of S_∞ after only a finite number of steps by realizing that after generating some word S_n that is longer than k, every longer word S_m for $m > n$, and therefore also the infinite word S_∞, must start with the same prefix S_n and therefore contain that same character in the position k.

This function should compute the k:th character of the infinite Fibonacci word, with position counting again starting from zero. Since this is the last problem of this problem set, to symbolize your reach towards the infinite as you realize that you can simply ignore the arbitrary laws imposed by society of men as you, little starling, fly boldly higher to bid and make this grand slam, your function must be able to work for such monstrously large values of k that they make even one googol seem like you could wait it out standing on your head. Of course, our universe would not have enough atoms to encode S_n for such a large n that you could look up its k:th character. Instead, use the recursive formula and the list of Fibonacci numbers that you will dynamically expand as needed, and ride the self-similar fractal nature of the infinite Fibonacci word through the finish line.

k	Expected result
`0`	`'0'`
`1`	`'1'`
`10`	`'0'`
`10**6`	`'0'`
`10**100 + 1`	`'1'`
`10**10000`	`'0'`
`1234**5678`	`'0'`

Bonus problem 110: Reverse the Rule 110

```
# for Suzanne: I think that I finally got it now
def reverse_110(current):
```

In elementary cellular automata, Rule 110 stands out as the only rule proven to be computationally universal, along with its mirror-image counterpart, Rule 124. Teetering at the borderline between chaos and order, where the really interesting patterns can temporarily exist, repeated iterations of this Turing-complete rule produce emergent fractal patterns that could, given enough time and memory, encode all possible computations.

The function to iterate an arbitrary elementary cellular automaton would have made an excellent problem earlier in this collection. But we didn't come all the way up here for something that trivial, oh no. Instead, this function must effectively **operate in reverse** by computing the `previous` state from which Rule 110 would produce the `current` state when applied one step forward! As much as we would like to turn back time, we can only go forward with the arrow of time. Simulating the reversal of even an otherwise simple deterministic process is usually exponentially harder than simulating it forward.

To ensure every position has both left and right neighbours, the left and right edges of each state wrap around cyclically. To make the returned answers unambiguous for automated testing, this function should return the **lexicographically smallest** of the `previous` states that Rule 110 takes to the same `current` state. For any `current` states that are Gardens of Eden, that is, impossible to produce from any `previous` state with Rule 110, this function should return `None`.

Straightforward computations can suddenly become complicated when you work backward from the expected results to the arguments that would produce them in the forward direction. Solve this problem with a separate recursive backtracking utility function that fills in the previous state from beginning to end. The body of the recursion should first check for conflicts between the `previous` and `current` states, and then try appending each of the two values 0 and 1 one at a time, recursively filling in the rest of the `previous` state for each value e.

`current`	Expected result
`[1, 0, 1, 1]`	`[1, 0, 0, 1]`
`[0, 0, 1, 1, 0]`	`[0, 0, 0, 1, 0]`
`[0, 1, 0, 1, 0, 1, 1, 1]`	`None`
`[1, 0, 1, 1, 0, 0, 0, 0]`	`[1, 0, 0, 1, 1, 1, 1, 1]`

Bonus problem 111: Aye, eye, I

```
# for Liz: Every day with you is a Wednesday
def wordomino(state, words):
```

Your nemesis, unceremoniously de-platformed in an earlier problem, has crawled her way out of Scylla's janitorial end in the style of *The Shawshank Redemption*. Having learned her life lesson about certain dishes best served with tea and a cookie, your nemesis will now cleverly engage you in a friendly combinatorial game of "word dominoes" that this author made up just for this problem.

Two players take turns, constrained by a shared list of legal four-letter English `words`. The `game state` so far is a string of letters. On her turn, each player must extend the current `state` with one letter from *a* to *z* so that the block of the last **four** characters of this new extended state is one of the legal `words`. However, to guarantee the eventual termination of this match, the block of the last **three** characters of this extended state may not previously appear as that block anywhere in the current `state`. The player unable to extend the `state` on her turn will fall into the formless void that separates this course from its successor, CCPS 209 *Computer Science II*.

As in the earlier combinatorial game of subtracting a square, a state is **hot** (**winning**) if and only if at least one legal move from that `state` leads to a **cold** (**losing**) successor state. Your function should therefore recursively evaluate the successor states that can be reached from the current `state` by extending it with a single letter. For a cold `state`, this function should return `None`. For a hot `state`, return its alphabetically lowest winning move that leads to a cold successor state and gives your opponent no chance to turn the tide.

`state`	Expected result (using the four-letter words extracted from `words_sorted.txt`)
`'perp'`	`None`
`'dunes'`	`'e'`
`'tropens'`	`'e'`
`'omethangairedeaditalitelaneten'`	`None`
`'damahabetashagingairana'`	`'l'`

With 7186 four-letter words in `words_sorted.txt` to choose from, the branches of the minimax search tree for this game can randomly get pretty deep. Fortunately, most of these moves tend to be pretty much forced (well, at least semi-forced). This keeps the effective branching factor down, which keeps the total running time of the minimax search tolerable.

Bonus problem 112: Count domino tilings

```
# for Tom: Systematic signals rein in unimaginable possibilities
def domino_tile(rows):
```

In the domino tiling problem, a room of contiguous unit squares is tessellated with blank 2-by-1 domino tiles, each placed either vertically or horizontally. Of course, this task is possible only if the room has an even area. This necessary condition alone does not guarantee a tiling, as the famous mutilated chessboard problem shows. In our version of domino tiling, the room consists of vertically stacked rows that share a common straight left wall. The list of `rows` contains the widths of the individual rows, as measured in squares. This formulation covers not only the usual rectangular-room version of this problem, but also the more general case of Young diagrams.

This function should compute the number of ways that the room described by the given `rows` can be tiled with dominoes. As with all combinatorial problems of this spirit, the solution requires careful consideration analogous to the Serenity Prayer; since you have no choice about covering every square of the room, you might as well grab the bull by the horns and recursively fill that room in order so that each level of recursion quickly finds the first uncovered square. There, recursion branches in two directions to count the ways to finish filling after placing a horizontal or vertical tile on that square, depending on what your previously placed dominoes still allow. Since this recursion repeatedly solves the same subproblems, you need some cha-ching scheme to rein in the **combinatorial explosion**. Once you get the logic working correctly, optimize everything that you possibly can.

`rows`	Expected result
`[2, 2]`	`2`
`[4, 3, 2, 1]`	`0`
`[5, 3, 4, 2]`	`7`
`[8, 8, 8, 8, 8, 9, 9]`	`1895245`
`[12, 12, 11, 11, 11, 11, 11, 11]`	`13571985717`
`[6] * 100`	`811112249237963267431468071118074884556482701215062538819292860703450 9`

The number of ways to tile the given room generally grows exponentially with the room's size. Some of these tilings appear more aesthetically pleasing because they exhibit symmetries or other visually pleasing forms. Enumerating all tilings under additional aesthetic constraints may either be easier or harder than enumerating the unconstrained tilings. For a good example of this phenomenon, see the Project Euler problem "Tatami-free rooms".

Bonus problem 113: Invaders must die

```
# for Ezzat: Intuition beats book learning when the stakes get high
def laser_aliens(n, aliens):
```

Having escaped from the icy *n*-by-*n* chessboard in the earlier problem "In space, no one can hear you bounce", Bishop has called in reinforcements to eliminate the scourge of aliens from this system once and for all. After arriving, space marines take positions along the edges of this chessboard. Each marine wields a laser cannon that can blast a single shot through either a row or a column, eradicating every alien in that row or column. Given the positions of `aliens` on this board as a list of tuples `(row, col)`, sorted by rows and breaking ties by columns, this function should compute and return the minimum number of space marines needed to slap every alien right off the map.

The key to solving this kind of combinatorial search and optimization problem is accepting what you can't change and having the courage to exploit what you can choose. The constraints you can't change serve as the backbone that guides your search, instead of you pointlessly branching at every turn like a headless chicken exploring an exponential number of redundant possibilities. For any row that still contains aliens, you have two choices: shoot a laser beam horizontally through that row, or shoot a laser beam vertically through **every** column that still contains an alien in that row. This unavoidable constraint gives you a nice two-way choice for each row. Furthermore, when shooting through those columns, clever updates and downgrades to auxiliary data structures maintained throughout the recursion to quickly tell you the positions of aliens in the given rows or columns will massively reduce branching in future rows with `aliens` in those same columns. You should also have the courage to make meaningful choices about the order that you process the individual rows…

n	aliens	Expected result
3	`[(0, 1), (0, 2)]`	1
4	`[(0, 1), (0, 2), (1, 0), (1, 1), (1, 2)]`	2
5	`[(0, 3), (3, 2), (3, 3), (4, 2)]`	2
8	`[(1, 4), (3, 6), (4, 6), (4, 7), (5, 0), (5, 4), (5, 6), (5, 7), (7, 0), (7, 7)]`	4
10**6	`[(42, 42), (999999, 999999)]`	2

Bonus problem 114: Stepping stones

```
# for Suzy: La dureté de deux diamants nous libérera aussi
def stepping_stones(n, ones):
```

This problem is adapted from the Numberphile video, "Stones on an Infinite Chessboard", which the reader should first watch to learn how this excellent puzzle works. To keep this problem manageable, we restrict stone placement to a finite *n*-by-*n* chessboard, and stones cannot cross the edges into the void. As usual, rows and columns are numbered starting from zero.

Starting with stone 2, place stones in ascending order so that, when placing stone k, the stones previously placed in its up to 8 neighbouring locations sum to exactly *k*. Given the board size `n` and the initial locations of the brown `ones`, this function should return the highest numbered stone in the longest sequence of stones that can be placed consecutively on the board within the rules of this puzzle. For example, as Neil Sloane shows in the video above, you can snake your way up to stone 16 from two brown stones placed diagonally with one space between them, provided the board is large enough to contain the resulting pattern. However, no matter how you twist and turn, no placement of stones will get you any further than that.

To speed up this search, you should maintain a list of sets whose *k*:th element is a set of positions whose current sum of neighbouring stones equals *k*. When placing the stone number *k* on the board, your function can quickly loop only through the positions in that set, rather than examining the entire board each time. Placing a new stone updates the sums of the currently empty cells in its neighbourhood, moving these cells from one set to another, then back to the original set once the recursive search to place the sequence of stones starting with stone *k* + 1 returns.

n	ones	Expected result
4	`[(0, 0), (3, 3)]`	1
5	`[(0, 1), (1, 1), (2, 2)]`	10
6	`[(2, 0), (5, 3), (1, 3), (0, 0)]`	19
7	`[(4, 4), (1, 2), (6, 5), (1, 5), (4, 2), (1, 4), (4, 5)]`	27
8	`[(4, 4), (1, 3), (1, 2), (1, 1), (4, 6), (1, 4), (1, 0), (4, 1)]`	29

Bonus problem 115: Ex iudiciis, lux

```
# for Joanne: Light heart, light touch
def illuminate_all(lights):
```

Each light in the given row has a brightness that determines how many adjacent positions to the left and right it illuminates, in addition to illuminating its own position. (A light with zero brightness illuminates only itself and doesn't shine on its neighbours.) Your task is to turn on as few individual lights as possible so that every position of the entire row is illuminated by at least one light. Since the smallest working subset of lights is not necessarily unique, your function should only return the number of lights needed to illuminate the entire row.

To organize this search into a workable recursion, start by noting that, same as in the **subset sum** recursion example seen in the class, you have a basic "take it or leave it" two-way choice for the last light in the list. You can explore both branches of this choice recursively and use the branch that produces a better solution. You can also discard many individual lights immediately, since the optimal solution will never need them to achieve anything it could just as well achieve without them.

Your recursion should have a second parameter to keep track of how much light is still "owed" to the non-illuminated positions skipped from the right, or how much extra light is spilling into the positions in the list from the lit-up positions skipped from the right. Since the recursion will keep repeating the same subproblems in exponentially many different ways, use some `@lru_cache` magic to eliminate the exponential blowup of running time.

`lights`	Expected result
`[2, 3, 3, 2]`	`1`
`[1, 0, 1, 0]`	`2`
`[0, 0, 0, 1, 2, 0, 2, 0, 0]`	`4`
`[0, 0, 2, 3, 4, 1, 0, 1, 2, 2, 5, 2, 3, 0, 1, 0, 0]`	`3`
`[0, 2, 2, 2, 2, 0, 1, 2, 0, 0, 1, 1, 1, 0, 1, 0, 0, 2, 1, 1, 0, 1, 1, 2, 0, 0, 1]`	`8`

Bonus problem 116: Flatland golf

```
# for Sharon: First there is hair, then there is no hair, then there is
def best_clubs(holes):
```

The denizens of Flatland want to play a friendly game of golf. Since their reality has no room for fancy golf resorts with spectacular views, Flatland golf courses are one-dimensional lines, with the hole located some distance from the teeing point at the origin. Each Flatland linksman can carry only two clubs. In the Flatland physics version of golf, the ball will always travel the same distance when swung with the same club. Each club is therefore rated for the fixed distance it will always carry the ball.

Since one-dimensional golf courses don't take much space, you can hit the ball as far as you like, then turn back on the next stroke. For example, with two clubs rated at 40 and 110 meters, you can reach a hole 210 meters away in six strokes: three with each club. In general, you can reach a hole at distance d using clubs c_1 and c_2 if and only if d is divisible by gcd(c_1, c_2).

This function should choose the two clubs in the bag to minimize the total score over the entire course, given the distances to each hole. Since this optimal pair of clubs may not be unique, this function should compute the smallest achievable total strokes for the given `holes`. Discrete math and number theory, especially linear Diophantine equations, will help a lot with these computations. Still, the test cases should run within the time limit with brute force, assuming a bit of cleverness.

`holes`	Expected result	(best clubs)
`[2, 3, 4, 5]`	`6`	`5, 2`
`[2, 6, 7, 9]`	`7`	`9, 2`
`[2, 5, 7, 10, 14]`	`8`	`7, 5`
`[19, 42, 51, 65, 85, 85, 104]`	`33`	`19, 14`
`[300, 250, 200, 325, 275, 350, 225, 375, 400]`	`26`	`125, 100`

This problem comes from the collection "536 Puzzles and Curious Problems" by Henry Ernest Dudeney and Martin Gardner, two historical heavyweights of recreational mathematics. Written in a simpler time when such calculations were performed by hand (even this very problem was titled "Queer Golf", heh), the entire puzzle was just the last row of the above table. Since we now have enough computational power under our thumbs to easily crank out more computations than the entire pre-computing world executed throughout its history, we can mechanize the logic for arbitrary golf courses instead of just this particularly tricky one.

Bonus problem 117: TextmirrororrimtxeT

```
# for Anita: All the stars say go
def both_ways(text):
```

This function should find the longest substring that appears in the `text` in both its original and reversed forms. These original and reversed occurrences may not overlap each other at any position. For example, the longest such substring of the title of this problem is `'Textmirro'`. Since the longest substring of `text` that fulfills this requirement is not necessarily unique, such as in the string `'zabbazuuza'` that allows three equally good solutions `'za'`, `'ab'`, and `'zu'` (and their three reversals), this function should return the length of that maximal substring.

Despite the apparent simplicity of this problem, especially this deep in the problem collection, how best to preprocess the text to tighten the search for the maximal substring should make this exercise worth the price of admission. Your function should run speedily regardless of how many (or few) different characters the `text` contains. The reader is also sternly advised not to be a Shlemiel by looping over the same positions repeatedly when searching for the substring and its reversal. The automated tester will produce test cases sufficient to reveal such dereliction of duty and a lack of proper care.

text	Expected result
`''`	`0`
`'bigchungus'`	`1`
`'ouch'`	`0`
`'bbbbcaacbc'`	`3`
`'ddbdabcbdcacbdcdbcbadd'`	`7`
`'howsutnaqhitixtglmwvzdceqzdwzqcteuxzsigecjqejnjasriyldiwxfotyzefvoibrfliwtpisrguetcqzwdzqecdzvwmlgtxitihqantuswz'`	`32`
`'axa' * 1000`	`1500`
`'q' * 10**6`	`500000`

This problem comes from Jeff Erickson's collection of old homework and exam problems for various courses on algorithms and algorithmic thinking that help solve problems similar to this one.

Bonus problem 118: Hy-phen-a-tion by com-put-er

```
# for Donna: A surprising squeeze play brings home a surprise slam
def liang_hyphenation(word, patterns):
```

Frank Liang's 1983 Ph.D. thesis presented an ingenious algorithm to hyphenate English words with amazing accuracy, without needing any knowledge of each word's semantics or pronunciation. This algorithm was later incorporated into Donald Knuth's classic TeX typesetting system, the gold standard for scientific publishing in computer science and mathematics to this day.

Liang's hyphenation algorithm uses a list of precomputed `patterns` consisting of lowercase letters interspersed with integer digits. To hyphenate a `word`, first find all its substrings that match some pattern, temporarily ignoring the digit characters in the pattern during matching. For example, the pattern `'um2p'` matches a substring in the word `'pumper'`. The digits in the pattern indicate the positions they precede in the match. For example, this particular match gives the position of `'pumper'` that contains the second `p` the value `2`. If several patterns match inside the same `word`, the value for each position is the **maximum** of the values given by these patterns. Special patterns such as `'.aster5'` and `'2f3ic.'` that begin or end with the period character match only at the beginning or at the end of the word, respectively.

To hyphenate the `word` after all this, simply place the hyphenation dashes before the positions that have an odd value! Of course, you never place a hyphenation dash before or after the word itself, since that would be silly. The 4447 original `patterns` in the public domain are hardcoded in our tester script, so you don't need to look for them online. Since these `patterns` never change during the test, your function should build a global data structure the first time it is called to speed up matching these patterns to the substrings of later words.

word	Expected result (using Liang's original patterns)
`'quartermastership'`	`'quar-ter-mas-ter-ship'`
`'ceremony'`	`'cer-e-mo-ny'`
`'leptomonad'`	`'lep-tomon-ad'`
`'pumper'`	`'pumper'`
`'asteroid'`	`'as-ter-oid'`
`'steroid'`	`'s-teroid'`
`'terrific'`	`'ter-rif-ic'`
`'milhouse'`	`'mil-house'`

Bonus problem 119: Jealous vs. zealous

```
# for Vicky: Bought by judgment of the eye
def othello_moves(othello, desdemona):
```

The classic complete information board game of Reversi has fallen out of fashion in recent years. However, it still has enough complexity to serve as a cute species of *Drosophila* for every budding student of game-playing algorithms and their heuristics in artificial intelligence. The rules are simple enough but too verbose to explain on this page, so we refer the reader to the linked Wikipedia page for details. In this first Python programming course, we will focus on writing a move generator that finds all available moves for Othello against Desdemona on the given board.

The game pieces of `othello` and `desdemona` on the board are listed as two-tuples of their *x*- and *y*-coordinates, both ranging from 0 to 7. This function should return a list of all possible moves available to Othello. Encode the returned moves as three-tuples `(x, y, flips)`, where `x` and `y` are the move coordinates, and `flips` is the number of Desdemona's pieces that the move would flip to Othello's. To make the expected answer unique, sort the returned list of moves in **descending** order of `flips`. Moves that flip the same number of pieces should be listed in **ascending** order of their *x*-coordinates, breaking ties by the *y*-coordinate also in ascending order.

`othello`	`desdemona`	Expected result
`[(3, 3), (4, 4)]`	`[(3, 4), (4, 3), (2, 3)]`	`[(1, 3, 1), (2, 4, 1), (3, 5, 1), (4, 2, 1), (5, 3, 1)]`
`[(3, 3), (4, 4), (2, 3), (2, 5)]`	`[(3, 4), (4, 3), (2, 4)]`	`[(1, 4, 2), (5, 2, 2), (1, 5, 1), (3, 5, 1), (4, 2, 1), (4, 5, 1), (5, 3, 1)]`
`[(3, 3), (4, 4), (2, 3), (1, 3), (5, 2), (0, 3), (1, 5)]`	`[(3, 4), (4, 3), (4, 2), (3, 5), (2, 5), (1, 4), (2, 2), (2, 4)]`	`[(3, 6, 4), (0, 4, 3), (1, 6, 3), (2, 6, 3), (4, 5, 3), (4, 1, 2), (4, 6, 2), (0, 5, 1), (1, 1, 1), (2, 1, 1), (3, 1, 1), (3, 2, 1), (5, 1, 1), (5, 3, 1)]`

Bonus problem 120: Count the permorsetations

```
# for Bianca: Staying above the Depeche Mode line
def count_morse(message, letters):
```

Everyone has surely heard of Morse code, the classic encoding of text into binary symbols canonically denoted with dots and dashes. However, this coding is technically ternary since the pause between two letters is really a separate symbol, analogous to how whitespace characters between the words you are now reading are characters just the same as those with visible glyphs. Without the pauses separating the individual codewords, the same `message` could theoretically be decoded in exponentially many different ways, as illustrated in our example script `morse.py`.

Your function receives the `message` without pauses along with the `letters` whose encoding originally produced it. Your function should count how many different permutations of those `letters` produce that same `message`. To rein in the potentially exponential blowup of possibilities in worst-case scenarios such as `'.....................'`, each character is guaranteed to appear in the `letters` at most once. Recursion is once again your good and faithful servant, aided by a tasty serving of some `@lru_cache` magic to prevent your recursion from redundantly exploring identical subproblems multiple times.

message	letters	Expected result
`'-------.'`	`'omg'`	2
`'.-.----..-.....-.-...-...-'`	`'etaoinshrdlu'`	122
`'-..----.....-..-.'`	`'xtmisuf'`	4
`'..------.-..-.-...-..-..-..-..'`	`'vocalnedxi'`	15
`'.-...-.-...-.....-..-.---.-.--...--...-..--'`	`'rlbzsvxagyiunfw'`	1

Samuel Morse obviously aimed to assign shorter codewords to frequent letters. For example, the letters E and T get encoded as a single dot and a single dash. From our comfortable peanut gallery up here in the 21st century, we cannot fault him for being unaware of the actual relative frequency distribution of letters in English text, or of Huffman codes and other twentieth-century algorithmic advances in coding theory that squeeze all air out of encoded messages.

Bonus problem 121: Count your lucky sevens

```
# for Laura: Open or closed, two pasts, one present
def count_sevens(n):
```

The earlier problem “Up for the count” introduced the Champernowne word, an infinite sequence of digits starting 12345678910111213141516171819202 1…, formed by writing consecutive positive integers together without separators. That problem asked you to determine the individual digit at a given position. However, this deceptively simple self-similar fractal word gives rise to other equally interesting problems.

This problem asks you to count how many times the lucky digit seven appears in the prefix of the Champernowne word that contains the integers from one to *n*, inclusive. Of course, the automated tester will again give your function values of *n* large enough that patiently tallying the sevens one at a time will not reach the final answer before our dying sun eventually engulfs the Earth and prematurely terminates this computation. Some elementary combinatorics is therefore necessary to add up the occurrences of sevens over exponential swaths of consecutive integers in a single swoop.

To get started, we realize that of the 10^k possible sequences of *k* digits (leading zeros allowed), precisely `Binomial[k, i]*9^(k-i)` of those contain exactly *i* sevens, as expressed in the syntax of *Mathematica*. (The other *k* – *i* digits can each be any of the nine other digits, independently of each other.) As nasty as the formula `Sum[i*Binomial[k, i]*9^(k-i),{i,1,k}]` might initially appear, Mathematica fortunately confirms that it simplifies to $10^{k-1}k$. Use that as you wish.

n	Expected result
4	0
8	1
78	17
776	231
778	236
10**50	50000000000000000000000000000000000000000000000000
123**456	A behemoth with 955 digits, the first ten of which are 9458651505, and the last ten are 0200733990

Bonus problem 122: Forks on the Gin Lane

```
# for Jenna: When going the wrong way, turn back sooner rather than later
def count_deadwood(hand):
```

The card game of gin rummy is not as popular as it was in the nostalgic Technicolour past. Still, at least in our imaginations, we can always pretend to be the suave James Bond playing this game against the dastardly Auric Goldfinger. As with other popular incomplete-information card games, creating a gin player that could beat, if not Stu Ungar, at least that one salty old guy at the local VFW outpost would be a nice problem for a course in artificial intelligence and machine learning.

In this introductory Python course, we shall again content ourselves with writing a function to tally the deadwood points left in a given hand of ten cards sorted by rank, assuming the cards are arranged into sets and runs optimally. As explained on the Wikipedia page linked above, a run consists of three or more cards of consecutive ranks in the same suit (aces are always low, never high), whereas a set consists of three or four cards of equal rank.

The recursive solution processes the cards one at a time, deciding for each card whether the best result comes from resigning it as deadwood, starting a new set or run, or joining an existing set or run. Depending on this choice, arrange the remaining cards recursively in the best manner, and see which of these five choices for the current card gives the overall best result. Some preprocessing of the hand will eliminate many branches as guaranteed dead ends, massively pruning the otherwise exponentially large search tree.

hand	Expected result
`[('ace', 'diamonds'), ('ace', 'hearts'), ('ace', 'spades'), ('two', 'diamonds'), ('two', 'spades'), ('three', 'diamonds'), ('four', 'clubs'), ('four', 'diamonds'), ('four', 'hearts'), ('four', 'spades')]`	`2`
`[('eight', 'diamonds'), ('eight', 'spades'), ('nine', 'clubs'), ('nine', 'diamonds'), ('nine', 'spades'), ('ten', 'clubs'), ('ten', 'diamonds'), ('ten', 'spades'), ('jack', 'clubs'), ('jack', 'spades')]`	`0`
`[('two', 'hearts'), ('three', 'clubs'), ('three', 'spades'), ('six', 'spades'), ('seven', 'diamonds'), ('seven', 'spades'), ('eight', 'hearts'), ('eight', 'spades'), ('nine', 'hearts'), ('jack', 'clubs')]`	`42`

Bonus problem 123: One, two, three, grow old with me

```
# for Danielle: Two strings of a lute, forever apart
def addition_chain(n, brauer=False):
```

An addition chain is a **strictly increasing** list of positive integers starting with 1, where each element is the sum of two numbers from the preceding elements. For example, the addition chain `[1, 2, 4, 6, 10, 20]` of length 6 ends at 20. Every addition chain of length two or more must begin with `[1, 2]`, but from three onward, each positive integer may or may not be included, depending on whether it is used later. The two numbers that add up to the current number don't have to be distinct, as in the example chain: 2 + 2 = 4 and 10 + 10 = 20.

Your function should find the shortest addition chain that ends with the given `n`. Since this shortest chain is not necessarily unique (for example, both `[1, 2, 3, 6]` and `[1, 2, 4, 6]` have the same length), your function needs to return only the length of this shortest chain… minus one, since customarily the leading one of the chain is not included in its length. Two is the new one.

Additionally (heh), a chain is a **Brauer chain** if each element is the sum of its **immediate** predecessor and some other element in the chain. For example, `[1, 2, 3, 6, 7, 12]` is not a Brauer chain, since it does not contain a 5 that could be added to 7 to give 12. If the parameter `brauer` is set to `True`, this function must return the length of the shortest Brauer chain. Once you have the general addition chain function working, a few small modifications should do the trick.

This ridiculously simple-to-state yet incredibly hard problem to solve, honestly, without a precomputed lookup table, fits the theme of this problem collection, since no efficient algorithms have been discovered for it. Every algorithm to find the shortest addition chain will need either a lot of time or a lot of memory to confirm the correct answer for some difficult values of `n`. That said, sometimes you have to ask people to do the impossible. (This is not Mission Difficult, Mr. Hunt.) Your humble instructor had to set the automated tester's timeout to an absurdly high value to let these test cases run to completion and emit the checksums for the expected correct answers. Twenty seconds. Tick, tock. If, else. If, else. If. *Else*.

Of course, the reader can cheat by hardcoding a sufficient prefix of the correct-answer sequence inside their function to pass all the test cases, giving their impenetrable black box the flawless external appearance of actually doing what it was supposed to do. On the other hand, lookup tables have their merits. Some people don't see them as manly enough, but in the end, the outcome is the same, even if you fast-forward to get there. Rationalize the if-else branches of your timeline however you wish. Even if everything ends up the same when we all lose our charms, the journey in between is what really matters.